Thesis Plan

Decentralized Multi-Agent Reinforcement Learning for Power Grid Topology Control

Corentin Plumet

August 9, 2026

Contents

Chapter 1

Thesis Plan

1.1 Working Title

Graph Neural Networks for Decentralized Multi-Agent Reinforcement Learning in Power-Grid Topology Control

1.2 Central Thesis Question

How can graph neural networks help decentralized reinforcement learning agents learn reliable power-grid topology control policies that preserve survival performance while scaling to combinatorial action spaces?

Three terms fix what the thesis has to measure:

- **Reliable** is episodic survival on held-out chronics, always reported from the checkpoint with the best test survival and re-evaluated on the full test split.
- **Decentralized** means each agent owns a fixed group of substations, observes a local subgraph, and acts on its own discrete topology actions without having any information about the other agent actions; only the critic is centralized.
- **Scaling** means larger grids, which a graph encoder addresses because its parameters are independent of grid size.

1.3 Research Questions

1. **Can a graph actor replace a flat one without paying for it?** A decentralized MAPPO agent has first to be trained to stable, high survival on `bus14`; only then is it meaningful to swap the flat MLP actor for a graph encoder. An MLP's input width is tied to one grid, so nothing transfers; a graph encoder is size-independent, but that is worth nothing if it does not match the tuned MLP first. (Chapter 3.)
2. **How should the grid be represented as a graph for a local actor?** Node granularity (busbars, or busbars with separate generator, load and line nodes), which relations exist and what they carry, the optional substation-level augmentations, and the graph readout are all free choices. (Chapter 4 defines them; Chapter 5 measures which ones move survival.)
3. **What is a local agent entitled to observe, and does enforcing that boundary matter?** A local graph carries contextual nodes from neighboring substations. Making a contextual node visible only while a live line connects it to the agent's own region, and choosing whether pooling covers the whole visible graph or only the agent's controlled domain, changes what the policy conditions on. The distinction is between context and leakage. (Sections 4.2.5 and 4.4.)

4. **Can the agents intervene sparsely without losing survival?** Evaluation-time rho overrides, an intervention-gated actor, fixed non-idle penalties, and adaptive Lagrangian intervention budgets are evaluated on two axes at once, survival and intervention rate, since improving either alone is not a result. (Chapter 6.)
5. **Does any of this transfer to a larger grid?** Moving from `bus14` to the 36-substation WCCI grid tests whether a graph encoder is portable in practice and not only in parameter shape, and what has to be corrected: mainly feature scaling and the size of the action space. (Chapter 7.)

1.4 Expected Contributions

- **A tuned decentralized MAPPO baseline on `bus14`, and evidence that a graph actor matches it.** The original framework trained unstably; the training and initialization changes that fix it are isolated one at a time, so the GINE actor is compared against a strong MLP rather than a weak one.
- **The local actor graph, specified and then screened.** Three representations sharing a fixed candidate graph with a dynamic edge mask, so tensor shapes stay constant while the topology varies, each with an explicit information boundary — the three are *not* information-equivalent, so comparing them is never a pure comparison of architectures. Encoder depth and width, message-passing operator, readout, and the structural augmentations of the busbar graph are then screened as balanced full factorials, three seeds per cell, each scored by full-test evaluation of the best checkpoint.
- **A size invariant mechanism for combinatorial action spaces.** A candidate-action head that scores each action from the encoded nodes it modifies, replacing the fixed-width layer over an action index
- **An ablation of sparse intervention mechanisms** reported jointly on survival and intervention frequency, connecting the learned behavior to what a control room would accept: act rarely, act locally, act when the grid is unsafe.
- **A measurement of what actually breaks under transfer.** Encoder parameters cross grids unchanged; the input distribution they were calibrated on does not. Two findings generalize beyond power grids: per-feature standardization is harmful when a channel is a mixture dominated by structural zeros, and for gauge-dependent quantities such as voltage angle, changing the representation beats any choice of normalization.

1.5 Open Decisions

- Confirm the final thesis title and whether to emphasize GNNs, sparse-control, or decentralized MARL most strongly.
- Decide which experiment families are final and which are only exploratory.
- Decide whether the main claim is performance improvement, improved sparsity at equal survival, or reproducible ablation insight.
- Choose the final bibliography style required by the university.

Chapter 2

Introduction

2.1 Topology control

A transmission grid is not a fixed graph. Inside a substation, every connected element, each transmission line, generator, and load, is attached to one of several busbars, and an operator can move an element from one busbar to another or split a substation into two electrically separate nodes. Doing so changes which paths the power actually flows through, and therefore redistributes loading across the network. This is topology control, also called busbar switching or network reconfiguration.

Its appeal is economic. Congestion is classically relieved by redispatching generation or by curtailing renewable production, both of which cost money at every intervention, or by building new lines, which costs a great deal once and takes years. Topology control uses assets that already exist and switching equipment that is already installed, so its marginal cost is close to zero [1]. The energy transition makes this attractive rather than merely elegant: variable renewable generation makes flows less predictable and pushes existing corridors closer to their thermal limits, while grid expansion is slow [2], [3].

The catch is that topology is hard to exploit. Operators use it, but mostly through a small repertoire of remedial actions established by experience and offline study, so a large part of the available flexibility is never searched [3].

2.2 Why it has to be automated

Three properties make manual search impractical.

The action space is combinatorial. A substation with d connected elements and two busbars admits 2^d assignments, and the number of joint configurations of a grid is the product over its substations. Even the small IEEE 14-bus benchmark used throughout this thesis exposes 178 distinct unitary topology actions; the 36-substation grid of Chapter 7 exposes more than 66 000, which is why it has to be reduced offline before an agent can be trained on it at all (Section 4.6).

The effect of an action cannot be read off the grid. Power flows are governed by non-linear AC equations, so the consequence of moving one line onto another busbar is a global redistribution, not a local change. There is no reliable shortcut for ranking actions other than simulating them.

Finally, the decision is sequential and time-constrained. An action taken now changes which contingencies are survivable later, cooldown periods forbid acting on the same substation twice in quick succession, and an operator has minutes, not hours. Optimizing a single step greedily is not the same problem as keeping a grid alive for a day.

These are the conditions under which reinforcement learning is a reasonable candidate: a sequential decision problem, a simulator that is expensive but available, no analytic optimum, and a clear success

signal — the grid does not black out. The Learning to Run a Power Network (L2RPN) competition series was created by RTE to make exactly this problem tractable for machine-learning research, and the Grid2Op simulator provides the environment, the chronics of load and generation, and the operational rules that go with it [2], [4].

2.3 Why decentralized agents

Most published work treats the grid as a single agent controlling everything. Real grids are not operated that way. Transmission system operators divide the network into regions, each with its own control centre and its own operators, who act on their own substations with a detailed view of their own area and a much coarser view of the rest [3]. A decentralized formulation is therefore closer to how the job is actually done, and to how any automated assistant would eventually be deployed, as support inside one control room, not as a single controller over an entire interconnection.

It is also the formulation that scales. A monolithic controller has to represent the joint action space, which grows multiplicatively with the number of substations. Partitioning the grid into regions and giving each region its own policy replaces that product by a sum: each agent chooses among its own local actions only. This is the classical argument for decentralized multi-agent reinforcement learning, and it comes with the classical cost [5]. Because every agent learns while the others are also changing, each one faces an environment that is non-stationary from its own point of view, and because each observes only part of the grid, agents can take individually sensible actions that are jointly harmful.

The usual answer is *centralized training with decentralized execution*: a critic is allowed to see the global state during training, while each actor conditions only on what it will have access to at deployment time [6], [7]. This thesis uses that setting throughout.

2.4 Reinforcement learning for topology control

Work on RL for topology control falls into four groups, in roughly historical order. A broader survey of graph-based RL for power systems is given by Hassouna et al. [8].

Single-agent policies. The first approaches train one agent to control the whole grid. Subramanian et al. [9] showed that a plain deep RL agent acting on a curated set of topology actions already outperforms a do-nothing policy on L2RPN benchmarks. The strongest example is SMAAC [10], which won the L2RPN WCCI 2020 challenge with an actor-critic method over *afterstates* — the grid configuration that results from an action, evaluated before the physics resolves it — combined with a hierarchical decomposition of where to act and how. The common obstacle in this group is the action space: it has to be reduced, curated, or factorized before learning becomes feasible at all.

Learned policies wrapped in heuristics. A second group accepts that a learned policy should not be in charge at every step, and surrounds it with rules. The dominant pattern is an activation threshold: the agent is queried only when the grid is stressed, measured by the maximum line loading $\rho_{\max}$, and does nothing otherwise. PowRL added a heuristic overload manager and a reduced action set on top of an RL policy and topped the L2RPN NeurIPS 2020 robustness track [11]. Lehna et al. [12] compared advanced rule-based agents against RL agents directly and found comparable survival, with the RL agent far cheaper at inference — a useful reminder that beating a well-engineered heuristic is not automatic. Heuristics of this kind are now standard practice rather than a baseline, and this thesis uses one.

Hierarchical and centrally coordinated multi-agent methods. A third group decomposes the decision but keeps a coordinator. Manczak et al. [13] split control into choosing whether to act, choosing where, and choosing which local configuration to apply. van der Sar et al. [14] assign a low-level agent to each substation under a higher-level agent that selects which substation acts. de Mol et al. [15] make the coordination explicit: regional agents propose actions and a central agent selects among the proposals.

These methods recover much of the scalability of factorization, but they retain a component that sees the whole grid and decides for everyone, so they do not answer whether purely local policies suffice.

Decentralized multi-agent control. The remaining question — whether agents that act on local observations, without a coordinator and without communication, can operate a grid — is much less explored, and it is where this thesis sits. The gap is recognized: the MARL2Grid-TR benchmark was built specifically because prior work "focused on single-agent settings, neglecting the often decentralized, multi-agent nature of grid control" [16].

Graph representations. Cutting across all four groups is the question of how the grid is given to the network. A flat vector ties the policy to one grid size and discards the topology entirely. Graph neural networks are the natural alternative [17], [18], [19], and their parameters are independent of the number of substations, which is what makes transfer between grids conceivable at all. The choice of representation is not neutral, however: de Jong et al. [20] show that the standard homogeneous graph suffers from *busbar information asymmetry* — it encodes the connections that currently exist but not the ones an action could create — and that a heterogeneous representation resolves it. That observation is close to the subject of Chapters 4 and 5.

2.5 Starting point: MARL2Grid and MAPPO

This thesis builds on MARL2Grid, the multi-agent benchmark and codebase of Marchesini et al. [16], itself the multi-agent successor of RL2Grid [21]. It wraps Grid2Op in a multi-agent interface and supplies the decomposition, the observation and action spaces, the reward, and the training loop that the rest of this work modifies.

Decomposition. The grid is partitioned into disjoint regions, one per agent, fixed in advance. On the IEEE 14-bus environment (`bus14`, 14 substations and 20 lines) three agents own substations $\{0, 1, 2, 4\}$, $\{3, 6, 7, 8\}$ and $\{5, 9, \ldots, 13\}$, giving them 52, 50 and 76 non-idle actions respectively — the 178 actions of the full grid, partitioned rather than multiplied. Each agent observes only quantities belonging to its own region and chooses among the topology actions available at its own substations; action 0 is always do-nothing. Agents act simultaneously and are given no information about what the others are doing.

Reward and heuristic. The training reward is dominated by a flat $+1$ per surviving step, plus an overload term, so the objective is essentially to keep the episode alive; the reported metric is episodic survival, the fraction of evaluation episodes completed without a blackout. Following the practice described above, an idle heuristic fast-forwards the simulation while the grid is safe: agents are queried only once $\rho_{\max} \geq 0.90$.

MAPPO. The learning algorithm is multi-agent PPO [7], the natural multi-agent extension of PPO [22] under centralized training with decentralized execution. Each agent i has its own stochastic policy $\pi_{\theta_i}(a_i \mid o_i)$ over its discrete local action space, and is updated with the clipped surrogate objective

$$L^{\mathrm{CLIP}}(\theta_i) = \hat{\mathbb{E}}_t\Big[\min\Big(r_t(\theta_i)\,\hat{A}_t,\ \mathrm{clip}\left(r_t(\theta_i),\,1-\epsilon,\,1+\epsilon\right)\hat{A}_t\Big)\Big], \qquad r_t(\theta_i) = \frac{\pi_{\theta_i}(a_{i,t} \mid o_{i,t})}{\pi_{\theta_i^{\mathrm{old}}}(a_{i,t} \mid o_{i,t})},$$

where $\hat{A}_t$ is a generalized advantage estimate. The advantages come from a single *centralized* value function trained on the global state, which is available in simulation but never given to the actors. Only the actors are kept at execution time, so the deployed system is decentralized even though training was not.

What this thesis starts from. The baseline is that system with flat multilayer-perceptron actors and critic, trained on `bus14`. It is unstable at low survival out of the box, and it is tied to one grid: an MLP's input width is a function of the number of substations, so nothing it learns can be reused elsewhere. Fixing both — first the training, then the representation — is the subject of the chapters that follow.

2.6 Outline

Chapter 3 tunes that baseline until it trains stably, then replaces the flat actor with a graph encoder and checks that nothing is lost. Chapter 4 defines the graph inputs: feature scaling, three representations of the grid, the substation-level augmentations, the readout, and the candidate-action head. Chapter 5 screens those choices experimentally. Chapter 6 changes the objective and asks whether the agents can keep their survival while intervening rarely. Chapter 7 moves to the 36-substation WCCI grid and tests what actually transfers.

Chapter 3

Establishing a Baseline

This chapter establishes a robust baseline for the IEEE 14-bus task. First, we tune a decentralized MLP-based MAPPO agent to achieve stable training and high survival rates. Second, we replace the flat MLP actor with a Graph Neural Network (GNN) encoder.

This architectural shift is essential for generalizability: an MLP's input size is strictly tied to a specific grid size, preventing transferability. In contrast, a shared GNN encoder processes graphs of arbitrary sizes, theoretically enabling zero-shot transfer across different power grids. If a GNN encoder proves viable and competitive with the optimized MLP, it justifies investigating the optimal graph representation of the power grid, which is addressed in Chapters 4 and 5.

3.1 Evaluation Protocol

All experiments use an 80/20 train/test chronic split. The 80% training split is used for policy updates, while the 20% test split estimates generalization. Evaluation rollouts run periodically during training using 10 episodes per split.

The primary performance metric is test episodic survival (expressed as a percentage), representing the fraction of evaluation episodes where the agent successfully prevents a blackout. Results are reported using seed-aggregated mean curves, with shaded regions indicating variability. Final evaluations report the performance of the checkpoint that achieved the highest test survival during training.

3.2 Experiment Index

Table 3.1: Current result blocks and their role in the thesis.

Block	Main question	Primary evidence
MLP baseline reruns	Which baseline training changes stabilize MAPPO?	Table 3.4, Figures 3.2 and 3.1
Initial do-nothing bias	Does biasing initial exploration toward do-nothing help?	Table 3.5, Figure 3.3
GINE reruns	Do graph encoders improve survival over the optimized MLP baseline?	Tables C.1 and 3.6, Figure 3.4
Heavy GINE	Does a larger GINE architecture help or overcomplicate optimization?	Table C.2, Figure 3.5

3.3 IEEE 14-Bus Experiments

The initial PPO/MAPPO baseline from the MARL2Grid framework yielded unstable policies and low survival rates on the IEEE 14-bus topology-control task. The following experiments systematically test training and architectural modifications to establish a high-performing baseline.

3.3.1 MLP Baseline

Question. Which changes are needed to obtain a stable MLP MAPPO baseline on the 14-bus task?

Baseline and changes. We start from the MLP MAPPO baseline of the MARL2Grid framework and test several main stabilization choices:

- increasing actor and critic capacity from two hidden layers to three hidden layers;
- using reward normalization;
- changing the critic update so that the critic is updated once for the whole batch instead of once per agent;
- tuning of optimization hyperparameters to stabilize the training, as summarized in Table 3.2.

Table 3.2: New hyperparameters for MAPPO training.

Hyperparameter	New Value	Paper Baseline
ENVIRONMENT		
n_envs (number parallel environment)	20	10
PPO OPTIMIZATION		
Total time step	15,000,000	25,000,000
Actor learning rate	1e-4	3e-5
Critic learning rate	1e-4	3e-5
Gamma	0.9	0.99
Update epochs	10	80
Minibatches	16	4
Maximum gradient norm	1.0	10.0

Evidence. Table 3.4 reports final and peak survival for the core ablations. Figure 3.2 shows each comparison against the `Optimized MLP Baseline`, and Figure 3.1 overlays all core MLP baseline curves.

Interpretation. The `Optimized MLP Baseline`, which combines three hidden layers, reward normalization, and an optimized critic update, achieves the best performance and convergence speed, yielding the highest and most stable final survival (∼93%). Removing or reducing any of these three elements, using a smaller architecture, disabling reward normalization, or reverting to a non-optimized critic update, results in a drop in either learning speed or final survival. The original unoptimized baseline fails entirely to learn a competitive policy. Because the `Optimized MLP Baseline` consistently outperforms the ablations, it is established as the standard reference for all subsequent comparisons.

Table 3.3: MLP baseline run configurations. The reference run is `Optimized MLP Baseline`.

Run family	Actor/critic hidden layers	Reward norm.	Initial action-0 prob.	Optimized critic update	Difference from `Optimized MLP Baseline`
`Optimized MLP Baseline`	3×256 / 3×256	true	0.0	true	Reference baseline
`Disable optimized critic update`	3×256 / 3×256	true	0.0	false	Disables optimized critic updates
`Smaller MLP actor/critic`	2×256 / 2×256	true	0.0	true	Uses a smaller MLP actor and critic
`Disable reward normalization`	3×256 / 3×256	false	0.0	true	Disables reward normalization
`Old baseline (no val)`	3×256 / 3×256	true	0.3	false	Uses action-0 bias 0.3 and non-optimized critic updates
`Initial Bias 50%`	3×256 / 3×256	true	0.5	true	Uses action-0 bias 0.5
`Initial Bias 70%`	3×256 / 3×256	true	0.7	true	Uses action-0 bias 0.7

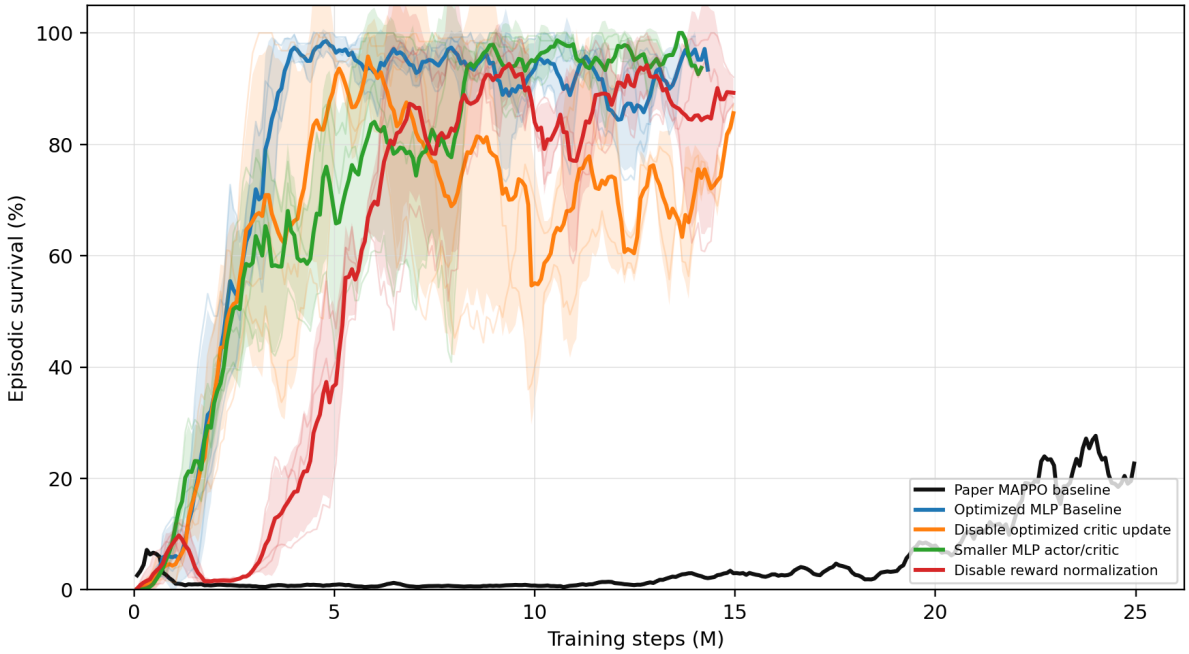


Figure 3.1: All MLP baseline core rerun survival curves shown on one axis.

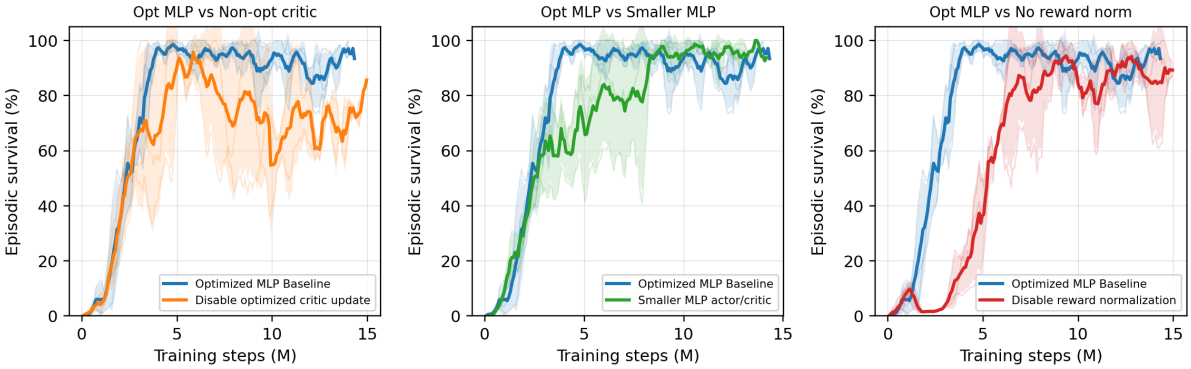


Figure 3.2: Pairwise MLP baseline rerun comparisons against the `Optimized MLP Baseline`.

Table 3.4: MLP baseline core rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
`Optimized MLP Baseline`	Optimized MLP baseline	93.4	98.6
`Disable optimized critic update`	Non-optimized critic update	93.1	95.8
`Smaller MLP actor/critic`	Smaller MLP architecture	93.7	100.0
`Disable reward normalization`	No reward normalization	81.8	97.1

3.3.2 Initial Do-Nothing Bias

Question. Does biasing the initial policy toward action 0, the do-nothing action, help or hurt learning? In theory, a do-nothing bias makes sense because in reality, human operators most often do not operate the grid (i.e., they take no action) under normal conditions.

Baseline and changes. Our baseline uses no initial do-nothing bias in this rerun set. The bias controls keep the same general protocol but change the initial probability mass assigned to action 0.

Implementation detail. The bias is applied only at initialization: it changes the starting action distribution before learning, without changing the reward, architecture, or PPO objective. This is implemented by artificially increasing the initial pre-softmax logits for the do-nothing action, such that the policy initially selects this safe action with a higher probability before any learning updates occur. This makes the ablation a test of exploration bias rather than a test of representational capacity.

Evidence. Table 3.5 and Figure 3.3 compare the zero-bias baseline with `Initial Bias 50%` and `Initial Bias 70%`.

Interpretation. The effect of the action-0 bias is large and non-monotonic. The 0.5-bias curve remains unstable and substantially below the zero-bias baseline, whereas the 0.7-bias curve eventually reaches the highest final survival in this comparison. The current conclusion is that adding an initialization bias is generally hurtful or highly unstable for the models. Instead of aiding exploration, a strong initialization prior appears to restrict the agent's ability to reliably discover diverse beneficial actions and should be treated cautiously as a first-order ablation variable.

Table 3.5: Initial do-nothing bias survival summary.

Run	Final survival (%)	Peak survival (%)
`Optimized MLP Baseline`	93.4	98.6
`Initial Bias 50%`	50.8	70.2
`Initial Bias 70%`	99.3	100.0

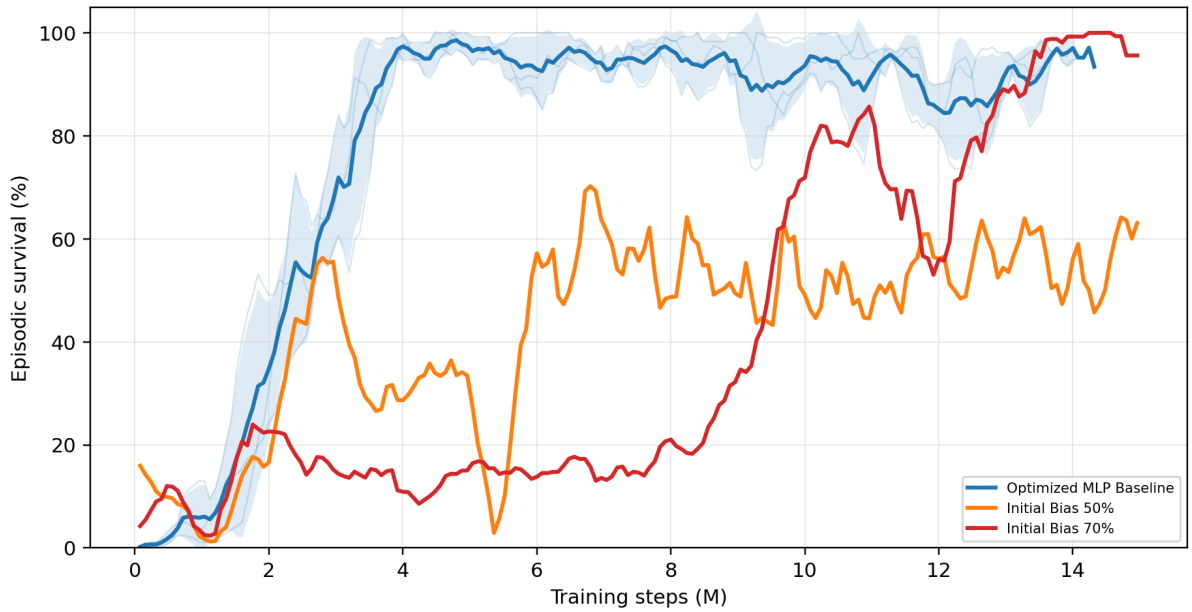


Figure 3.3: Optimized MLP baseline compared with initial do-nothing bias controls `Initial Bias 50%` and `Initial Bias 70%`.

3.3.3 Actor GNN Encoder Viability and Architecture

Question. Can a graph encoder perform as well as the MLP? This is a crucial question because if all agents share the same GNN encoder, the architecture becomes independent of the grid size and could maybe transfer to any grid topology. If each agent requires its own separate encoder or if the GNN cannot stabilize, this transferability is lost.

Baseline and changes. We conducted an exploratory study comparing several variations of the GINE architecture. We tested the encoder size (light vs. heavy GINE), actor sharing (shared vs. unshared encoder), the critic architecture (MLP vs. GNN), and the impact of the optimizations we established for the MLP baseline (optimized critic updates, reward normalization).

Protocol. All GINE runs use `gnn_type = gine`, 15,000,000 total timesteps, evaluation every 80,000 timesteps, 2,000 rollout steps, deterministic evaluation, and the same 80/20 chronic split used by the MLP experiments. We use GINE because it naturally incorporates edge features and has been shown to be powerful at distinguishing graph topologies [19]. The complete graph-actor pipeline is described in Appendix A, and Appendix C details the exhaustive parameter configurations used for these ablations in Tables C.1 and C.2.

Evidence. Table 3.6 and Figure 3.4 compare the core architectural improvements against the `MLP Baseline`. Figure 3.5 provides a broader search, showing how the baseline reacts to individual isolated changes.

Table 3.6: GINE rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
`Heavy GINE Baseline`	Historical GINE baseline	59.2	93.9
`Light GINE (Concat, MLP Critic)`	Light GINE with MLP critic	78.9	90.4
`No Bias, Opt Critic`	Optimized critic and bias 0.0	82.5	85.7
`Optimized Light GINE`	Light optimized GINE with MLP critic	95.7	97.4

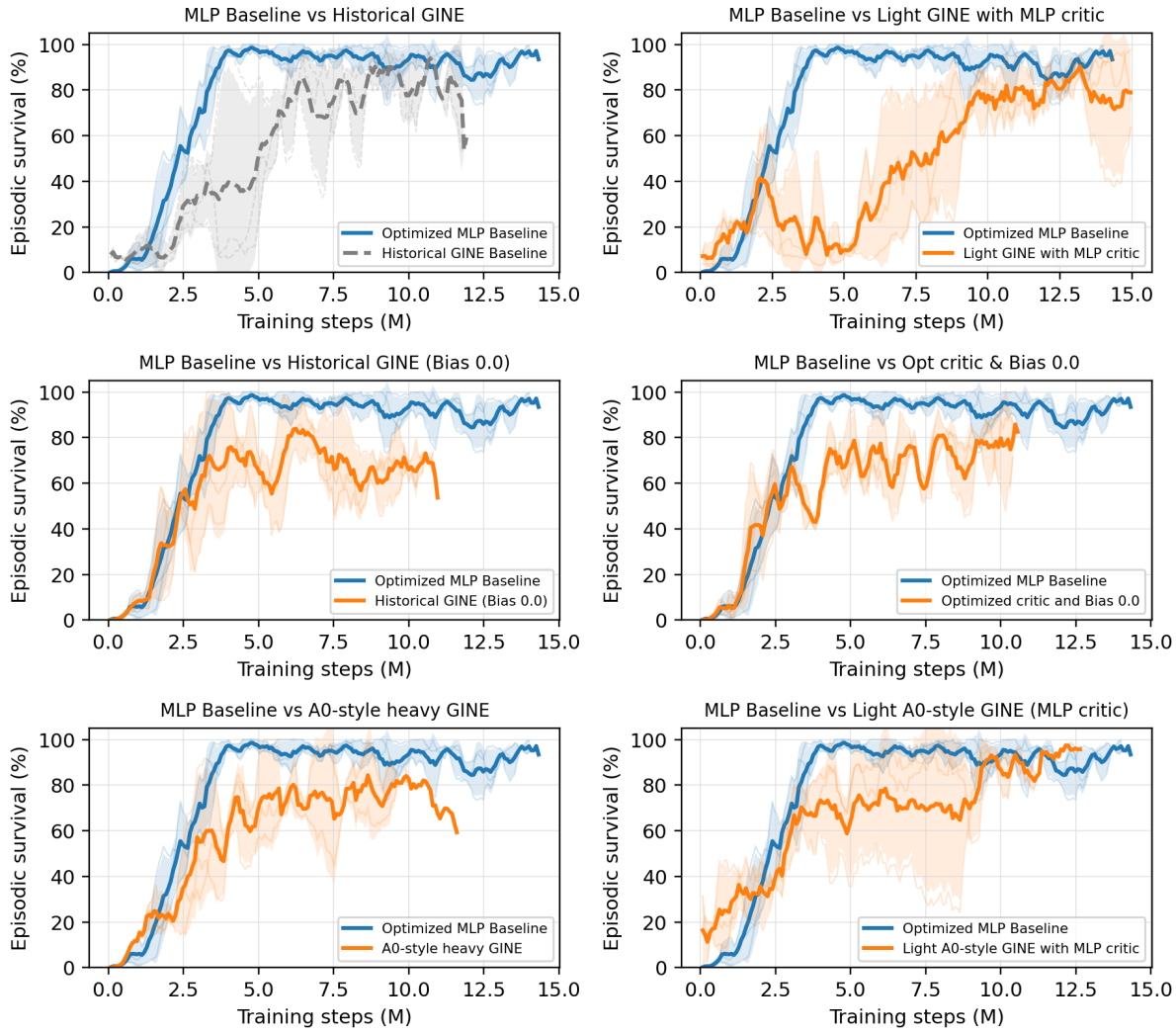


Figure 3.4: Seed-aggregated test episodic survival comparing the `MLP Baseline` against high-performing GINE architectural modifications.

Interpretation. The results from the broad search provide several critical takeaways regarding the GNN architecture:

- **Shared Encoder Viability:** As shown in Figure 3.5, the `Unshared Actor` variant does not outperform the other runs. This proves that a single shared GNN encoder for all agents is achievable and does not bottleneck performance, which is exactly what is needed for transferability.
- **Critic Architecture:** Attempting to use a GNN for the centralized critic proved unstable and degraded performance. The `MLP Critic` variants were significantly more stable. The best performance is clearly achieved by pairing the shared GNN actor with a standard MLP critic.
- **Optimization Synergies:** The GNN architecture is heavily dependent on the same optimizations that stabilized our MLP baseline. Specifically, optimized critic updates, reward normalization, and removing the initial do-nothing bias (bias 0.0) were crucial for getting the GNN to learn reliably.

The strongest overall result is the `Optimized Light GINE`, which combines the shared light GINE actor, the MLP critic, and all the baseline optimizations to reach 95.7% final survival. This validates that a GNN encoder is highly competitive on the 14-bus task.

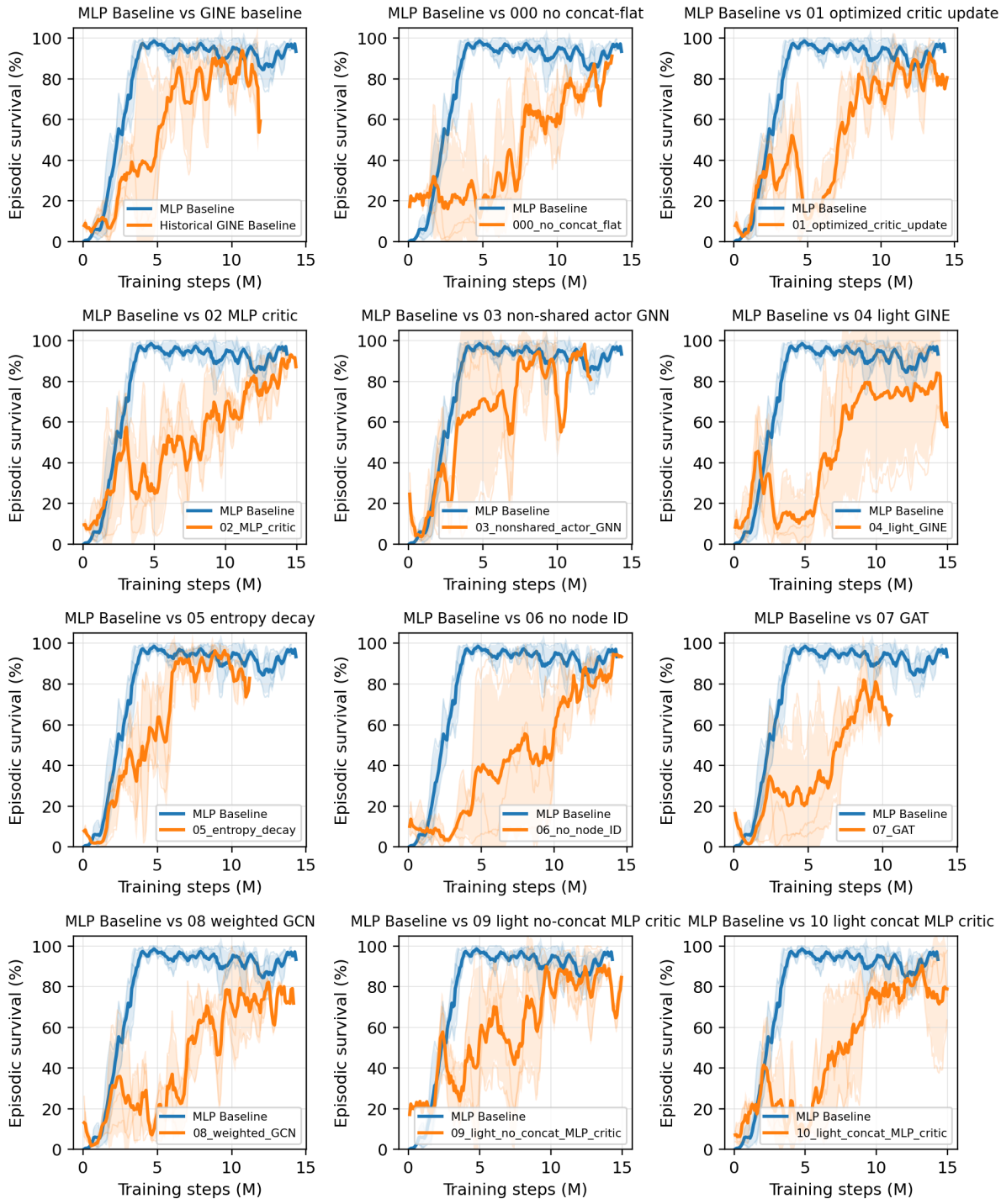


Figure 3.5: GINE survival comparisons. Each subplot compares the `MLP Baseline` against a specific GINE variant from `configs/gine_s0_s1_s2`. All runs are modifications of a baseline GINE model; full hyperparameters and architectural details are provided in Appendix A and Appendix C.

Chapter Summary

Objective: Establish a stable, high-performing MAPPO baseline and validate the viability of a GNN encoder.

Findings:

- **MLP Optimization:** Tuning hyperparameters, enabling reward normalization, and optimizing critic updates increased the flat MLP baseline survival to ∼93%.
- **GNN Viability:** Replacing the MLP actor with a GNN (GINE) is highly competitive. Combined with our baseline optimizations, the `Optimized Light GINE` achieved 95.7% survival.
- **Transferability (Shared Actor):** A shared GNN encoder for all agents performs optimally. This is critical: because the encoder processes subgraphs of consistent dimensions regardless of the global grid size, a shared GNN actor theoretically enables zero-shot transferability to larger grids.
- **Critic Architecture:** The centralized critic observes the entire global graph, inherently tying it to the training grid's specific size. Consequently, a GNN critic offers no transferability benefits and was empirically found to induce training instability. We therefore retain a standard MLP critic.

Conclusion: We have proven that a shared GNN encoder is viable, providing us with an architecture that is structurally transferable and performs excellently on the IEEE 14-bus grid. However, while the architecture allows for transferability in theory, we do not yet know how it will perform in practice when transferred to a different grid. To address this, the next logical step is to explore the optimal ways to represent and process the power grid as a graph, identify the best representation, and attempt transferring it to a larger environment. Chapter 4 defines these representational spaces, and Chapter 5 evaluates them.

Chapter 4

Methods

This chapter describes the graph inputs and graph-actor variants used in the experiments. The presentation separates four design choices: feature scaling, the base graph representation, substation-level augmentations, and graph pooling.

4.1 Physical Scaling and Voltage-Angle Representation

Two input choices are used to make observations more comparable across grids: fixed physical scaling and a reference-independent representation of voltage angles. Both operate before the graph encoder and are independent of the graph schema.

4.1.1 Physical scaling

Each scaled continuous feature x_f is divided by a fixed reference with the same unit,

$$x'_f = \frac{x_f}{s_f}.$$

This changes the unit of the feature without changing the information it contains and, importantly for sparse graph features, keeps zero at zero. The references are listed in Table 4.1. In all reported experiments that use scaling, the power reference is the rating of the largest installed generator in the corresponding grid, $s_P = \max_g P_g^{\max}$.

Table 4.1: Deterministic physical scales used for continuous graph features.

Features	Scale s_f	Source
`gen_p`, `load_p`, `p`	Power base in MW	Largest installed generator rating.
`gen_theta`, `load_theta`, `theta`, `theta_diff`	180 degrees	Fixed angular scale.
`time_before_cooldown_sub`	Maximum substation cooldown	Grid2Op environment parameter.
`time_before_cooldown_line`	Maximum line cooldown	Grid2Op environment parameter.
`timestep_overflow`	Allowed overflow duration	Grid2Op environment parameter.
Maintenance time and duration	Episode horizon	Grid2Op chronic horizon.

The dimensionless loading ratio `rho` and all binary, mask, type, and relation indicators are left unchanged.

4.1.2 Angle representation

An absolute voltage angle requires an arbitrary zero, normally fixed by the slack bus. Adding the same constant to every bus angle therefore describes the same physical state. Absolute angles depend on this

choice of reference, whereas the angle difference across a line does not. For transfer, the graph instead uses

$$\Delta\theta_\ell \;=\; \theta_\ell^{\mathrm{or}} - \theta_\ell^{\mathrm{ex}}$$

This quantity is *gauge-invariant*, meaning that it is unchanged when the angle reference is shifted. It also provides one angle value per line instead of only at nodes with an attached generator or load. The reference-dependent absolute-angle channels are removed when the angle drop is enabled.

Chapter 7 shows why this special treatment is necessary in practice: the source and target grids exhibit a 5–7° offset in absolute angles due largely to their different reference choices. Fixed scaling cannot remove such an offset, whereas the line-angle difference is independent of the reference.

Table 4.2: Where the voltage angle enters each representation, under the two ways of expressing it.

Representation	Absolute angle	Angle drop
Busbar	On busbar nodes, averaged over the attached assets	On the physical line edge
Disaggregated	On generator and load nodes	On the physical line edge
Disaggregated with line nodes	On generator and load nodes	On the *line node*

In the busbar and disaggregated schemas, physical lines are edges, so the angle drop is stored on those edges. In the explicit-line schema, each line is a node and its attachment edges contain no physical measurements; the drop is therefore stored on the line node. This also gives each local actor the angle drop for every line it observes, including boundary lines. A disconnected line is assigned a zero drop because the simulator may retain stale endpoint angles after an outage. These placements are the only schema-specific part of the transformation; the encoder and graph readout are unchanged.

4.1.3 Use across the experiments

The screening experiments of Chapter 5 and the sparse-control experiments of Chapter 6 use the original raw features and absolute angles. The cross-grid transfer study of Chapter 7 uses physical scaling and angle drops on both grids. The code also supports running standardization, but no reported policy run uses it. Section 7.8.1 analyzes it offline as a rejected alternative and finds that it increases the mismatch between grids.

4.2 Graph Representations

The graph observation can use one of three base representations. The `bus` representation aggregates equipment values onto busbar nodes. The `heterogeneous` representation disaggregates generators and loads into explicit asset nodes while keeping physical lines as busbar-to-busbar edges. The `heterogeneous_line` representation further disaggregates transmission lines into explicit line nodes.

Table 4.3 states what separates them. The node schema also determines how a boundary transmission line is represented in a local actor graph. Consequently, the three representations are not information-equivalent: they expose different amounts of context about the substation at the far end of a shared line. This distinction is made explicit in Section 4.2.5.

Table 4.3: The three graph schemas. Each row is a consequence of how much equipment receives its own node.

Aspect	`bus`	`heterogeneous`	`heterogeneous_line`
Node types	Busbar	Busbar, generator, load	Busbar, generator, load, transmission line
Generator and load state	Summed and averaged onto the busbar	One node per asset	One node per asset
Physical line	Direct busbar-to-busbar edge	Typed direct busbar-to-busbar edge	Typed node between its endpoint busbars
Line state	Continuous edge attributes	Continuous edge attributes	Features of the line node
Minimum useful depth	One layer for adjacent busbar exchange	One layer for adjacent busbars; two for asset-to-remote-busbar flow	Two layers for adjacent busbar exchange through a line node
Hidden line state	None: the edge modifies a message	None: the edge modifies a message	Yes: each line obtains and updates an embedding
Far-end context in a local actor graph	Live neighboring busbar, including aggregated power and angle	Live neighboring busbar, but no neighboring generator or load nodes	None; the shared line node replaces the need for a far-end busbar
Node count	SB	$SB + N_g + N_d$	$SB + N_g + N_d + L$

4.2.1 Candidate graph and dynamic mask

All three schemas share one construction.

Grid2Op exposes the instantaneous electrical state and topology through a structured observation [4]. For each representation, the graph construction keeps the set of candidate nodes and edges fixed throughout an episode while using a dynamic edge mask to express the topology that is electrically active at time step t.

For agent a, the graph observation can be written as

$$\mathcal{G}_t^{(a)} = \left(\mathcal{V}^{(a)}, \mathcal{E}_{\text{cand}}^{(a)}, \mathbf{X}_t^{(a)}, \mathbf{Z}_t^{(a)}, \mathbf{m}_t^{(a)}, \mathbf{n}_t^{(a)}, \mathbf{c}^{(a)}\right), \tag{4.1}$$

where $\mathcal{V}^{(a)}$ and $\mathcal{E}_{\text{cand}}^{(a)}$ are static, $\mathbf{X}_t^{(a)}$ contains node features, $\mathbf{Z}_t^{(a)}$ contains edge features, $\mathbf{m}_t^{(a)}$ indicates which candidate edges are active, $\mathbf{n}_t^{(a)}$ marks currently visible nodes, and $\mathbf{c}^{(a)}$ marks nodes belonging to the agent's controlled domain. The controlled mask is static, whereas the edge and visibility masks change with line status and busbar assignments. This separates the fixed candidate layout from the electrical topology that can change after every action.

The mechanism is easiest to see on a transmission line in the `bus` and `heterogeneous` schemas, where a physical line is an edge. Consider a line ℓ whose origin and extremity belong to substations $o(\ell)$ and $e(\ell)$. For every pair of possible endpoint busbars (b_o, b_e), the graph reserves the two directed candidate edges

$$i(o(\ell), b_o) \to i(e(\ell), b_e) \quad \text{and} \quad i(e(\ell), b_e) \to i(o(\ell), b_o), \tag{4.2}$$

so each physical line contributes $2B^2$ candidate directed edges and the two directions share the same line attributes. For the usual case $B = 2$, one line reserves eight candidate edges even though at most two, one per direction, are electrically active at any time.

Which of them is active is decided by the mask

$$m_{\ell,b_o,b_e,t} = \mathbf{1}[\text{line } \ell \text{ is connected at } t]\, \mathbf{1}\left[b_{\ell,t}^{\text{or}} = b_o\right]\, \mathbf{1}\left[b_{\ell,t}^{\text{ex}} = b_e\right]. \tag{4.3}$$

A disconnected line has all of its candidates masked; a connected one keeps only the pair matching its current origin and extremity busbars, provided both endpoint nodes are visible. A topology action therefore changes connectivity by changing $\mathbf{m}_t^{(a)}$, never by rebuilding the graph, and the node and edge tensors keep a fixed shape for the whole episode. The same candidate and mask pattern governs the asset attachments of Section 4.2.3 and the line-node attachments of Section 4.2.4; Appendix B gives the resulting sizes and the masking conventions.

Why a fixed shape rather than rebuilding the graph. Rebuilding the graph after every topology action would be the more obvious implementation, and it is rejected for four reasons, the last of which is the substantive one.

1. **The interface requires it.** Each agent's observation is declared once as a fixed-size space, and rollouts are collected from many environments stepping in parallel. The rollout buffer is allocated once, from the shape of the first observation, and written into at every step; an observation whose node or edge count changed mid-episode could neither be stacked across environments nor stored in that buffer.
2. **Batching stays trivial.** A fixed layout means node and edge tensors for different environments, agents and time steps are stacked by concatenation, and the message-passing index is built once and reused, rather than a variable-size graph being assembled and re-indexed on every forward pass.
3. **Row identity is stable.** Row i always denotes the same physical busbar, line or asset. The candidate-action head of Section 4.5 depends on this directly: it maps each discrete action to the graph rows it modifies once, before training. If the node set were rebuilt per step, that mapping would have to be recomputed at every step and would refer to different objects at different times.
4. **Absent connections stay visible.** A rebuilt graph would contain only the connections that currently exist, which hides the ones an action could create: the empty busbar an element would be moved onto would simply not be in the graph, so no encoder reading it could evaluate that action. Keeping the candidate set fixed means the reachable topologies are present as masked structure, and the mask makes the current topology an explicit input rather than something inferable only from which messages happen to exist. This is the busbar information asymmetry identified by de Jong et al. [20].

The cost is memory. With $B = 2$ a line reserves eight directed candidates of which at most two are ever active, so the edge tensor is roughly four times larger than the active topology requires. On the grids used here that is affordable, and Appendix B gives the exact sizes.

The following subsections define the three schemas.

4.2.2 Busbar Representation

The `bus` schema is the compact busbar representation. Busbars are the only nodes, active transmission lines form the graph edges, and generator and load quantities are aggregated into their currently assigned busbars.

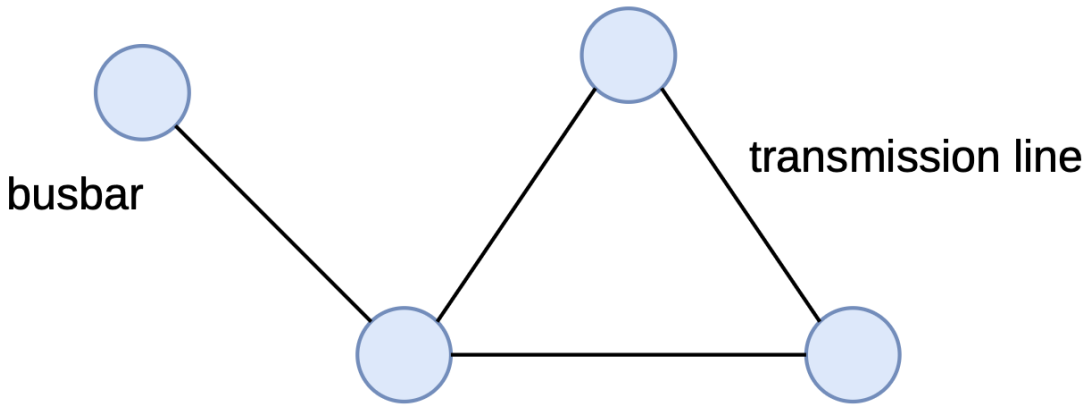


Figure 4.1: Busbar graph representation. Blue nodes denote busbars and black connections denote active transmission-line relations. Generator and load quantities are aggregated into the features of their associated busbars.

Nodes and features

Let S denote the number of substations and let B denote the maximum number of busbars per substation. A node is created for every substation–busbar pair, including busbars to which no asset is connected at

the current time. A global node identifier is assigned as

$$i(s, b) = sB + b, \qquad s \in \{0, \dots, S-1\}, \quad b \in \{0, \dots, B-1\}. \tag{4.4}$$

The complete graph therefore contains SB nodes.

Table 4.4 lists the six node features. Generator and load quantities are assigned to the busbar specified by the current topology vector. Active powers are summed when several assets occupy the same busbar, whereas the corresponding angles are averaged. Disconnected assets do not contribute to any node. The substation cooldown is repeated on every busbar node of that substation.

Table 4.4: Features attached to each busbar node.

Feature	Definition
`gen_p`	Sum of active generator power connected to the busbar.
`gen_theta`	Mean voltage angle of generators connected to the busbar.
`load_p`	Sum of active load power connected to the busbar.
`load_theta`	Mean voltage angle of loads connected to the busbar.
`time_before_cooldown_sub`	Remaining substation cooldown.
`domain_mask`	One for a substation controlled by the agent and zero for a contextual neighbor.

This aggregation produces a compact representation but deliberately removes the identities and counts of individual generators and loads.

Relations and edge features

The only physical relations (edges) of the busbar schema are the busbar-to-busbar transmission line. The dynamic attributes copied onto active physical-line edges are given in Table 4.5. The two maintenance features are present only for environments with scheduled maintenance.

Table 4.5: Features attached to physical-line edges.

Feature	Definition
`line_status`	Binary connection status of the physical line.
`rho`	Line loading relative to its thermal limit.
`timestep_overflow`	Number of consecutive time steps in overload.
`time_before_cooldown_line`	Remaining cooldown before another line action is legal.
`time_next_maintenance`	Time until scheduled maintenance, when available.
`duration_next_maintenance`	Duration of scheduled maintenance, when available.

The edge embedding of a busbar-to-busbar transmission edge is exactly the row of Table 4.5. Optional same-substation edges use the same feature width: all physical channels are zero throughout. The homogeneous schema does not append relation one-hots, so these optional relations are distinguished only by their endpoints and neutral physical attributes.

Local information boundary

For an agent-local `bus` graph, all busbars of controlled substations are present. A live boundary line additionally requires exactly one contextual node: the far-end busbar to which that line is currently attached. This node is needed because the shared line is an edge, and both endpoint nodes must exist for its features to reach the controlled busbar through message passing. Because generator and load quantities are aggregated onto busbars, the visible far-end busbar also exposes aggregated neighboring `gen_p`, `load_p`, and angle features. The other busbar candidates at that neighboring substation are masked, even if they carry generation or load. If the boundary line is disconnected, no neighboring busbar remains visible.

4.2.3 Disaggregated Representation

The `heterogeneous` schema keeps busbars as the backbone and adds one node per generator and load. Transmission lines remain busbar-to-busbar edges. The node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}}, \tag{4.5}$$

with typed directed relations between these sets. Generator and load states are therefore preserved individually.

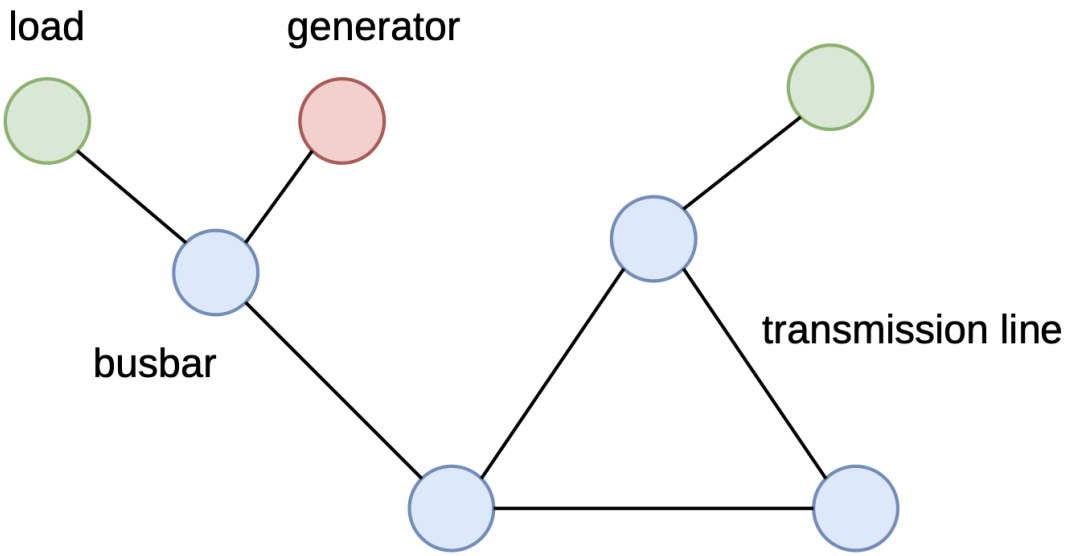


Figure 4.2: Heterogeneous equipment representation. Busbars are blue, generators are red, and loads are green. Physical transmission lines remain direct relations between busbar nodes.

Nodes and features

For a graph with S included substations, B busbars per substation, N_g included generators, and N_d included loads, the number of nodes is

$$|\mathcal{V}| = SB + N_g + N_d. \tag{4.6}$$

Busbar nodes are created exactly as in Section 4.2.2. In addition, each generator and load whose substation is present in the graph receives its own node.

Every busbar, generator, and load node is represented by the same ordered eight-dimensional feature vector

$$\begin{aligned}\mathbf{x}_v = [&\texttt{is\_busbar}, \texttt{is\_generator}, \texttt{is\_load}, \texttt{p}, \texttt{theta},\\ &\texttt{time\_before\_cooldown\_sub}, \texttt{domain\_mask}]^{\mathsf{T}} \in \mathbb{R}^7.\end{aligned} \tag{4.7}$$

The width and meaning of this vector are identical for all node types. Table 4.6 shows the value placed in every shared column. A zero in the table means that the column is still present for that node type but is explicitly zero-filled.

Table 4.6: Values placed in the eight feature columns shared by every node in the direct-line heterogeneous graph. Disconnected assets retain their node row but have zero power and angle.

Shared feature column	Busbar node	Generator node	Load node
`is_busbar`	1	0	0
`is_generator`	0	1	0
`is_load`	0	0	1
`p`	0	`gen_p` if connected; 0 otherwise	`load_p` if connected; 0 otherwise
`theta`	0	`gen_theta` if connected; 0 otherwise	`load_theta` if connected; 0 otherwise
`time_before_cooldown_sub`	Substation value	0	0
`domain_mask`	1 controlled; 0 contextual	1 controlled; 0 contextual	1 controlled; 0 contextual

There is no per-asset connection flag. An earlier version of this schema carried a `connected` column, written as a constant on busbar rows, as a copy of `line_status` on line rows, and as the attachment state on generator and load rows. Only the last of those carried information, and it never varied: a *change* action toggles an element between busbars and cannot detach it, and a substation split that isolated a load or generator ends the episode. Measured over 600 random topology actions on `bus14` spanning 180 episode ends, and over 400 on the WCCI grid, no generator or load was ever observed detached, against 150 and 493 line disconnections respectively. The column was therefore removed rather than carried.

A detached asset would still be identifiable: its measured channels are zeroed while its type indicator stays set. What is lost is only the distinction between that state and an attached asset reading zero, which would matter under an action space exposing `set_bus`. Note also that connection in this sense was never the same thing as energization, which is reported separately by the energized mask of Section 4.4: on agent 0 of `bus14` four of eight busbars carry no current under the nominal topology.

Relations and edge features

Every candidate directed edge is represented by the same ordered feature vector. Its physical block is

$$\begin{aligned}\mathbf{p}_{uv} = [\,&\texttt{line\_status},\ \texttt{rho},\ \texttt{timestep\_overflow},\\ &\texttt{time\_before\_cooldown\_line},\\ &(\texttt{time\_next\_maintenance},\ \texttt{duration\_next\_maintenance})_{\text{optional}}\,]^{\mathsf{T}}.\end{aligned} \tag{4.8}$$

The two maintenance columns are appended only when scheduled maintenance is available. The relation block is the same six-column vector for every edge:

$$\begin{aligned}\mathbf{r}_{uv} = [\,&\texttt{relation\_physical\_line},\\ &\texttt{relation\_same\_substation\_busbar},\\ &\texttt{relation\_generator\_to\_busbar},\ \texttt{relation\_busbar\_to\_generator},\\ &\texttt{relation\_load\_to\_busbar},\ \texttt{relation\_busbar\_to\_load}\,]^{\mathsf{T}}.\end{aligned} \tag{4.9}$$

The complete common edge vector is therefore

$$\mathbf{e}_{uv} = [\,\mathbf{p}_{uv}^{\mathsf{T}} \mid \mathbf{r}_{uv}^{\mathsf{T}}\,]^{\mathsf{T}} \in \begin{cases}\mathbb{R}^{10}, & \text{without maintenance columns,}\\ \mathbb{R}^{12}, & \text{with maintenance columns.}\end{cases} \tag{4.10}$$

Its width, column order, and meaning do not change with the relation type. An active physical-line edge fills $\mathbf{p}_{uv}$ with its measured line state. Every active structural or asset-attachment edge instead uses

$$\mathbf{p}_{\text{neutral}} = \mathbf{0}, \tag{4.11}$$

so a relation carrying no flow reports none. In every active edge row, exactly one column of $\mathbf{r}_{uv}$ is one and the other five are zero.

Table 4.7 shows the values placed in these common blocks for every relation.

A relation carrying no flow reports no loading. An earlier version set `rho`= 1 on structural and attachment relations. That value was a multiplicative identity for the weighted-GCN variant, which reads the `rho` column directly as a scalar message weight and would delete any message it weighted by zero. It was not a measurement, and for the GINE and GAT encoders used throughout – which consume the edge vector as an attribute through a multilayer perceptron and never touch the weight path – it made a relation carrying no current read as a line at its thermal limit.

The attribute is now zero, like every other physical channel of the neutral block, and the substitution moved to the encoder: the weighted variant takes the measured loading on physical lines and unit weight on everything else. The attribute stays an honest measurement for the encoders that read it as one, and the weighted variant still behaves as intended.

Table 4.7: Values placed in the common edge-feature vector shared by every active directed relation in the direct-line heterogeneous graph. "Other five" refers to the remaining columns of the six-column relation block.

Directed relation	Active rule	Shared physical block	Shared relation block
$i_o \to i_e$ and reverse (physical line)	Line online and candidate busbars match the current endpoints	Measured $\mathbf{p}_{uv}$	`relation_physical_line`= 1; other five = 0
$i(s,b) \to i(s,b')$, $b \neq b'$	Optional; every ordered same-substation busbar pair	$\mathbf{p}_{\text{neutral}}$	`relation_same_substation_busbar`= 1; other five = 0
$g \to i(s(g),b)$	Generator connected to candidate busbar b; direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_generator_to_busbar`= 1; other five = 0
$i(s(g),b) \to g$	Same mask; reverse direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_busbar_to_generator`= 1; other five = 0
$d \to i(s(d),b)$	Load connected to candidate busbar b; direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_load_to_busbar`= 1; other five = 0
$i(s(d),b) \to d$	Same mask; reverse direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_busbar_to_load`= 1; other five = 0

Generator and load directions are selected independently as `bidirectional`, `asset_to_busbar`, or `busbar_to_asset`. The default is `bidirectional`. Physical-line and same-substation relations remain bidirectional.

Local information boundary

The agent-local `heterogeneous` graph contains all busbars, generators, and loads of controlled substations. For each live boundary line it also contains the single far-end busbar to which the line is attached, since the line is still represented as a busbar-to-busbar edge. It never constructs a generator or load belonging to a neighboring substation, even when that asset is connected to the visible neighboring busbar. Neighboring private power and angle therefore cannot enter message passing or graph readout. Inactive far-end busbar candidates are masked in the same way as in the `bus` schema.

The consequence: a contextual busbar here carries no measurement. This is worth stating plainly because it is easy to read the far-end busbar as carrying the neighbor's state, as it does in the busbar schema. It does not. Power and angle live on asset nodes in this representation, and neighboring assets are never constructed, so the far-end busbar row reduces to its type and role indicators. Built on agent 0 of `bus14` under the nominal topology, the two visible contextual rows are

$$\underbrace{[0,\ 0,\ 44.3,\ -5.51,\ 0,\ 0]}_{\texttt{bus}} \quad \text{against} \quad \underbrace{[1,\ 0,\ 0,\ 0,\ 0,\ 0,\ 1,\ 0]}_{\texttt{heterogeneous}},$$

that is 44.3 MW of neighboring load and its angle in one schema, and exactly zero in every measured channel in the other. The two contextual rows of the disaggregated graph are also identical to each other, so they are not even distinguishable as different substations.

Whether this is the right choice depends on which question is being asked, and the two answers point in opposite directions.

- **As an information boundary it is the defensible one.** A neighboring substation's generation and load are another region's private state. A regional operator sees the flow on the tie line, not

the neighbor's dispatch. On that reading the busbar schema is the leaky one, and this schema is closer to the decentralized setting the thesis claims to model.

- **As a graph design it is incoherent.** Having decided to withhold the neighbor's power, keeping its busbar as a node buys little: it is a constant, anonymous, dead-end node, since edges are only created for lines touching the controlled domain and a contextual node therefore never connects to another contextual node. What the agent learns about the neighbor arrives through the *edge* attributes of the tie line, chiefly its loading. The node contributes a nonlinear round trip, not new information.

The explicit-line schema resolves this by not constructing the far-end busbar at all and placing the shared-line state on the line node instead. The three schemas are therefore not a clean ladder of decreasing context; the disaggregated one occupies an awkward middle position. This also makes a `bus` against `heterogeneous` comparison something other than a pure comparison of architectures, which is the point developed in Section 4.2.5.

4.2.4 Disaggregated Representation with Line Nodes

The `heterogeneous_line` schema replaces each direct physical-line edge with a transmission-line node between its endpoint busbars. Its node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}} \cup \mathcal{V}_{\text{line}}. \tag{4.12}$$

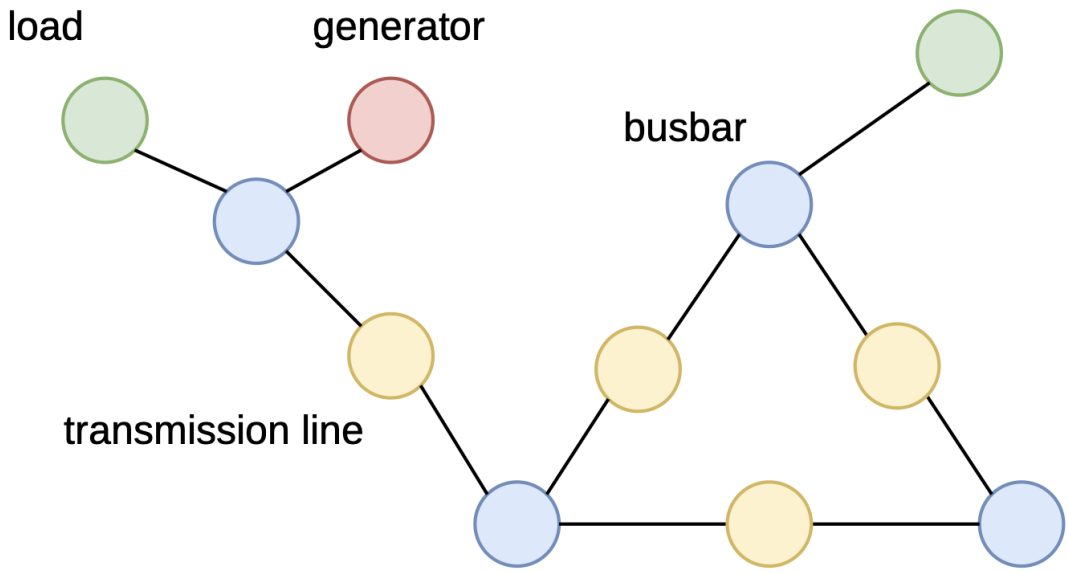


Figure 4.3: Heterogeneous representation with explicit transmission-line nodes. Yellow nodes represent physical lines and connect the blue busbars at their endpoints; generators and loads remain explicit terminal nodes.

Nodes and features

For S included substations, B busbars per substation, N_g generators, N_d loads, and L included transmission lines, the graph contains

$$|\mathcal{V}| = SB + N_g + N_d + L \tag{4.13}$$

nodes.

Every node type uses the same ordered feature vector:

$$\begin{aligned}\mathbf{x}_v = [&\texttt{is\_busbar},\ \texttt{is\_generator},\ \texttt{is\_load},\ \texttt{is\_transmission\_line},\\ &\texttt{p},\ \texttt{theta},\ \texttt{line\_status},\ \texttt{rho},\ \texttt{timestep\_overflow},\\ &\texttt{time\_before\_cooldown\_line},\\ &(\texttt{time\_next\_maintenance},\ \texttt{duration\_next\_maintenance})_{\text{optional}},\\ &\texttt{time\_before\_cooldown\_sub},\ \texttt{domain\_mask}]^{\mathsf{T}},\\ \mathbf{x}_v \in &\begin{cases}\mathbb{R}^{12}, & \text{without maintenance columns},\\ \mathbb{R}^{14}, & \text{with maintenance columns}.\end{cases}\end{aligned} \tag{4.14}$$

The optional block sits in the middle of the vector rather than at its end because the ordering groups columns by the object each quantity belongs to: type indicators, then asset measurements, then line measurements, then the substation measurement, then the role flags. Maintenance countdowns are per-line quantities, so appending them would place a line measurement after the substation measurement and the role flags and split the line block in two. The ordering is a readability convention and nothing more, since the first linear layer is applied to the whole vector and a permutation of input columns is absorbed by a permutation of its weight columns.

It has one practical cost. Appending the optional block would make the thirteen-column layout a prefix of the fifteen-column one, so a checkpoint trained under one setting could be loaded into the other by padding. With the block inserted, three columns change meaning between the layouts and no such correspondence exists. This is not hypothetical in general – the transfer loader already contains a routine that splices an obsolete relation column out of an older checkpoint so that it fits the current edge vector – but it does not arise here, because `bus14` carries no maintenance and the transfer study removes it from WCCI as well (Section 7.1.1), leaving both sides at thirteen columns.

The width and meaning of this vector are identical for busbar, generator, load, and transmission-line nodes. Table 4.8 specifies every entry; 0 denotes an explicitly zero-filled column, not an omitted feature. For generators and loads, power and angle are set to zero when the asset is disconnected.

Table 4.8: Values placed in the common node-feature vector of the explicit-line heterogeneous graph, when voltage angles are carried as absolute per-asset values. The two maintenance rows are included only when maintenance observations are enabled. When the angle is instead carried as the drop along each line, the angle row changes owner: the generator and load entries become 0 and the transmission line holds the drop across it, set to 0 while the line is out (Section 4.1.2).

Shared feature column	Busbar	Generator	Load	Transmission line
`is_busbar`	1	0	0	0
`is_generator`	0	1	0	0
`is_load`	0	0	1	0
`is_transmission_line`	0	0	0	1
`p`	0	`gen_p` if connected; 0 otherwise	`load_p` if connected; 0 otherwise	0
`theta`	0	`gen_theta` if connected; 0 otherwise	`load_theta` if connected; 0 otherwise	0
`line_status`	0	0	0	Observed line value
`rho`	0	0	0	Observed ρ_ℓ
`timestep_overflow`	0	0	0	Observed line value
`time_before_cooldown_line`	0	0	0	Observed line value
`time_next_maintenance`	0	0	0	Observed line value
`duration_next_maintenance`	0	0	0	Observed line value
`time_before_cooldown_sub`	Substation value	0	0	0
`domain_mask`	1 controlled; 0 contextual	1 controlled; 0 contextual	1 controlled; 0 contextual	1 if either endpoint is controlled; 0 otherwise

Relations and edge features

Every candidate directed edge uses the same ordered nine-dimensional feature vector:

$$\begin{aligned}\mathbf{e}_{uv} = [&\texttt{relation\_same\_substation\_busbar},\\ &\texttt{relation\_busbar\_to\_line\_origin}, \texttt{relation\_line\_origin\_to\_busbar},\\ &\texttt{relation\_busbar\_to\_line\_extremity}, \texttt{relation\_line\_extremity\_to\_busbar},\\ &\texttt{relation\_generator\_to\_busbar}, \texttt{relation\_busbar\_to\_generator},\\ &\texttt{relation\_load\_to\_busbar}, \texttt{relation\_busbar\_to\_load}]^{\mathsf{T}} \in \mathbb{R}^9.\end{aligned} \tag{4.15}$$

These edges contain no continuous line-state attributes because the transmission-line node already contains `rho`, status, overflow, cooldown, and maintenance. For an active edge, exactly one relation column is one and the other eight are zero. An inactive candidate has all nine entries set to zero and is removed by `edge_mask`$= 0$ before message passing. Table 4.9 gives the active rule and the nonzero entries for every relation.

Table 4.9: Nonzero entries in the common 9-column edge-feature vector of the explicit-line heterogeneous graph. All relation columns not named in a row are zero.

Directed relation	Active rule	Relation column set to 1
$i(s,b) \to i(s,b')$, $b \neq b'$	Optional; every ordered same-substation pair	`relation_same_substation_busbar`
$i_o \to \ell$	Online line, current origin busbar, direction enabled	`relation_busbar_to_line_origin`
$\ell \to i_o$	Same mask; reverse direction enabled	`relation_line_origin_to_busbar`
$i_e \to \ell$	Online line, current extremity busbar, direction enabled	`relation_busbar_to_line_extremity`
$\ell \to i_e$	Same mask; reverse direction enabled	`relation_line_extremity_to_busbar`
$g \to i(s(g), b)$	Generator attached to b; direction enabled	`relation_generator_to_busbar`
$i(s(g), b) \to g$	Same mask; reverse direction enabled	`relation_busbar_to_generator`
$d \to i(s(d), b)$	Load attached to b; direction enabled	`relation_load_to_busbar`
$i(s(d), b) \to d$	Same mask; reverse direction enabled	`relation_busbar_to_load`

Local information boundary

In the agent-local `heterogeneous_line` graph, every line touching a controlled substation is itself part of the agent's observation/action graph. Only controlled busbars, generators, loads, and the corresponding line nodes are constructed. A boundary line node is connected only to its endpoint busbar inside the controlled domain; no candidate edge to its non-controlled endpoint and no neighboring busbar, generator, or load node is created. The line node already carries the shared measurements, so no far-end context is needed. If the line is disconnected, the line node remains visible with `line_status`$= 0$, but all of its busbar-attachment edges are inactive. This construction is unchanged by the neighbor-inclusion option.

4.2.5 Local actor information boundary and the global state graph

Each agent a controls a set of substations $\mathcal{C}_a$. A boundary line connects $\mathcal{C}_a$ to another agent's domain, and its far-end busbar is the only possible neighboring busbar. The local graph exposes only the boundary information required by its representation.

Why one hop, and why not more. The one-hop cut is a modelling decision and it is worth separating from two arguments that do not support it. The first is that an agent cannot act on a neighboring substation, so it need not observe one. Controllability and observability are independent: a regional control room does not operate its neighbor's substations but does observe them, through state estimation on an extended model, tie-line telemetry, and inter-operator data exchange. On realism alone, the local graphs used here are more restrictive than a real control room, not less.

The second is that one hop is the natural amount of exterior to keep. Power flow has no one-hop boundary. The established treatment in power systems is an external network equivalent, which reduces everything outside the area to a small model preserving *boundary* behaviour, and which deliberately discards the neighbor's internal dispatch while retaining the boundary injections and the stiffness seen

from the boundary. Read that way, the neighbor's own generation and load is the least useful part of the exterior: it is a single bus of an unbounded remainder. What determines whether a switching action is safe inside $\mathcal{C}_a$ is the post-action flow there, and the exterior enters that calculation only through the boundary condition – the loading of the tie line and the angle across it, both of which the schemas already carry on the line itself (Section 4.1.2).

The argument that does support keeping the actor local is not physical but methodological. An actor that observes everything is a centralized controller with extra steps, and the question this thesis asks is how far a policy conditioned on a regional view can go. The centralized critic already reads the global state, so the value estimate is not blind; only the policy is. That compensates in part, but not completely: where the optimal action depends on exterior state the actor cannot see, the policy can only learn an average over it, which is the standard limitation of a decentralized partially observed setting and is exactly the regime in which a remote contingency decides whether a local action was correct.

The honest position is therefore that context depth is an ablation rather than a justified constant, and it is treated as one: `gnn_include_neighbors` switches it, and the two settings are compared under otherwise identical configurations.

In what sense the earlier construction was defective. Since observing a neighbor is not in itself improper, it is worth being precise about what was wrong with the construction this section replaces, because it was not simply that it revealed more.

The earlier rule expanded every substation of the region into all of its busbars and marked them permanently valid. In effect an agent could see anything that shared a substation label with something it touched. That is a convention of the data model, not an electrical relation: two busbars of one substation are separate electrical nodes whenever the substation is split. Built on agent 0 of `bus14`, with contextual substation 3 split so that its 44.3 MW load sits on the second busbar while the agent's lines attach to the first, the earlier construction placed that load in the observation on a node with *no active edge at all*. Electrically it was as remote from the agent as any other bus in the grid.

Two things follow, and the second matters more than the first. Such a node is isolated, so it receives and sends no messages: the value never entered message passing and instead contributed its raw feature row directly to the mean readout. The information was therefore not usable context but an unstructured additive term in the pooled vector. And because the split is an action of the *neighboring* agent, that agent's decisions moved quantities inside this agent's observation without any change in this agent's electrical situation, adding an observation-level non-stationarity on top of the one decentralized training already has.

The correction is therefore about the criterion and the delivery path rather than about volume. Requiring a live electrical connection replaces a naming convention with a physical one, and removes rows that the graph could not process in any case. It does not follow that the resulting boundary is the correct amount of exterior; as argued above, it is a conservative one, and an explicit exterior equivalent would be the principled alternative.

Visibility and masking

Candidate rows are retained for fixed tensor shapes, but a contextual busbar is visible only while a live boundary line connects it to the controlled region. Otherwise its entire feature row is zeroed, every incident edge is deactivated and removed before message passing, and the row is excluded from pooling—including the denominator of a mean. It therefore affects neither message passing nor the readout. Same-substation edges cannot restore visibility.

Controlled nodes remain visible when disconnected. In `heterogeneous_line`, a shared-line node also remains visible because it belongs to the agent graph, although its attachment edges are inactive when the line is offline. Physical and attachment relations are active only when the line is online and the current busbar assignments match their candidate endpoints. Neighboring generators and loads are never constructed, and the explicit-line representation creates no far-end attachment. Pooling is thus a summary operation, not the information-security boundary.

Representation-dependent information

The representations are not information-equivalent. The busbar schema exposes the most neighboring information because it aggregates assets at the far-end busbar; the heterogeneous schema exposes only that busbar; and the explicit-line schema exposes only the shared line.

Table 4.10: Information from another agent's domain visible to a local actor.

Representation	Neighbor information available
`bus`	Shared-line edge and attached far-end busbar, including aggregated generator/load power and angles
`heterogeneous`	Shared-line edge and attached far-end busbar; no neighboring generator/load nodes or private-asset state
`heterogeneous_line`	Shared-line node only; no neighboring busbar, equipment, or far-end attachment

Comparing these representations therefore compares both architectures and actor information sets.

Legacy compatibility and centralized state

The legacy construction kept all neighboring busbars, and in heterogeneous graphs their private assets, visible even without a live boundary connection. Such rows could enter pooling even when isolated from message passing. Setting `gnn_context_requires_connection=false` restores this legacy visibility for A/B tests; legacy and leakage-free checkpoints are therefore not strictly comparable.

This boundaries apply only to decentralized actor graphs. The centralized critic still uses the full-grid state graph during training, while execution uses each actor's local graph, as required by MAPPO's centralized- training, decentralized-execution setup [7].

4.2.6 Graph actor and centralized critic

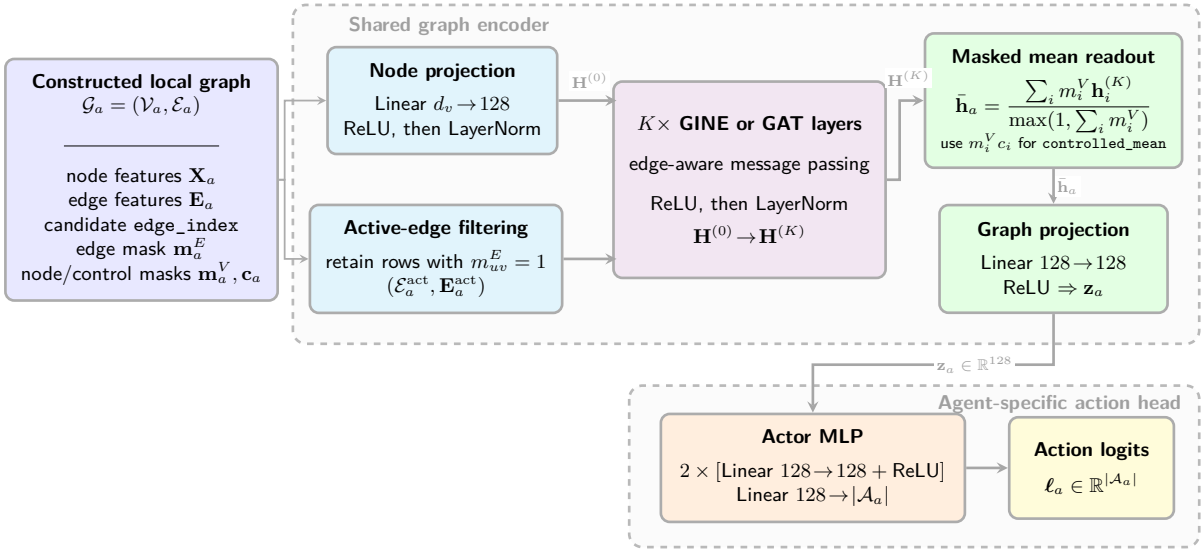


Figure 4.4: End-to-end graph-actor architecture. Candidate edges masked by the current topology are removed before message passing. The graph encoder and its output projection are shared across agents, whereas the final MLP has one output per action available to agent a and is therefore agent-specific.

GINE and GAT use the same actor pipeline and differ only in the message-passing operator. In the reported configuration, the input node projection and the two message-passing layers have 128 features. LayerNorm is applied after the input projection and after each message-passing layer. The graph readout

produces a 128-dimensional embedding, followed by an agent-specific action head with two 128-unit hidden layers. The graph encoder and output projection are shared across agents, whereas the action heads are separate because their action-set sizes may differ.

This is the default configuration. The candidate-scoring alternative of Section 4.5 replaces this head.

The centralized critic is a separate MLP over the normalized flat state. It does not reuse the actor graph embedding and is used only during centralized training. Its flat-state normalization is independent of the graph-input normalization. The exact GINE and GAT updates, graph readout, output projection, and action-head equations are given in Appendix A.

Chapter Summary (1/3): Graph Construction and Model

Purpose: Define how the electrical state becomes the input of each actor and how that input is encoded.

- **Input processing:** Continuous physical quantities can be divided by fixed physical scales and optionally standardized from training statistics. The loading ratio `rho`, binary features, and masks remain unchanged. The graph-screening experiments use raw features; the cross-grid transfer study uses deterministic physical scaling.
- **Dynamic topology:** Nodes and candidate edges remain fixed, while time-dependent edge and node masks retain only the relations and contextual nodes that are electrically connected to the controlled region. An inactive contextual candidate has a zero feature row and participates in neither message passing nor readout.
- **Graph representations:** The `bus` schema aggregates equipment on busbars; `heterogeneous` gives generators and loads their own nodes; and `heterogeneous_line` also represents each transmission line as a node. This changes both the feature location and the number of message-passing steps between physical objects.
- **Actor and critic:** Each actor receives its controlled region and only the schema-specific boundary context of Table 4.10. The reference architecture uses a two-layer, 128-feature GINE or GAT encoder shared across agents, followed by an agent-specific action head. The centralized critic remains a separate MLP over the normalized full-grid flat state.

Takeaway: The graph variants change both how state is represented and how much neighboring information is available. The MAPPO training structure and agent partition remain unchanged, but the three representations must not be described as information-equivalent.

4.3 Substation Representation: Direct Edges and Summary Nodes

Two optional factors change how information is exchanged inside a substation. The factor e adds direct same-substation busbar edges. The factor n adds one learned substation-summary node per included substation. Both augmentations are part of the constructed graph G_a shown at the left of Figure 4.4; their nodes and edges therefore enter the same node projection, edge filtering, and message-passing pipeline as the base graph.

4.3.1 Direct same-substation edges (e)

In addition to physical-line relations, the augmentation inserts $i(s, b) \to i(s, b')$ for every ordered pair $b \neq b'$ of busbars in the same included substation. The edges are therefore bidirectional between each pair of busbars, remain active throughout the episode, and do not represent an electrical connection. Figure 4.5 shows this relation for the busbar and disaggregated schemas.

Table 4.12 gives the exact feature values of the added edges. Every physical attribute is zero: a

relation that carries no flow reports none.

These edges are added inside neighboring substations too. The augmentation runs over every substation of the region, contextual ones included, so a neighboring substation's busbars are joined to each other exactly as controlled ones are. That interacts with the information boundary of Section 4.2.5 in a way worth making explicit.

A same-substation relation does *not* carry visibility. Grid2Op has no bus coupler: busbar assignment is the topology, and two busbars of one substation are separate electrical nodes. The relation is a computational device encoding that an element can be moved between them, which is a statement about an action space rather than about conductivity – and, for a contextual substation, about *another* agent's action space. The visibility rule therefore excludes it, together with self relations, when deciding whether a contextual node is connected to the controlled region.

The effect is measurable. Take agent 0 of `bus14` with contextual substation 3 split so that its 44.3 MW load sits on the busbar the agent's lines do *not* attach to:

Table 4.11: Status of the contextual busbar carrying 44.3 MW, under the two constructions and the two settings of this augmentation.

Construction	Same-substation edges	Visible	Active degree	Load seen
Earlier	off	yes	0	44.3
Earlier	on	yes	2	44.3
Current	off	no	0	0
Current	on	no	0	0

The second row is the important one. Under the earlier construction this augmentation upgraded the leak: without it the unreachable busbar was an isolated node contributing only to the readout, whereas with it the busbar became connected to its sibling and its load propagated into the agent's own region through message passing. The cells of Screen C that enable this factor are therefore the most affected by the construction they were run under. Under the current rule the node is invisible and its structural edges are inactive in both settings, so the augmentation changes message passing only among nodes the agent is entitled to see.

Being entitled to see two nodes is not a reason to relate them. One case survives the correction. When a contextual substation is split so that both of its busbars are attached to the agent by live lines, both are visible and the same-substation relation between them is active: on agent 0 of `bus14` with substation 3 split in this way, the pair of directed edges between its two busbars carries messages. This is not a boundary violation, since each endpoint is legitimately observable, but it is questionable on two other grounds.

The relation asserts that an element can be moved between the two busbars. That is a statement about an action space, and for a contextual substation it is a statement about *another agent's* action space, which this agent can neither exercise nor anticipate. The same reasoning already excludes structural relations from carrying visibility, so allowing them to carry messages between two contextual nodes is inconsistent with a decision already taken elsewhere in the construction.

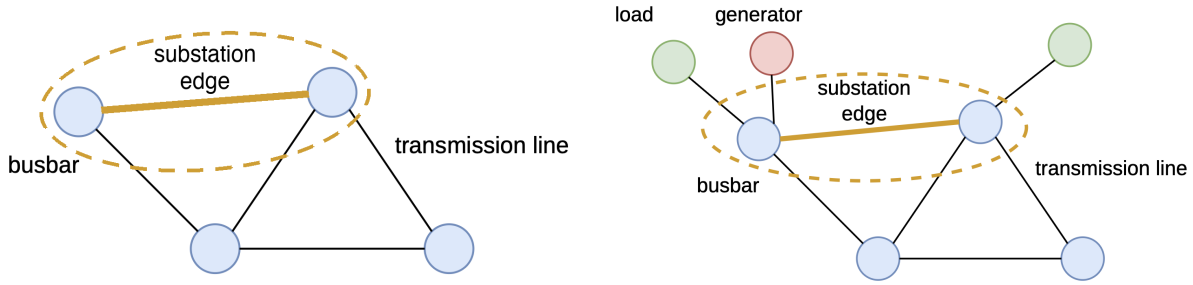
The physical objection is stronger. Two busbars of one substation are separate electrical nodes, and the relation is justified inside the controlled domain only because the agent can join them by acting. Between two contextual busbars it is neither electrical nor reachable by any action available here, and it creates a message path that crosses the boundary twice: information leaving the agent on one line can return on another through a coupling that does not exist, where the true path runs through the agent's own internal topology.

Both structural augmentations are therefore restricted to controlled substations. The fixed candidate shape does not prevent this, since the controlled substations are themselves a static set, and it also separates a factor that would otherwise be confounded: enabling the augmentation now changes relations inside the controlled domain only, rather than changing them there and between contextual nodes at the

same time. On agent 0 of `bus14` with substation 3 split as above, the number of active same-substation edges falls from ten to eight, and the two that related the neighbor's busbars to each other are gone.

Table 4.12: Node and edge features introduced by the direct same-substation augmentation. "None" means that the augmentation neither adds nodes nor changes the features of existing nodes.

Graph schema	Added node features	Feature values on each added directed edge
Busbar	None	All physical-line features are 0.
Disaggregated	None	`relation_same_substation_busbar`= 1; all other entries are 0.
Disaggregated with line nodes	None	`relation_same_substation_busbar`= 1; the other eight relation entries are 0.



(a) Busbar graph.

(b) Disaggregated graph.

Figure 4.5: Direct same-substation busbar relation, shown in the two schemas of Section 4.2. The highlighted orange edge allows two busbars belonging to the same substation to exchange messages; it is a computational relation and not a physical transmission line. In (a) the generator and load quantities are aggregated into the busbar features, whereas in (b) they are separate nodes. The relation is identical in both cases: it connects two busbars of one substation and does not depend on the node types attached to them.

4.3.2 Substation summary nodes (n)

In all experiments, augmentation n adds one learned summary node u_s for each included substation s and connects it in both directions to every busbar in $\mathcal{B}(s)$:

$$\mathcal{E}_{\text{sub}} = \{(i, u_s), (u_s, i) \mid i \in \mathcal{B}(s)\}\,; \tag{4.16}$$

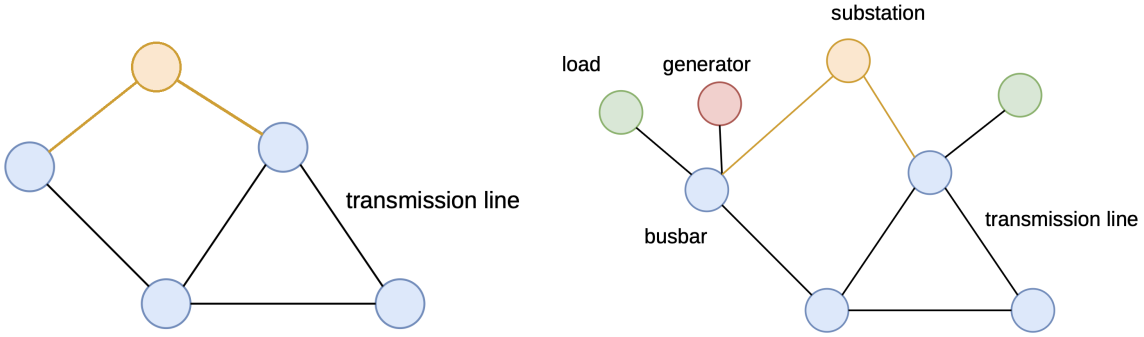
For S included substations with B busbars each, this adds S nodes and $2SB$ directed edges. Each u_s gathers information from its busbars and sends the result back, so two busbars in the same substation communicate in two steps: $i(s,b) \to u_s \to i(s,b')$. It therefore provides the same intra-substation communication role as the direct relation in Figure 4.5. Only busbars connect directly to u_s; other node types reach it through their busbar attachments. Figure 4.6 shows this construction.

Summary nodes exist only for controlled substations. The augmentation is restricted to the agent's own substations, for the reason given in Section 4.3.1: a summary node relates every busbar it gathers, and outside the controlled domain that relation is one the agent can neither act on nor make sense of. Without the restriction a neighboring substation would receive a summary node as well, and two of its busbars could exchange through it even with the direct relation removed, so both augmentations have to be restricted together for either restriction to mean anything.

The earlier behaviour was also close to inert. On agent 0 of `bus14` under the nominal topology, each controlled substation's summary node gathers both of its busbars, whereas a contextual one gathered exactly one. Summarizing a single node is an identity up to the learned transform: the round trip

$i(s,b) \to u_s \to i(s,b)$ returns to its origin while consuming two message-passing steps, the entire depth at the two-layer setting used throughout, and adds a learned constant row to an ordinary mean readout.

Two consequences are worth recording. Summary nodes are appended by the encoder after visibility has been decided, and they gather only busbars that are both controlled and currently visible, so no contextual state reaches the agent by this route under either setting. And because the virtual-node readout attaches to summary nodes where they exist, contextual busbars no longer reach the graph summary directly; they influence it through the controlled busbars they are connected to, consistent with reading the graph embedding as a description of the region the agent controls.



(a) Busbar graph.

(b) Disaggregated graph.

Figure 4.6: Learned substation-summary augmentation, shown in the two schemas of Section 4.2. The orange substation node receives messages from each of its blue busbar nodes and redistributes the resulting substation-level representation through the reverse edges. In (a) the generator and load quantities are aggregated into the busbar features, whereas in (b) they are separate nodes; in both cases the summary node attaches only to busbars, so generators, loads, and transmission-line nodes reach it through their busbar attachments.

After the raw node projection in Figure 4.4, the model initializes u_s from a *structural descriptor* of substation s rather than from its identity. Let $\boldsymbol{\phi}_s \in \mathbb{R}^4$ collect the number of busbars, incident lines, attached generators, and attached loads of that substation, each divided by a constant that is the same on every grid. The summary node starts at

$$u_s^{(0)} = \mathbf{W}_{\text{sub}}\,\boldsymbol{\phi}_s + \mathbf{b}_{\text{sub}}, \qquad \mathbf{W}_{\text{sub}} \in \mathbb{R}^{d_h \times 4},\ \mathbf{b}_{\text{sub}} \in \mathbb{R}^{d_h}, \tag{4.17}$$

and carries no current grid measurement. Messages from the busbars then produce an observation-dependent state for u_s, while PPO learns $\mathbf{W}_{\text{sub}}$ and $\mathbf{b}_{\text{sub}}$ with the other parameters.

The descriptor replaces what would be the obvious construction, a trainable matrix $\mathbf{Q} \in \mathbb{R}^{S \times d_h}$ whose row s is selected by substation identifier. That version cannot cross grids: its first dimension is the number of substations, so a checkpoint trained on a 14-substation network carries a $14 \times d_h$ matrix that no 36-substation model can accept. Transfer does not degrade quietly in that case, it stops – the loader treats the mismatch as an architecture difference and refuses, since the shape is not one of the graph-describing buffers it is allowed to rebuild. The augmentation would therefore have been unusable in exactly the setting Chapter 7 is about, and a design screened on `bus14` could not have been carried to the larger grid.

Equation 4.17 removes that, because both learned tensors are shaped by the descriptor and not by the network. It also keeps what the identity version was useful for. The bias $\mathbf{b}_{\text{sub}}$ is a single vector shared by every substation, which is what an identity-free design would give on its own; the weight $\mathbf{W}_{\text{sub}}$ adds a modulation by what the substation is made of, so a busy junction and a radial end point do not start from the same state. The quantities involved are counts rather than physical magnitudes, so the same descriptor means the same thing on any network, which is what makes the projection transferable at all.

Table 4.13: Node and edge features introduced by the substation-summary augmentation. Here d_h is the hidden node width; $d_h = 128$ in the screened GINE models.

Graph schema	Added summary-node state	Feature values on $i \leftrightarrow u_s$
Busbar	$\mathbf{W}_{\text{sub}}\boldsymbol{\phi}_s + \mathbf{b}_{\text{sub}} \in \mathbb{R}^{d_h}$, from the substation's structural descriptor; no measured feature.	All physical-line features are 0.
Disaggregated	Same structural projection; no measured feature.	All physical and relation entries are otherwise 0.
Disaggregated with line nodes	Same structural projection; no measured feature.	All nine relation entries are 0.

In runs with mean readout, each u_s is marked as a valid node and is included once in the mean together with all other valid nodes. In runs with a virtual node, there is no terminal mean: the virtual node connects to the summary nodes, and its final state is used as the graph representation.

The summary-node and direct-edge augmentations may also be enabled together, and either may be combined with the virtual-node readout.

4.4 Pooling and Graph Readout

The final graph embedding is obtained either by pooling node embeddings directly or by reading out a learned virtual node. The output projection and action head are the same in both cases.

4.4.1 Mean, maximum, and sum pooling

Mean pooling averages the final embeddings of the valid nodes:

$$\overline{\mathbf{h}}_G = \frac{\sum_v m_v \mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \tag{4.18}$$

where $m_v = n_{v,t}^{(a)}$ is the visibility mask. A single mean is computed over all visible node types; there is no separate pooling operation for each type. An inactive fixed-shape candidate contributes neither an embedding to the numerator nor one unit to the denominator. The mask is therefore an additional enforcement of the construction boundary, not an averaging of zero placeholders.

Maximum pooling instead selects the largest value independently in every hidden feature channel:

$$\left(\overline{\mathbf{h}}_G^{\max}\right)_c = \max_{v\,:\,m_v=1} \left(\mathbf{h}_v^{(K)}\right)_c . \tag{4.19}$$

It therefore keeps the strongest activation in each channel without depending on the number of nodes, but it does not preserve how many nodes produced a similar activation.

Sum pooling is available but is not used in any reported experiment. Its magnitude grows with the number of nodes. That number differs across agents, graph representations, and grids, so the scale of a sum-pooled embedding would also change across the transfer studied in Chapter 7.

For ordinary mean, maximum, or sum pooling, the pooled set is every *visible* node in the agent's local graph. It contains controlled nodes, including empty controlled busbars and disconnected controlled assets; the currently connected far-end busbars in `bus` and `heterogeneous`; and all boundary/internal line nodes in `heterogeneous_line`. It never contains an inactive contextual candidate, a neighboring private asset, or any neighboring equipment in the explicit-line schema. Valid substation-summary nodes are included when that augmentation is enabled. Direct same-substation edges add no nodes and do not change the pooled set.

The alternatives `controlled_mean`, `controlled_sum`, and `controlled_max` restrict terminal readout to the action domain. For controlled mean, define $q_v = m_v c_v$, where c_v is the static `controlled_node_mask`; then

$$\overline{\mathbf{h}}_G^{\text{ctrl}} = \frac{\sum_v q_v \mathbf{h}_v^{(K)}}{\max(1, \sum_v q_v)}. \tag{4.20}$$

Contextual busbars can still affect the controlled embeddings through active message-passing paths, but do not enter the final aggregation directly. The controlled pooled set is fixed for an episode, whereas the ordinary visible set can change when a neighboring line switches busbar or goes offline. This stabilizes the denominator, but it is a readout choice rather than the leakage fix: the graph construction and visibility masks enforce the information boundary for every pooling mode.

The optional `energized_mean` readout further requires a node to be incident to an active electrical relation; self and same-substation relations do not qualify. `controlled_energized_mean` additionally intersects this dynamic mask with the controlled-node mask. These masks affect only the terminal mean and its denominator, not message passing. They are therefore pooling ablations, not leakage fixes. Virtual-node runs do not use terminal pooling and are described in Section 4.4.2.

Which of the four is the defensible default. The readout produces the single vector the action head reads, and in the candidate-scoring head it is the global context concatenated to every action. It should therefore answer one question: what is the state of the region this agent is deciding about? That region is the action domain, which makes `controlled_mean` the appropriate default. Excluding context from the average does not discard it, because it arrives through message passing, which is what the graph is for: on a three-substation fixture the `controlled_mean` embedding differs across topologies that differ only in their context. The choice is not whether context informs the summary but whether it does so as structure or as an addend, and structure is the answer that uses the model rather than bypassing it.

The mechanical argument is the stronger one. A fixed pooled set makes any systematic offset – including the zeros contributed by empty controlled busbars – a constant that the readout projection absorbs during training. A pooled set whose size moves turns that offset into a quantity varying with topology, which the network must then separate from genuine signal. On agent 0 of `bus14` the visible set holds ten nodes under the nominal topology, eleven once a contextual substation splits, and nine after a boundary line trips, so an ordinary mean rescales every node it keeps when a *neighbouring* agent acts.

The energised variants are the least defensible of the four, because they remove precisely what actions target. Four of agent 0's eight busbars are de-energised under the nominal topology: the empty second busbar of each substation, which is the busbar a splitting action moves an element onto. A readout that drops them cannot represent the availability of a free busbar, which is the most directly actionable fact in the observation, and its denominator moves with every switch.

Two qualifications. When one-hop context is disabled the controlled and visible sets coincide, so the distinction is vacuous and the ordinary readout is already the controlled one. And maximum pooling is less exposed throughout: it does not depend on the size of the pooled set, and a de-energised row of zeros seldom wins a maximum, so the energised variants matter far less there. Its own risk is different – with context included, a neighbour under greater stress dominates the channel maxima, so the graph summary reports the neighbour's worst channel rather than the agent's, which is an argument for `controlled_max` on the same grounds.

This also sharpens the reason sum pooling is set aside. The objection above is that its magnitude follows the number of pooled nodes, and under a controlled readout that number is fixed for the episode, so the objection would dissolve. What remains is that the controlled sets differ *between* agents – eight, eight and twelve busbars on `bus14` – while the encoder is shared, so a sum would hand different agents systematically different magnitudes.

4.4.2 Virtual-node graph readout

A virtual-node readout replaces terminal mean, sum, maximum, or attention pooling with one learned graph-summary node. It can be combined with any of the three graph schemas.

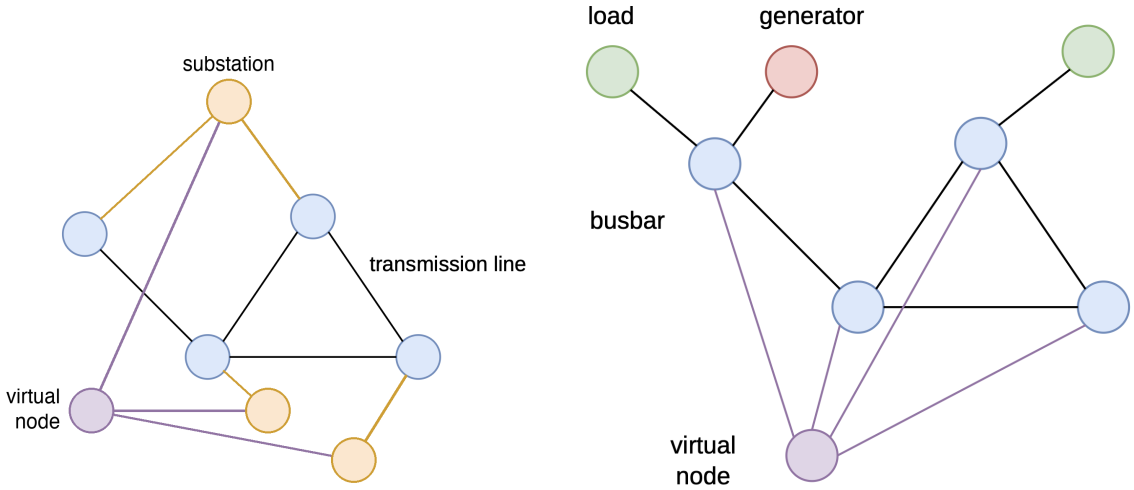
Augmented graph

Let $v_\star$ denote the virtual node. Define the summary set $\mathcal{V}_{\text{sum}}$ as the included substation nodes when the augmentation from Section 4.3.2 is enabled, and as the included busbar nodes otherwise. The augmented graph is

$$\mathcal{V}^+ = \mathcal{V} \cup \{v_\star\}, \tag{4.21}$$

$$\mathcal{E}^+ = \mathcal{E} \cup \{(i, v_\star) \mid i \in \mathcal{V}_{\text{sum}}\}. \tag{4.22}$$

All reported experiments use a toward-summary direction: every added edge points from a node in $\mathcal{V}_{\text{sum}}$ to $v_\star$. The virtual node therefore receives information but never sends messages back. It connects to all visible busbars, or to all visible substation-summary nodes when that augmentation is enabled, and has no direct edge to generator, load, or transmission-line nodes. Candidate virtual edges incident to an inactive contextual row are masked, so the virtual readout cannot bypass the local information boundary.



(a) With substation-summary nodes.

(b) Without substation-summary nodes.

Figure 4.7: Virtual-node graph readout, shown for both definitions of the summary set $\mathcal{V}_{\text{sum}}$ in Equation 4.22. The purple node aggregates that set and its final embedding replaces terminal node pooling. Every purple edge is directed toward the virtual node, which sends no message back. In (a) the substation-summary augmentation of Section 4.3.2 is enabled, so the virtual node attaches to the orange substation nodes and reaches the busbars only through them, which gives a two-level hierarchy busbar → substation → virtual node. In (b) the augmentation is disabled and the virtual node attaches directly to every included busbar. In neither case does it connect to generator, load, or transmission-line nodes.

Learned summary instead of terminal pooling

The initial virtual representation $\mathbf{h}_\star^{(0)}$ is learned. A message-passing layer jointly updates the physical and virtual nodes:

$$\left(\{\mathbf{h}_i^{(k+1)}\}_{i\in\mathcal{V}}, \mathbf{h}_\star^{(k+1)}\right) = \text{GNN}^{(k)}\left(\{\mathbf{h}_i^{(k)}\}_{i\in\mathcal{V}}, \mathbf{h}_\star^{(k)}, \mathcal{E}^+\right). \tag{4.23}$$

Because every virtual edge points toward $v_\star$, it only aggregates the node states and cannot influence them during message passing. After K layers, the graph embedding is

$$\mathbf{z}_G = \text{ReLU}\left(\mathbf{W}_o \mathbf{h}_\star^{(K)} + \mathbf{b}_o\right), \tag{4.24}$$

rather than from $\text{Pool}(\{\mathbf{h}_i^{(K)}\}_{i\in\mathcal{V}})$. The output dimension is unchanged.

Relation attributes and encoder compatibility

Virtual connections are structural: their continuous edge attributes are zero, except for a neutral unit value in the `rho` position when that feature exists.

Without substation nodes, a graph containing N_{bus} included busbars receives one virtual node and N_{bus} directed edges. With the hierarchy enabled, the virtual-node stage instead adds S edges from the S substation-summary nodes. After one layer the virtual node has aggregated its immediate busbar or substation neighbors.

Chapter Summary (2/3): Augmentations and Graph Readout

Purpose: Separate changes to information exchange inside the graph from changes to the final graph-level representation.

- **Direct substation edges (e):** Add structural relations between busbars of the same substation, without adding nodes or representing a physical transmission line.
- **Substation nodes (n):** Add one learned node per substation. It has no raw electrical measurement, is updated from its busbars, and can be included in the graph readout.
- **Terminal pooling:** Ordinary mean pooling averages every visible node, including electrically connected contextual busbars and valid added substation nodes; masked contextual candidates are excluded. A `controlled_*` readout instead aggregates only the fixed action-domain nodes, while still allowing visible context to influence them through message passing. Maximum pooling keeps the largest value in each hidden channel. Sum pooling is not used because its scale changes with the number of nodes.
- **Virtual readout (v):** Replace terminal pooling with one learned grid-summary node. Messages travel only toward this node, from the busbars or from the added substation nodes, and its final state becomes the graph embedding.

Takeaway: Factors e and n modify message passing, whereas pooling and factor v determine how the encoded local graph is reduced to one vector for action selection. These choices can be combined with any base graph schema.

4.5 Candidate-Action Scoring from the Explicit-Line Graph

The default action head produces one output column per discrete action. That column is tied to an action index and knows nothing about the physical objects the action modifies. This section describes an alternative head that scores each action from the graph nodes it touches. It is available only for the explicit-line schema of Section 4.2.4 and is selected with `actor_action_head=candidate_pool`; the default remains the action-index head of Appendix A.5. The PPO objective, the categorical policy, sampling, and deterministic evaluation are unchanged.

4.5.1 From action indices to candidate scoring

Throughout this section, $\mathbf{z}_a$ denotes the graph embedding of agent a: the node states of its local graph pooled as in Section 4.4 and passed through the output projection (Equation A.10). It is one 128-dimensional vector per agent per step, and it is the only thing the actor has ever seen of the graph.

With the default head, agent a produces its logit vector in one step from $\mathbf{z}_a$:

$$\boldsymbol{\ell}_a = \mathrm{MLP}_a(\mathbf{z}_a), \qquad \boldsymbol{\ell}_a \in \mathbb{R}^{|\mathcal{A}_a|}. \tag{4.25}$$

The last layer has one row per action index, so what makes an action good must be learned separately for every index, and the head grows with $|\mathcal{A}_a|$.

Candidate scoring replaces that by one shared scalar-valued network applied to every action in turn:

$$\ell_{a,j} = \mathrm{score}\left(\underbrace{\mathbf{z}_a}_{\text{whole graph}}, \underbrace{\mathbf{c}_j}_{\text{nodes action } j \text{ touches}}, \underbrace{\boldsymbol{\phi}_j}_{\text{what action } j \text{ does}} \right). \tag{4.26}$$

The same parameters score all actions, so the head no longer grows with $|\mathcal{A}_a|$, and two actions are ranked by the difference between their touched-node contexts rather than by two independent output rows. This is what makes the explicit line nodes usable: an action next to a loaded line is scored from that line's own embedding.

Two ingredients are needed: a static table saying which nodes each action touches (Section 4.5.2), and a rule for turning those nodes into the context vector $\mathbf{c}_j$ (Section 4.5.3).

4.5.2 What each action touches

The first ingredient is one lookup table per agent, with one row per action, answering a single question: which nodes of the local graph does this action physically touch? It is built once, when the environment is created, and never recomputed during training.

A Grid2Op topology action states which entries of the topology vector it changes, and each entry belongs to a known object: a line endpoint, a generator, or a load. Reading those entries gives the objects the action modifies, and the graph builder's row maps turn each object into the row of the graph that holds its embedding. Table 4.14 follows one action through this.

Table 4.14: One action, from Grid2Op to node rows: moving the origin end of line 4 onto the other busbar of substation 6. Row indices are illustrative.

Step	Result for this action
Objects the action changes	Substation 6, line 4
Their rows in the graph	Both busbars of substation 6, here 12 and 13, and the line node of line 4, here 22
Stored for scoring	Busbar $[12, 13]$, line $[22]$, generator and load empty

Two decoding rules are worth stating. A touched substation contributes *all* of its busbar rows, not only the busbar in use, because the action changes which of them the equipment sits on. A boundary line contributes its explicit line-node row and the controlled endpoint rows that are present in the local graph; scoring does not add the far-end busbar or any neighboring private asset. The rows are stored as fixed-size arrays with a mask, padded to the widest action, so that a single gather serves the whole action space.

Write $\mathcal{N}_{\text{bus}}(j)$, $\mathcal{N}_{\text{line}}(j)$, $\mathcal{N}_{\text{load}}(j)$, and $\mathcal{N}_{\text{gen}}(j)$ for the four sets of node rows that action j touches. Because a topology change moves line endpoints, the explicit line node must be present whenever the line touches the agent's controlled domain. The leakage-free construction guarantees that membership without constructing the non-controlled endpoint.

Each action also carries a static feature vector $\boldsymbol{\phi}_j \in \mathbb{R}^{12}$, listed in Table 4.15. It describes what the action does independently of the current grid state, so the scorer can tell a single endpoint move from an action that reconfigures a whole substation even when both touch the same nodes.

Table 4.15: The static action-feature vector ϕ_j. The five indicators are binary; the seven counts are divided by their largest value over that agent's action space, so every entry lies in $[0, 1]$ and is independent of the grid size.

Entry	Meaning
Indicators	
`is_do_nothing`	The action is the idle action
`has_busbar_context`	It modifies at least one substation
`has_line`	It touches at least one line
`has_load`	It touches at least one load
`has_generator`	It touches at least one generator
Scaled counts	
`n_substations`	Substations modified
`n_topology_changes`	Topology-vector entries changed
`n_line_status_changes`	Lines connected or disconnected
`n_line_endpoint_changes`	Line endpoints moved to another busbar
`n_generator_changes`	Generators moved to another busbar
`n_load_changes`	Loads moved to another busbar
`n_other_changes`	Changed objects of any other type

4.5.3 Pooling the touched nodes

The second ingredient turns the touched rows into $\mathbf{c}_j$. In $\mathbf{z}_a$ the nodes have already been averaged away, so the encoder also returns the node states themselves:

$$(\mathbf{z}_a, \mathbf{H}) = \text{encoder}_{\text{nodes}}(\mathcal{G}_t^{(a)}), \qquad \mathbf{H} \in \mathbb{R}^{|\mathcal{V}| \times d_h}, \tag{4.27}$$

where row v of $\mathbf{H}$ is the state $\mathbf{h}_v^{(K)}$ of node v after the last message-passing layer. These are exactly the vectors that mean pooling averages into $\mathbf{z}_a$; nothing new is computed, and a run with the default head is unaffected. Only the physical nodes appear in $\mathbf{H}$: substation-summary and virtual nodes are excluded, since an action touches equipment and not a learned hub.

The basic operation is a masked mean over the rows of one object type,

$$\mathbf{c}_j^{(t)} = \frac{1}{\max\left(1, |\mathcal{N}_t(j)|\right)} \sum_{v \in \mathcal{N}_t(j)} \mathbf{h}_v^{(K)}, \tag{4.28}$$

where the clamped denominator makes an untouched type give the zero vector instead of a division by zero. Table 4.16 lists the ways of combining these.

Table 4.16: Pooling modes. All three are implemented; `typed_mean` is the default.

Mode	Context $\mathbf{c}_j$	Width	What it buys
`mean`	One masked mean over all touched rows, whatever their type	d_h	The smaller control; mixes roles together
`typed_mean`	The four typed means of Equation 4.28, concatenated	$4d_h$	The scorer can tell a stressed line from the load beside it
`typed_attention`	The action queries the eligible rows and weights them, typed as above	$4d_h$	Can focus on the one endpoint that matters instead of averaging it away

Learned attention replaces the flat average by weights the action computes for itself, and `candidate_action_attention_sc` decides which rows compete for that weight: `affected` keeps the touched rows of the two means above, `all` opens the attention to every node of the local graph, and `soft_prior` opens it to every node but biases the touched ones, so the head can look beyond what the action modifies without being forced to. The uniform modes are still the controls, because attention adds parameters and a second failure mode to a head whose value is being established at the same time.

4.5.4 Scoring, including the idle action

All non-idle actions are scored by one network with a single output unit:

$$\ell_{a,j} = \mathrm{MLP}_{\mathrm{score}}\left(\left[\mathbf{z}_a; \mathbf{c}_j; \boldsymbol{\phi}_j\right]\right), \qquad j \geq 1. \tag{4.29}$$

The idle action touches nothing, so its pooled context is identically zero and would otherwise be scored by a network trained on non-zero contexts. It therefore gets its own head over the graph embedding alone:

$$\ell_{a,0} = \mathrm{MLP}_{\varnothing}(\mathbf{z}_a) + b_{\varnothing}, \tag{4.30}$$

with $b_{\varnothing}$ a learned scalar applied to action 0 alone. This makes the idle logit an explicit judgment of whether the local grid state is safe enough not to intervene.

4.5.5 Cost and current limits

The candidate contexts form a tensor of shape $[\text{batch}, |\mathcal{A}_a|, d_c]$, so memory grows with the size of the action space. This is the reason to start on `bus14` and on reduced action spaces before the full WCCI one.

Two properties matter when comparing this head against the default. The head no longer scales with $|\mathcal{A}_a|$, so the two actors do not have the same parameter count even at identical graph settings, and both counts should be reported. The idle logit also comes from a different mechanism than the other logits, so the action-0 frequency has to be compared against the action-index baseline rather than assumed comparable.

The head currently requires the explicit-line graph and does not combine with the intervention gate, which is refused at construction rather than silently ignored. One-hop context is not required when using the angle-drop representation: the neighbor-inclusion setting adds no far-end equipment to this representation, because the shared-line state is supplied by the line node itself. Auxiliary heads over the same node embeddings, such as predicting which lines are about to overload, are a possible later addition.

4.6 Action-Space Reduction for Large Grids

The representations above change what an agent sees. This section changes what it can do, and is what makes the larger grids tractable at all: the WCCI bus36 setting has a far larger discrete topology-action space than `bus14`, so before training MAPPO on it the action space is reduced offline from simulated action outcomes. During rollouts, states are collected when the grid is stressed,

$$\rho_{\max} > 0.90.$$

For each collected state s, candidate actions are simulated and compared against do nothing:

$$\Delta(s, a) = \rho_{\mathrm{after}}(s, a) - \rho_{\mathrm{after}}(s, 0).$$

An action is counted as useful when it lowers the post-action loading relative to do nothing, i.e. $\Delta(s, a) < -\epsilon$. Each action is then scored by its empirical improvement rate,

$$\mathrm{score}(a) = \frac{\#\{\text{tested states where } a \text{ improves the grid}\}}{\#\{\text{tested states where } a \text{ is evaluated}\}},$$

with a minimum-count filter to avoid selecting rarely sampled actions. The final reduced action space keeps the best actions per agent, while always preserving action 0. The candidate sets are then sanity-checked with a one-step simulator-greedy controller: at each risky state, the controller simulates the available reduced actions and executes the best improving one. That check also sizes the reduced space, since it shows how much survival is still reachable once the action set is cut. Table 4.17 reports it for five sizes; the survival of trained agents on this environment is a separate question and is not covered here.

Table 4.17: One-step greedy evaluation of WCCI reduced action spaces on 50 random test chronics. All rows use the same chronic sample (`chronic_sample_seed=0`); the do-nothing mean survival on this sample is 28.30%.

Reduced action space	Kept actions per agent (a_0, a_1, a_2, a_3)	Greedy survival (%)	Do-nothing survival (%)	Survival delta (%)	Greedy full survival (%)
Do-nothing reference	$1, 1, 1, 1$	–	28.30	–	–
WCCI $h = 1$, `mk64`	$64, 64, 64, 64$	55.24	28.30	26.94	22.00
WCCI $h = 1$, `mk128`	$77, 128, 127, 128$	60.65	28.30	32.35	22.00
WCCI $h = 1$, `mk256`	$77, 256, 127, 256$	68.61	28.30	40.31	36.00
WCCI $h = 1$, `mk512`	$77, 512, 127, 512$	71.21	28.30	42.90	40.00
WCCI $h = 1$, `mk1024`	$77, 1024, 127, 1024$	78.18	28.30	49.87	54.00

Chapter Summary (3/3): Action Scoring and Large-Grid Reduction

Purpose: Define how graph embeddings become action logits and how the large WCCI action space is made tractable.

- **Default action head:** An agent-specific MLP maps the graph embedding directly to one logit per discrete action.
- **Candidate-action head:** The optional explicit-line variant scores each non-idle action from the encoded nodes it modifies and from static action descriptors. The idle action has a separate score. The PPO objective and categorical policy are unchanged.
- **Current scope:** Candidate scoring requires the explicit-line graph, and its memory cost still grows with the number of candidate actions. One-hop context is optional, since the line node already carries the shared measurements.
- **WCCI reduction:** Actions are ranked offline by how often they reduce loading in simulated stressed states. Action 0 is always retained, and the reduced sets are checked with a one-step simulator-greedy controller before policy training.

Takeaway: Graph representation, action scoring, and action-space size are separate design choices. This chapter defines them; the following chapters evaluate the resulting policies.

Chapter 5

Graph-Design Screening

This chapter reports the graph-design screens. All are balanced full factorials on `bus14` with three seeds per cell, and all ask the same kind of question: does a specific choice in the actor of Chapter 4 change how well a decentralized MAPPO policy survives on held-out chronics?

Information-boundary status of the reported screens

These runs predate the corrected local-graph construction of Section 4.2.5. Their fixed candidate graphs marked all rows of a neighboring substation as visible, including unattached busbars and, in heterogeneous graphs, neighboring private assets. Those isolated rows could enter graph readout even when no live boundary line connected them to the agent. The results in this chapter therefore characterize the *legacy, information-leaking observation* and must not be presented as strictly comparable to checkpoints trained with the corrected boundary. Screening conclusions must be rerun or at least confirmed under the leakage-free construction before being carried forward.

- **Screen A** varies the depth and width of the actor encoder, which applies to every graph schema: a 3×4 grid, 36 runs. *Seed 0 complete, seeds 1 and 2 training.*
- **Screen B** varies the message-passing operator and the readout, GINE against GAT and mean against maximum pooling: a 2×2 factorial, 9 new runs plus one reused cell. *Training at the time of writing.*
- **Screen C** varies the three structural augmentations of the busbar graph – same-substation edges, substation-summary nodes, and virtual-node readout (Sections 4.3 and 4.4.2): a 2^3 factorial, 24 runs. *Complete.*
- **Screen D** varies the direction of the generator and load attachment relations in the disaggregated graph (Section 4.2.3): a 3×3 factorial, 27 runs. *Complete.*

Screens A, B, and C all run on the busbar graph and share one reference configuration, so they can be read as a sequence; only Screen D changes the schema. Screens C and D are complete, 51 finished runs between them; Screen A has its first seed.

5.1 How This Chapter Is Organized

Each screen is presented as one block containing its design and its outcome together:

1. **Question.** Which design choice is uncertain?
2. **Design.** Which factors are varied, at which levels, and what is held fixed.
3. **Result.** The measured tables.
4. **Reading.** What follows, and what does not.

5.2 Shared Protocol

Every screen uses the configuration in Table 5.1. Every setting not named as a screened factor is identical across all runs, so each screen is a paired comparison against its own baseline cell.

Table 5.1: Settings shared by all screens. Only the graph type and the screened factors differ between them.

Component	Fixed setting
Environment	`bus14`, difficulty 0, topology actions, decentralized agents, normalized observations and rewards, 80/20 train/test chronic split with split seed 0
Actor	One GNN encoder shared by all agents; LayerNorm; node pre-encoder; two 128-unit action-head layers; legacy whole-substation one-hop context (information-leaking)
Critic	Centralized MLP with three 256-unit hidden layers
Graph preprocessing	None: neither deterministic physical scaling nor running standardization
Training budget	15,000,000 environment steps, 72 parallel environments, 576 rollout steps
MAPPO update	10 epochs, 16 minibatches, actor and critic learning rates 10^{-4} with linear annealing, $\gamma = 0.9$, $\lambda = 0.95$, clip 0.2, target KL 0.02, entropy coefficient 0.01
Exploration	No initial do-nothing prior and no action-0 logit bonus
Sparse control	Disabled: no intervention gate, no intervention penalty, no evaluation-time rho heuristic
Evaluation	Deterministic evaluation every 82,944 environment steps, 10 episodes on each split
Seeds	0, 1, 2

5.2.1 Endpoint statistics

Survival and the train/test split are as defined in Section 3.1: survival is the percentage of an episode's maximum length that the agent keeps the grid alive, reported on the test split. What differs here is how a run is reduced to one number. Each run saves the checkpoint with the highest test survival seen during training. That checkpoint is then re-evaluated deterministically over all 201 held-out test chronics.

Every run reaches the 15M-step budget, to within a negligible margin, so the screens do not report how far their runs got. Each cell of a screen aggregates its seeds over the same 201 chronics.

5.2.2 Compute

The runs were executed on two clusters, JED (CPU) and IZAR (GPU); seeds of the same cell are spread across both, so the compute backend is not confounded with any screened factor.

Training is deterministic at a fixed seed. Two separate launches of one configuration produced the same best checkpoint, at the same step, with the same full-test survival to full precision. Run-to-run variation at a fixed seed is therefore zero, and every difference reported in this chapter comes from the configuration or from the seed.

What limits the campaign is simulation throughput rather than model size. A training step is dominated by stepping 72 Grid2Op environments, so the actor forward and backward passes are a small part of the wall clock: across the depth and width grid of Section 5.3 the actor varies by a factor of three in parameters, 83.6k to 244.9k, without a comparable effect on run time. Parameter count is therefore not the cost to minimize when choosing an architecture here.

5.3 Screen A: Encoder Depth and Width

Question

We need to fix a message-passing depth K and an encoder width d_h. They apply to every schema, and every other screen in this chapter holds them constant while varying something else, so the pair has to be defensible before any of those comparisons mean anything. This screen asks which pair we should standardize on.

The two factors do different work on the busbar graph, which is the only schema this grid uses. Depth is a question of reach: with K layers a node embedding sees K hops away, and here one hop already crosses a transmission line into an adjacent substation. A local actor graph extends only one substation beyond the controlled ones, so by two layers a controlled busbar has already reached the edge of its own graph and a third layer mostly re-mixes what the second collected. Width is a question of capacity rather than reach: it sizes every node embedding and the pooled vector the action head consumes, and it is what limits how much of the local state survives pooling.

Because the grid runs on the busbar graph alone, what it measures is depth and width for that schema; carrying the chosen pair over to a schema with explicit asset or line nodes is an assumption rather than a measurement.

Design

A complete 3×4 grid with three seeds per cell, 36 runs. Table 5.2 lists the screened parameters and their levels. The graph output dimension is matched to the hidden dimension, so a cell varies one width rather than two. Every other setting is inherited from the reference configuration of Section 5.2, seed by seed.

Table 5.2: Screen A. Screened parameters and the values tested. All other settings are held at Table 5.1.

Parameter	Setting	Values tested
Message-passing layers K	`gnn_layers`	1, 2, 3
Encoder width d_h	`gnn_hidden_dim`	16, 32, 64, 128

Twelve of these runs have finished so far: seed 0 of each of the twelve architectures. Everything below therefore rests on one run per cell, so a difference between two cells cannot yet be separated from the variation another seed would have produced.

Result

Table 5.3: Screen A. Full-test survival of the best checkpoint, one seed per cell, by message-passing depth and encoder width. The reference depth and width used by every other screen is `mp2_h128`.

	Encoder width d_h			
Layers K	16	32	64	128
1	**98.68**	97.07	96.54	84.18
2	84.19	67.39	68.38	96.66
3	20.50	93.39	92.97	100.00[†]

[†]Best checkpoint selected at 0.17M steps; see the reading.

Table 5.4: Screen A. Marginal effect of each factor, averaged over the other. Four runs per depth, three per width.

Level	Mean (%)	Min (%)	Max (%)
Message-passing depth			
$K = 1$	94.12	84.18	98.68
$K = 2$	79.15	67.39	96.66
$K = 3$	76.71	20.50	100.00
Encoder width			
$d_h = 16$	67.79	20.50	98.68
$d_h = 32$	85.95	67.39	97.07
$d_h = 64$	85.96	68.38	96.54
$d_h = 128$	93.61	84.18	100.00

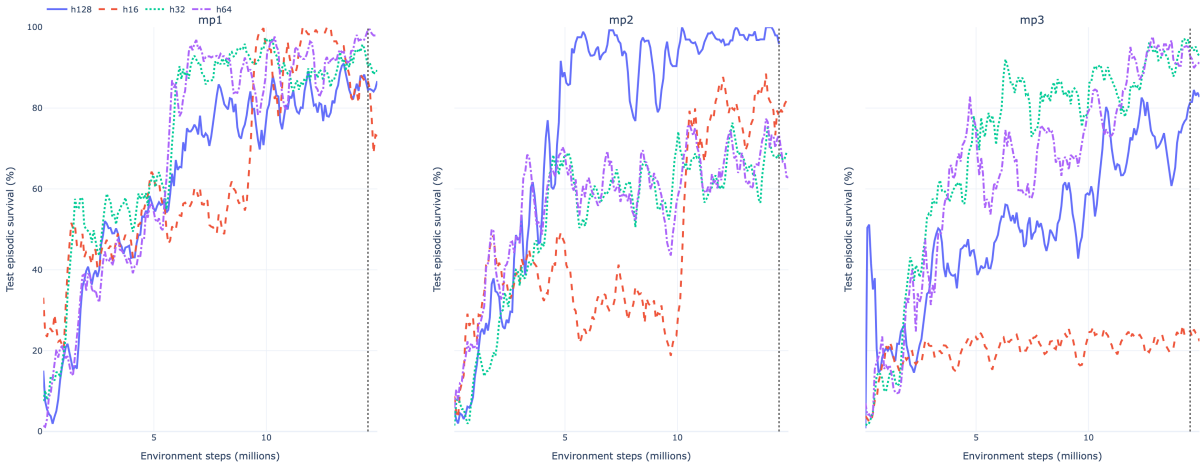


Figure 5.1: Screen A. Test episodic survival during training, one panel per message-passing depth, with the four widths compared inside each panel.

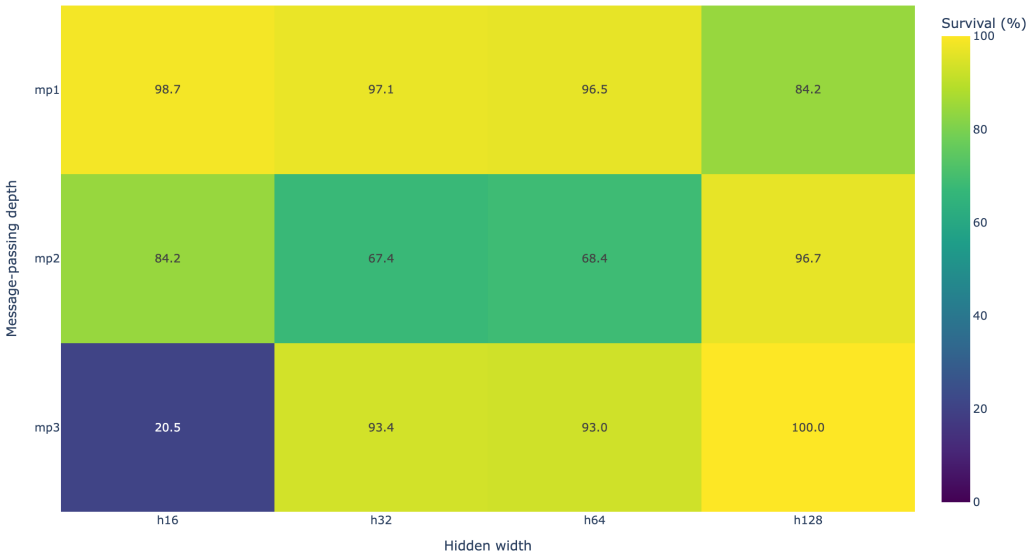


Figure 5.2: Screen A. The same twelve cells as a heatmap of the full-test survival of each run's best checkpoint.

Figure 5.3: Screen A. Full-test survival against actor parameter count on a logarithmic axis, joined within each depth. The grid spans 83.6k to 244.9k actor parameters, a factor of three.

Reading

One cell is unexplained. `mp3_h128` survives all 201 test chronics, the only perfect score measured anywhere, from a checkpoint saved after 0.17M steps. The surrounding facts are unusual rather than obviously wrong: the 10-episode evaluation at step 165,888 returned exactly 1.000, that checkpoint was kept as the run's best, and no later evaluation matched it, the same run ending training between 0.63 and 0.85. The score itself is a deterministic evaluation over the complete held-out split, so it is not a small-sample accident of the kind a 10-episode evaluation is prone to: a policy trained for 0.17M steps did survive every test chronic. Why an early checkpoint should generalize better than anything the same run reached later is not something this screen can answer, and it is worth returning to. Until it is explained, the cell is reported but is not used to argue for depth or width.

At the opposite extreme, `mp3_h16` reaches 20.50%, within a quarter of a point of a policy that never acts. That run did not learn, so its cell measures inaction rather than the effect of depth.

Depth does not separate cleanly on the endpoint statistic. The $K = 1$ row averages 94.12%, above $K = 2$ at 79.15% and $K = 3$ at 76.71%, but the three rows overlap heavily: the best and the worst cell of the whole grid, `mp3_h128` and `mp3_h16`, are both at $K = 3$, and the $K = 1$ row spans 84.18% to 98.68%. A row mean built from four single-seed cells with that much internal range does not establish an ordering, so the depth marginal is an observation here and not a result.

Width behaves more simply: wider is safer. Averaged over depth, survival rises at every step in width, 67.8%, 86.0%, 86.0%, 93.6%. The clearer way to read it is the worst case rather than the average. The weakest cell at $d_h = 16$ survives 20.5%, at $d_h = 32$ it is 67.4%, and at $d_h = 128$ it is 84.2%. Wide encoders are not dramatically better when a run goes well; they are much better when it goes badly, and the narrowest ones sometimes fail to learn at all.

The endpoint statistic is strong evidence, but not the only evidence. Each value is a deterministic evaluation over all 201 test chronics, which makes it far more reliable than any single training-time evaluation. What it does not account for is which checkpoint of a run is evaluated and which seed the run was trained from. Both move the number: the checkpoint is picked by a maximum over roughly 180 ten-episode evaluations, and with one seed per cell there is no second draw to average against. The training curves address the first of the two, since they use every evaluation of a run rather than the one that happened to be highest. Table 5.5 summarizes them three ways.

Table 5.5: Screen A. Convergence measured from the training curves rather than from a single checkpoint. Steps to 90% is the first evaluation at which the smoothed test survival reaches 90%; the curve mean is the time-average of the smoothed curve over training; the final-window mean averages the last ten evaluations. Dashes mark architectures that never reached the threshold.

Architecture	Steps to 90% (M)	Curve mean (%)	Final-window mean (%)
`mp2_h128`	**4.81**	**75.1**	**98.5**
`mp1_h32`	6.14	73.3	91.5
`mp1_h64`	6.55	69.9	98.0
`mp3_h32`	6.30	69.8	95.3
`mp1_h16`	9.46	64.6	81.3
`mp3_h64`	11.78	63.9	92.8
`mp1_h128`	13.35	63.2	86.0
`mp2_h64`	–	56.5	69.1
`mp2_h32`	–	52.6	68.3
`mp3_h128`	–	50.9	80.7
`mp2_h16`	–	46.7	79.4
`mp3_h16`	–	20.4	24.3

`mp2_h128` leads all three: it is the only architecture to reach 90% before 5M steps, it has the highest curve mean, and it ends training highest. Its one near-tie is the 80% threshold, which `mp3_h64` crosses 0.08M steps earlier. The reordering against Table 5.3 is itself informative: `mp1_h16` is first on the best-checkpoint statistic and sixth here, because its curve reaches 100% once and averages 81.3% at the end, while `mp3_h128` is first there and tenth here.

Which pair to standardize on. Three considerations point the same way. On the endpoint statistic the cells between 92% and 99% are close enough that one seed cannot order them, so that table alone does not select a winner. On the curves, which use every evaluation rather than the highest one, `mp2_h128` is the clearest and fastest learner of the twelve.

The third consideration is the grid itself. Two message-passing layers are what the `bus14` local graphs ask for: inside each agent's observation every pair of controlled substations lies within two line hops, with three exceptions in the largest region, that of agent 2, where three pairs are three hops apart. With $K = 2$ a controlled busbar can therefore integrate almost all of the region it acts on, whereas $K = 1$ leaves part of that region out of reach and $K = 3$ mostly re-mixes what the second layer already gathered. Depth is matched to the size of the observation rather than chosen for capacity, and since simulation throughput rather than parameter count limits the campaign (Section 5.2.2), the extra parameters of the wider encoder cost little.

The evidence therefore supports $(K, d_h) = (2, 128)$: it is the baseline for the screens in this chapter and the depth and width used for the rest of the study.

What this does not license. Choosing a default is one question; quantifying how much depth and width matter is another, and one seed per cell cannot answer the second. Wider encoders are visibly the safer ones and the $K = 1$ row is the most consistent on the endpoint statistic, but neither observation survives a single seed on its own, so neither is reported here as an effect. Seeds 1 and 2 would turn them into one, and would also show whether the perfect score of `mp3_h128` reappears or was particular to that run.

5.4 Screen B: Encoder Operator and Readout

Runs in progress. The design below is final; the coverage, result, and reading subsections are placeholders to be filled from the completed runs.

Question

Two independent choices have been held fixed at their defaults. The message-passing operator can be GINE, which adds the projected edge vector to the sender state inside every message, or GAT, which uses the edge vector to weight how strongly each incoming message counts (Appendix A). The readout can be the mean over valid nodes or a per-channel maximum (Section 4.4.1). The two interact in principle: a softmax-weighted aggregation followed by a maximum readout selects twice, while mean pooling after an unweighted sum preserves how many nodes are in a given state. Which combination suits the busbar graph?

The busbar schema is the relevant setting for a second reason. Its edge vector carries only the physical line block and no relation one-hot (Table A.3), so GAT's attention here is computed from line state — loading, status, cooldown — rather than from relation identity. If attention helps anywhere, a graph whose edges differ only by physical measurement is where it should show.

Design

A 2×2 factorial over encoder $\in \{\text{GINE}, \text{GAT}\}$ and readout $\in \{\text{mean}, \max\}$. To conserve computational resources, these initial exploratory runs were evaluated on a single seed ($s = 0$). Only three cells were newly launched — 3 runs in `configs/gnn_graph_screening/stage3_encoder_pooling` — because the GINE/mean cell is the configuration of Table 5.1 at the depth and width fixed in Section 5.3, and is reused rather than repeated. GAT uses four heads averaged rather than concatenated, so all four cells return the same 128 features. Every other setting is inherited from the reference configuration.

Reusing a cell would be a problem if a re-launch could disagree with the original, but training is deterministic at a fixed seed (Section 5.2.2), so the inherited numbers are exactly what a re-run would produce. Any difference against the three new cells is therefore a configuration difference and not launch noise.

Result

Table 5.6: Screen B. Test survival by encoder and readout for seed 0. The GINE/mean cell is inherited from the reference configuration.

	Readout	
Encoder	Mean	Max
GINE	96.66[†]	93.53
GAT	91.06	86.73

[†]Reference configuration, full-test survival of the best checkpoint; not a new run.

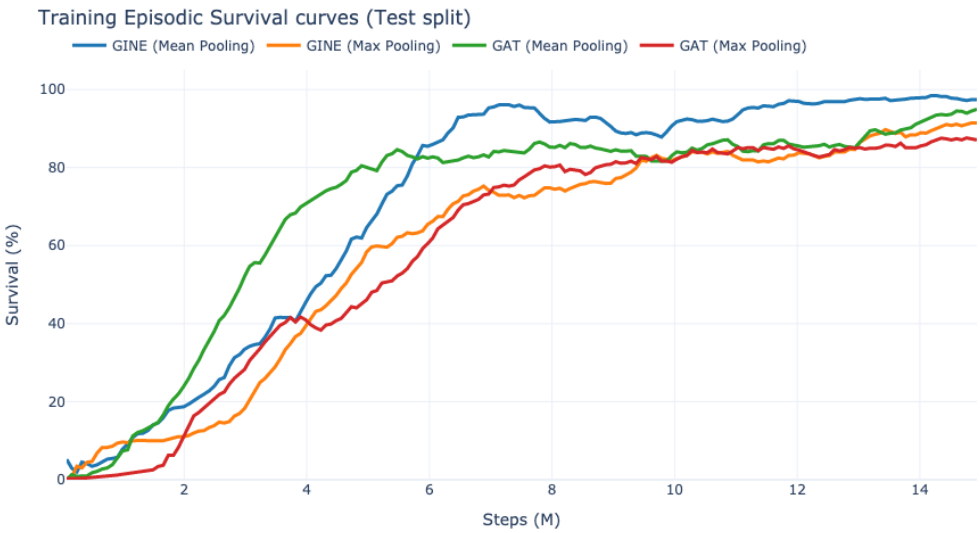


Figure 5.4: Screen B. Training episodic survival curves by encoder and readout configuration (Seed 0). The curves are heavily smoothed to reveal the underlying trends, showing a clear separation between the architectures.

Reading

[pending: report the four cells, then answer three questions.] The results show strong, independent main effects for both factors. Mean pooling consistently outperforms max pooling by a margin of 3 to 4 percentage points, regardless of the encoder used. Similarly, the GINE encoder strictly dominates GAT, maintaining a comfortable lead of roughly 5 to 7 percentage points over its attention-based counterpart.

Because the effects are independent and consistent, there is no complex interaction to untangle: GINE with mean pooling is simply the best combination for this environment. Furthermore, since GAT performs strictly worse despite its additional computational overhead from attention mechanisms, its cost is unjustified for the busbar graph. The inherited GINE/mean configuration remains our default and was not re-run, as the deterministic training environment guarantees identical results for the exact same seed.

5.5 Screen C: Structural Augmentations of the Busbar Graph

Question

Three optional augmentations add computational structure to the busbar graph without adding any physical measurement: same-substation edges (e), which let the busbars of one substation exchange messages directly; substation-summary nodes (n), which insert one learned hub per substation; and virtual-node readout (v), which replaces mean pooling with a learned graph-summary node. Each is motivated by the same intuition, shorten the path between related busbars, so the question is whether any of them earns its cost, and whether they compose.

Design

A complete 2^3 factorial with three seeds per cell, 24 runs. The graph type is `bus` throughout and the baseline cell is the unaugmented `e0n0v0`, which is the reference configuration derived from Section 5.3 and Section 5.4. Cells are named by their three switches, so `e1n0v1` means same-substation edges on, summary nodes off, virtual-node readout on.

Result

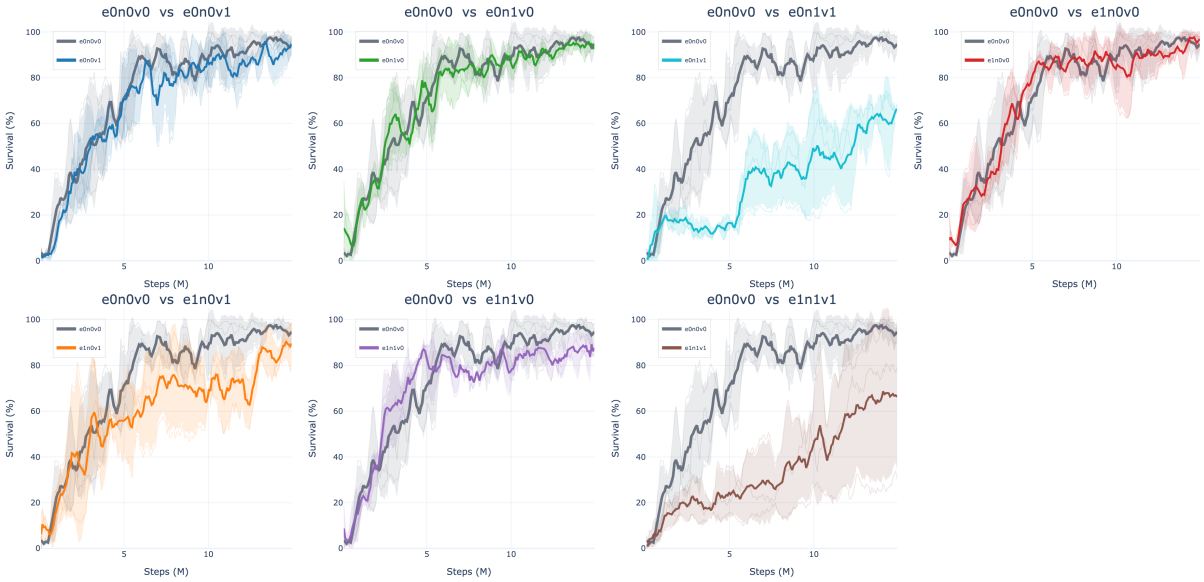


Figure 5.5: Screen C. Test episodic survival during training, one panel per augmented cell against the unaugmented `e0n0v0` baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.

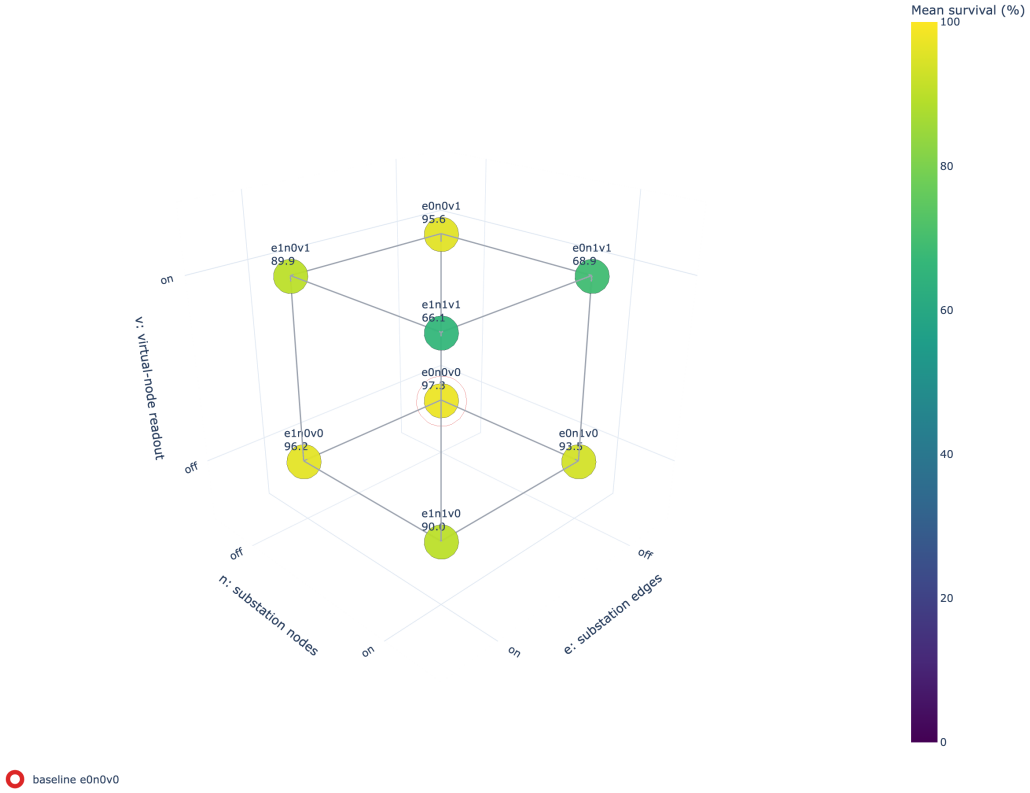


Figure 5.6: Screen C. The factorial as a cube: each vertex is one of the eight cells and each edge flips a single factor with the other two held fixed. The baseline `e0n0v0` is circled in red.

Figure 5.7: Screen C. The same eight cells as a heatmap of the full-test survival of each run's best checkpoint, with the three individual seed values printed under each cell mean.

Table 5.7: Screen C. Cells ranked by full-test survival, with the difference from the unaugmented `e0n0v0` baseline.

Cell	Mean (%)	Std (pp)	Min (%)	Max (%)	Δ baseline (pp)
`e0n0v0`	97.31	1.22	96.55	98.72	0.00
`e1n0v0`	96.20	3.94	91.66	98.68	−1.11
`e0n0v1`	95.63	6.19	88.52	99.82	−1.68
`e0n1v0`	93.51	6.39	86.72	99.40	−3.80
`e1n1v0`	90.01	4.92	85.42	95.19	−7.30
`e1n0v1`	89.92	9.42	80.48	99.33	−7.39
`e0n1v1`	68.91	5.77	62.59	73.91	−28.40
`e1n1v1`	66.11	33.95	27.86	92.68	−31.20

Table 5.8: Screen C. Main effect of each factor (12 runs per level) and the four single-factor flips of the cube, each one enabling that factor with the other two held fixed. `Spread` is the range of those four flips.

Factor	Off (%)	On (%)	Effect (pp)	Flips (pp)	Mean flip (pp)	Spread (pp)
e (substation edges)	88.84	85.56	−3.28	−1.1, −5.7, −3.5, −2.8	−3.28	4.60
v (virtual readout)	94.26	80.14	−14.11	−1.7, −24.6, −6.3, −23.9	−14.11	22.92
n (summary nodes)	94.77	79.63	−15.13	−3.8, −26.7, −6.2, −23.8	−15.13	22.92

Reading

No augmentation earns its place, and the result is unambiguous: the unaugmented `e0n0v0` baseline is the best cell at 97.31%, and all twelve single-factor flips in Table 5.8 are negative. Not one of the three

mechanisms helps in any context. The baseline is also the most stable cell in the screen, with a seed spread of 1.22 points against 3.9–33.9 for every augmented cell, so the augmentations cost reliability as well as survival. The training curves in Figure 5.5 separate the cells well before the budget ends, so this is not an artefact of where the runs were cut.

The two hierarchy factors dominate, at −15.13 points for summary nodes and −14.11 for virtual-node readout, and they are not independent: each is mild in the cell where the other is off (n costs 3.80 points at `e0v0`; v costs 1.68 at `e0n0`) and severe once the other is on, with both $n1v1$ cells landing between 66% and 69%. The upper face of the cube in Figure 5.6 shows this directly, and the spread column of Table 5.8 makes it explicit: for n and v the four flips range over 23 percentage points, so the main effect alone is not a usable summary of either factor. Two of the weaker runs also peaked early and never recovered: the `e0n1v0` seed-1 checkpoint was selected at 2.65M steps and the `e1n1v1` seed-2 checkpoint at 9.87M, against 13.7M to 14.9M for the rest.

The interpretation is that the two hierarchy mechanisms are redundant rather than complementary. Both insert a learned aggregation node above the busbars, and the virtual node attaches to the summary nodes when they exist (Figure 4.7), so enabling both stacks two learned hubs in series between the busbars and the readout. With only two message-passing layers, the readout of the `n1v1` cells is separated from most physical nodes by more hops than the encoder has layers. Screen A varies exactly that depth, so its $K = 3$ row is the direct test of this explanation.

Same-substation edges are the mildest of the three, at −3.28 points on average and within a narrow band: its four flips range only from −1.11 to −5.71, so unlike n and v it behaves like a small, consistent cost rather than a context-dependent one. Its best cell `e1n0v0` sits 1.11 points below the baseline, which is inside the seed spread of both cells, and the per-seed values in Figure 5.7 show why nothing stronger can be claimed: two of its seeds reach 98.7% and 98.3% while the third reaches only 91.7%, straddling the baseline's much tighter 96.6–98.7%. The honest reading is that the edges neither help nor clearly hurt on their own, and that they add candidate edges for no measured return.

The operational conclusion is to keep the plain busbar graph with mean readout for subsequent stages, and to treat any future hierarchy proposal as needing a deeper encoder rather than another augmentation on top of two layers.

5.6 Screen D: Generator and Load Message Direction

Question

In the disaggregated graph, each generator and load is a node attached to its busbar. Section 4.2.3 leaves the direction of that attachment open: the asset may send messages to the busbar, receive them from it, or both. The two directions are not symmetric in effect, because the mean readout of Section 4.4.1 pools asset nodes as well as busbars. Does the choice matter, and do the generator and load relations behave independently?

Design

A complete 3×3 factorial over the generator and load directions with three seeds per cell: 27 runs. The graph type is `heterogeneous` throughout; the line and summary directions stay `bidirectional` and are inert for this schema. The encoder matches the reference configuration: GINE, two layers, 128 features, mean readout, no augmentations, so the only screened factors are the two attachment directions. The baseline cell is bidirectional on both relations. Directions are abbreviated `bi` (bidirectional), `a2b` (asset to busbar), and `b2a` (busbar to asset).

Result

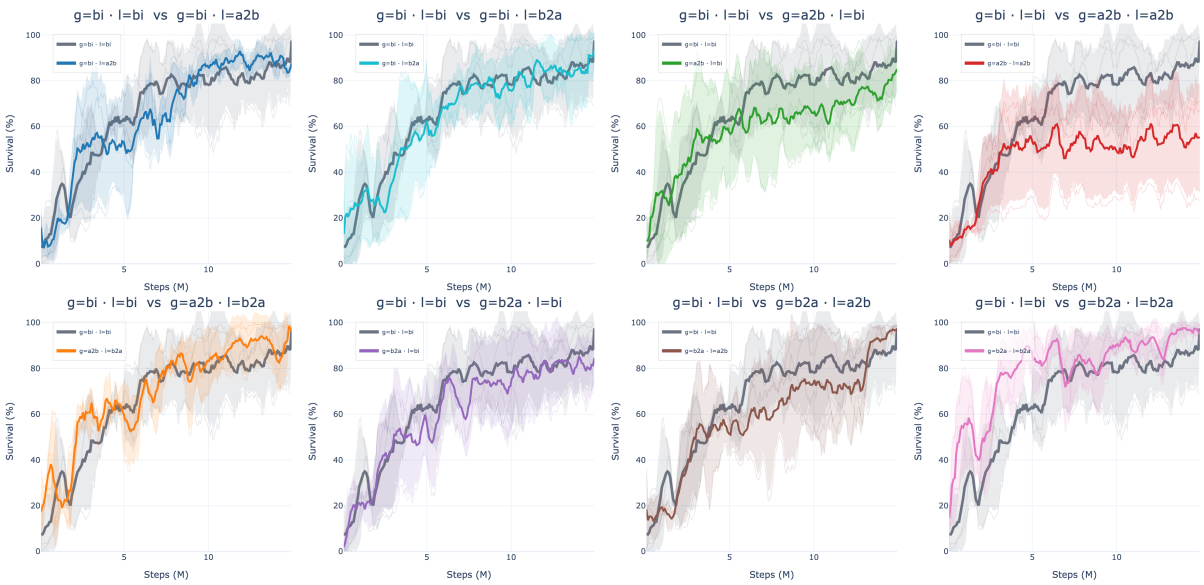


Figure 5.8: Screen D. Test episodic survival during training, one panel per direction pair against the bidirectional baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.

Figure 5.9: Screen D. Full-test survival of the best checkpoint by generator direction (rows) and load direction (columns), with the three individual seed values printed under each cell mean. The baseline cell is `bi/bi`.

Table 5.9: Screen D. Direction pairs ranked by full-test survival, with the difference from the `bi/bi` baseline.

Generator / load	Mean (%)	Std (pp)	Min (%)	Max (%)	Δ baseline (pp)
`b2a` / `b2a`	97.94	2.20	95.40	99.25	+5.30
`b2a` / `a2b`	97.43	1.42	96.32	99.03	+4.79
`a2b` / `b2a`	94.47	7.53	85.78	99.03	+1.83
`bi` / `bi`	92.65	8.24	83.21	98.40	0.00
`bi` / `a2b`	91.35	5.54	86.52	97.40	−1.30
`bi` / `b2a`	87.76	11.04	77.64	99.53	−4.89
`b2a` / `bi`	84.93	13.01	70.11	94.46	−7.72
`a2b` / `bi`	83.16	12.75	72.27	97.19	−9.49
`a2b` / `a2b`	56.41	22.12	30.88	69.60	−36.24

Reading

Busbar-to-asset is the better direction for both relations. Averaging over the other relation, the generator marginal is 93.43% for `b2a`, 90.58% for `bi`, and 78.01% for `a2b`; the load marginal follows the same order at 93.39%, 86.91%, and 81.73%. The strongest part of the result is not the mean but the spread: the two best cells, `b2a`/`b2a` and `b2a`/`a2b`, have seed standard deviations of 2.20 and 1.42 percentage points against 8.24 for the bidirectional baseline, and their worst seeds (95.40% and 96.32%) are above the baseline's mean. The seed values printed in Figure 5.9 show this at a glance: the bottom row holds three tightly grouped seeds per cell, while the baseline cell spans 83.2% to 98.4%.

The failure case is equally clear. Making both relations asset-to-busbar collapses the policy to 56.41% with a 22.12-point spread and a worst seed at 30.88%, which is 36 points below the baseline and the largest single effect measured anywhere in this chapter. Its panel in Figure 5.8 separates from the baseline within the first two million steps and never recovers, so this is a training failure rather than a late collapse; one of its seeds never improved on a checkpoint from 4.31M steps.

A plausible mechanism, consistent with the readout defined in Section 4.4.1 but not directly measured here, is that the mean pooling includes generator and load nodes. Under `b2a` an asset node receives busbar context and its updated embedding enters the readout, so the asset row carries information about its local neighborhood. Under `a2b` the asset node never receives a message, so after K layers its embedding is still a projection of its own raw features, and it dilutes the pooled vector with context-free rows; with both relations set that way, every generator and load row is inert.

Chapter 6

Sparse Intervention Control

Every experiment so far has maximized one quantity, survival on held-out chronics. This chapter changes the objective. A policy that keeps the grid alive by reconfiguring substations at every opportunity is not what a control room wants: interventions have an operational cost, and an agent that acts constantly is neither auditable nor trustworthy. The question here is whether the same decentralized agents can keep their survival while intervening rarely, locally, and mainly when the grid state makes an intervention necessary.

The mechanisms are therefore evaluated on two axes at once, survival and how often the agents leave the do-nothing action. A mechanism that improves sparsity by collapsing survival has not solved the problem, and neither has one that keeps survival by never becoming sparse.

Question. Can the agents keep high survival while behaving more like operators: acting rarely, acting locally, acting mainly when the grid state makes an intervention useful, and keeping the final policy decentralized?

Baseline and change. This block starts from the flat decentralized MAPPO topology-control actor. For agent i, the baseline policy is one categorical distribution over the full local action space:

$$\pi_i(a_i \mid o_i), \qquad a_i \in \{0, 1, \ldots, |\mathcal{A}_i| - 1\}.$$

Action 0 is the no-op action. During training, actions are sampled from the policy, while deterministic evaluation uses the greedy action by default:

$$a_{i,t}^{eval} = \arg\max_a \pi_i(a \mid o_{i,t}).$$

The sparse-control roadmap tests four ways to reduce unnecessary topology changes: evaluation-time rho overrides, an intervention-gated actor, fixed non-idle action penalties, and adaptive Lagrangian intervention budgets.

Motivation. The intervention gate is motivated by hierarchical topology-control work, where the action can be decomposed into "do nothing versus intervene", then where to act, then which local topology configuration to apply [13], [14]. In this thesis only the first level is adapted: each regional agent independently decides whether to do nothing or to execute one of its existing local topology actions. The fixed and adaptive sparse-control objectives are closer to constrained and budgeted RL: non-idle topology interventions are treated as a limited resource, following the general Lagrangian view of constrained policy optimization and budgeted control [23], [24], [25]. This is also operationally meaningful because switching frequency and topology deviation are real power-grid objectives, not just cosmetic metrics [26].

6.1 Evaluation-Time Rho Heuristics

Implementation detail. The rho heuristic is an evaluation-only diagnostic. It does not change the training objective and it does not prove that the policy learned sparse behavior. The policy first proposes the deterministic action $\bar{a}_{i,t}$, then the evaluator may replace it with action 0.

The global version uses the pre-action maximum line loading

$$\rho_t^{global} = \max_{\ell \in \mathcal{L}} \rho_{\ell,t},$$

and executes

$$a_{i,t}^{exec} = \begin{cases} 0, & \rho_t^{global} < \rho_{safe}, \\ \bar{a}_{i,t}, & \text{otherwise}, \end{cases} \qquad \rho_{safe} = 0.90.$$

The local version is decentralized. Agent i computes the maximum rho on the lines touching its regional substations,

$$\rho_{i,t}^{local} = \max_{\ell \in \mathcal{L}_i} \rho_{\ell,t},$$

and only that agent is forced to no-op when $\rho_{i,t}^{local} < \rho_{safe}$. A boundary line can appear in the local neighborhood of both adjacent agents, but no communication is required: both agents can observe the line through their own physical neighborhood.

Interpretation. The heuristic is useful to ask what happens if safe-state interventions are blocked, but it is not a learned method. Its action-0 rates must therefore be reported as final executed evaluation actions, not as evidence that the policy itself prefers action 0.

6.2 Intervention-Gated Actor

Implementation detail. The gated actor changes the actor factorization. Instead of one categorical head over all actions, each agent has a gate head and a non-idle action head:

$$\pi_i^{gate}(g_i \mid o_i), \qquad g_i \in \{0, 1\},$$

where 0 means do nothing and 1 means intervene, and

$$\pi_i^{nonidle}(b_i \mid o_i), \qquad b_i \in \{1, \ldots, |\mathcal{A}_i| - 1\}.$$

During training, the gate is sampled first. If $g_{i,t} = 0$, then $a_{i,t} = 0$. If $g_{i,t} = 1$, the final action is sampled from the non-idle head. The PPO log-probability of the executed action is therefore

$$\log P(a_i = 0) = \log P(g_i = 0),$$

and, for $a_i > 0$,

$$\log P(a_i) = \log P(g_i = 1) + \log P(b_i = a_i \mid g_i = 1).$$

Two deterministic decoders are used. The `final_action_map` decoder selects the most likely final executed action:

$$P(a_i = 0) = P(g_i = 0), \qquad P(a_i = a > 0) = P(g_i = 1)P(b_i = a \mid g_i = 1).$$

The `hierarchical_greedy` decoder first chooses the most likely gate decision, then chooses the most likely non-idle action only if the gate selects intervention.

The original coupled entropy term is

$$H_i = H(g_i) + P(g_i = 1)H(b_i).$$

This can accidentally reward intervention because larger $P(g_i = 1)$ unlocks more non-idle entropy. The separated entropy variant is

$$H_i = \alpha_{gate} H(g_i) + \alpha_{nonidle} H(b_i),$$

and is used in the gate-plus-budget diagnostic runs.

Interpretation. The gate is learned and decentralized, but it can restrict exploration more strongly than a reward penalty. If it collapses early to no-op, the non-idle head is rarely executed and receives little learning signal. For this reason, the gate is treated as an important diagnostic rather than the main sparse control contribution.

6.3 Fixed Sparse-Action Penalty

Implementation detail. The Sparse16 sweep adds a fixed reward penalty to the final executed action of each agent:

$$r'_{i,t} = r_{i,t} - \lambda_{fixed}\mathbf{1}[a_{i,t} \neq 0],$$

with

$$\lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}.$$

The grid is

$$\text{actor} \in \{\text{flat}, \text{gated}\}, \qquad \lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}, \qquad \text{seed} \in \{0, 1, 2\},$$

which gives $2 \times 4 \times 3 = 24$ runs. All Sparse16 runs set `safe_intervention_penalty = 0.0`, so the penalty is not rho-weighted.

Interpretation. Sparse16 changes only the training reward. It does not override actions during evaluation and it does not hard-mask exploration during training. The policy can still sample non-idle actions, but each such action must be useful enough to pay its fixed cost. The key comparison is `sparse16_flat_p010` versus `sparse16_gated_p010`, which asks whether the gate adds value once a simple sparse objective already exists.

6.4 Adaptive Intervention Budget

Implementation detail. The adaptive intervention budget turns sparse topology control into a local constraint. For each agent,

$$c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0] w_i(s_t),$$

and the desired constraint is

$$\mathrm{E}_{\pi_i}\left[c_i(s_t, a_{i,t})\right] \leq d.$$

The implemented rollout reward removes the constant term that does not affect the PPO action gradient:

$$r'_{i,t} = r_{i,t} - \lambda_i c_i(s_t, a_{i,t}).$$

After each rollout, the local multiplier is updated by

$$\lambda_i \leftarrow \mathrm{clip}\left(\lambda_i + \eta(\hat{C}_i - d), 0, \lambda_{\max}\right), \qquad \hat{C}_i = \frac{1}{T}\sum_{t=1}^{T} c_i(s_t, a_{i,t}).$$

If the observed cost $\hat{C}_i$ is above the budget d, future interventions become more expensive; if it is below the budget, the penalty relaxes. The default settings are `intervention_budget_lr = 0.02`, `intervention_budget_init_lambda = 0.0`, and `intervention_budget_max_lambda = 10.0`.

Three cost modes are used:

- **Non-idle cost:** $w_i(s_t) = 1$, so $c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]$. This is an adaptive version of a plain sparse-action penalty.
- **Global-safe cost:** $w^{global}(s_t) = \sigma(k(\rho_{safe} - \rho_t^{global}))$. This penalizes non-idle actions mostly when the whole grid is safe.
- **Local-safe cost:** $w_i^{local}(s_t) = \sigma(k(\rho_{safe} - \rho_{i,t}^{local}))$. This is the decentralized version: safe local states make interventions expensive, while stressed local states make them cheap.

In the AIB setting, $\rho_{safe} = 0.90$ is not a hard action override. It is the center of a smooth sigmoid safety weight.

AIB experiment grid. The first AIB folder, `configs/adaptive_intervention_budget_7`, contains the main local-budget ablations:

- `aib_00_flat_local_t020`: flat actor, local-safe cost, $d = 0.20$;
- `aib_01_flat_local_t010`: stricter local-safe budget, $d = 0.10$;
- `aib_02_flat_local_t035`: looser local-safe budget, $d = 0.35$;
- `aib_03_gate_hgreedy_sep_local_t020`: gated actor, hierarchical-greedy evaluation, separated entropy, local-safe cost, $d = 0.20$;
- `aib_04_flat_nonidle_t020`: flat actor with adaptive plain non-idle cost, $d = 0.20$.

The comparison `aib_00_flat_local_t020` versus `aib_04_flat_nonidle_t020` isolates the value of local rho weighting.

The mechanism-ablation folder `configs/adaptive_intervention_budget_mechanism_15` contains 15 additional flat-actor runs. It separates fixed versus adaptive coefficients, plain non-idle versus state-dependent costs, global versus local rho weighting, and rho-threshold sensitivity.

Table 6.1: Sparse-control mechanisms and where they affect the policy.

Method	Training effect	Evaluation effect	Main risk
Baseline	Samples from the flat policy	Greedy argmax by default	Standard entropy-controlled exploration
Rho heuristic	No training change	Hard no-op override in safe states	Evaluation is manually changed
Intervention gate	Changes action factorization and sampling	Final-action MAP or hierarchical greedy	Gate collapse can starve non-idle exploration
Sparse16	Fixed reward penalty on non-idle actions	No action override	Penalty coefficient must be tuned
AIB non-idle	Adaptive reward penalty on non-idle actions	No action override	Multiplier can grow if target is too strict
AIB local-safe	Adaptive state-dependent reward penalty	No action override	Depends on the local rho safety-weight design

Table 6.2: Main sparse-control comparisons to report.

Question	Comparison
Does a fixed sparse penalty already work?	Baseline versus `sparse16_flat_p010`
Does the gate help beyond a sparse objective?	`sparse16_flat_p010` versus `sparse16_gated_p010`
Fixed versus adaptive coefficient?	`sparse16_flat_p010` or `aibm_00_fixed_nonidle_p006` versus `aib_04_flat_nonidle_t020`
Does local-safe weighting matter?	`aib_00_flat_local_t020` versus `aib_04_flat_nonidle_t020`
Does global-safe weighting matter?	`aibm_02_adaptive_global_t020` versus `aib_00_flat_local_t020`
Is $\rho_{safe} = 0.90$ special?	`aibm_03` with 0.85, `aib_00` with 0.90, and `aibm_04` with 0.95
Is the heuristic only an evaluation trick?	Rho-heuristic runs versus learned sparse/AIB runs

Full-test checkpoint summaries. The following tables report the average full-test episodic survival over the three evaluated seeds. The survival values are computed directly from the JSON files in `Topology_Task/outputs/full_test_eval`. These JSON files also record where the compact action-distribution artifacts were written. The action-0 fractions are computed from the corresponding local `Topology_Task/outputs/full_test_eval_actions/*/action_summary.json` files and averaged over

seeds. The all-agent action-0 column is the arithmetic mean of the three per-agent action-0 fractions. The do-nothing row is a reference policy that always selects action 0; its survival is computed from `Topology_Task/outputs/do_nothing_eval/bus14_test_20`. Remaining "–" entries indicate that the compact action summaries for those full-test jobs are not available locally.

Table 6.3: Full-test summary for evaluation-time rho heuristics.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Global rho heuristic, $\rho_{safe} = 0.90$	98.36	0.936	0.933	0.936	0.940
Local rho heuristic, $\rho_{safe} = 0.90$	97.07	0.968	0.992	0.985	0.928

Table 6.4: Full-test summary for intervention-gated actors.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Intervention gate, final-action MAP	89.82	0.802	0.826	0.889	0.690
Intervention gate, hierarchical greedy	93.66	0.395	0.066	0.844	0.276

Table 6.5: Full-test summary for Sparse16 flat-policy runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat Sparse16, $\lambda_{fixed} = 0.000$	98.23	0.494	0.524	0.463	0.496
Flat Sparse16, $\lambda_{fixed} = 0.001$	96.75	0.486	0.642	0.401	0.416
Flat Sparse16, $\lambda_{fixed} = 0.003$	93.97	0.575	0.691	0.346	0.688
Flat Sparse16, $\lambda_{fixed} = 0.010$	97.44	0.905	0.907	0.954	0.853

Metrics to report. The main performance metric remains episodic survival. It should be paired with sparsity and physical-diagnostic metrics: per-agent action-0 fractions, per-agent non-idle rates, the distribution of how many agents act simultaneously, intervention-budget multipliers, budget costs and violations, the rate of interventions in costly safe states, pre- and post-action max rho, and topology-distance changes.

Interpretation. The current roadmap interpretation is that the gate is not the strongest contribution by itself. Sparse reward shaping already improves action-0 usage, the adaptive budget makes the sparse penalty self-tuning, and the local-safe cost makes the penalty more physically meaningful. In particular, `sparse16_flat_p010` is a strong fixed-penalty baseline, while the most principled learned sparse-control candidate is `aib_00_flat_local_t020`: it keeps the original decentralized actor, does not rely on an evaluation-time override, learns the intervention penalty adaptively, and uses a local state-dependent safety weight.

Table 6.6: Full-test summary for Sparse16 gated-policy runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Gated Sparse16, $\lambda_{fixed} = 0.000$	78.51	0.787	0.747	0.949	0.665
Gated Sparse16, $\lambda_{fixed} = 0.001$	73.82	0.862	0.956	0.832	0.799
Gated Sparse16, $\lambda_{fixed} = 0.003$	94.59	0.887	0.934	0.929	0.799
Gated Sparse16, $\lambda_{fixed} = 0.010$	85.69	0.974	0.990	0.984	0.947

Table 6.7: Full-test summary for adaptive intervention budget runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat local-safe AIB, $d = 0.20$	98.31	0.978	0.980	0.989	0.966
Flat local-safe AIB, $d = 0.10$	98.39	0.979	0.992	0.995	0.949
Flat local-safe AIB, $d = 0.35$	96.61	0.934	0.959	0.947	0.896
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	33.87	0.999	1.000	1.000	0.998
Flat non-idle AIB, $d = 0.20$	95.08	0.986	0.997	0.992	0.970

Table 6.8: Full-test summary for the teacher-student distillation runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Teacher: MAPPO with local rho heuristic	97.07	0.968	0.992	0.985	0.928
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	92.14	0.939	0.974	0.925	0.919
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	90.76	0.957	0.982	0.948	0.940

Chapter 7

Transfer to the 36-Substation WCCI Grid

Every result so far was obtained on `bus14`. That grid is small enough to iterate on but too small to claim anything about scale: fourteen substations, twenty lines, three agents. This chapter moves to the L2RPN WCCI 2020 grid, more than twice as many substations and three times as many lines, and asks the question that motivates a graph encoder in the first place. If the actor reads the grid as a graph rather than as a flat vector, its weights are independent of how many substations exist, and a representation learned on `bus14` should carry over.

Before running that transfer, the two environments have to be made comparable and the encoder's inputs have to be checked. This chapter does both. The comparison turns out to matter more than expected: the graph features the encoder consumes are raw physical quantities, and they do not follow the same distribution on the two grids.

Legacy graph-observation qualifier

Unless a result is explicitly identified as a leakage-free rerun, the graph statistics and transferred checkpoints in this chapter were produced before the information-boundary correction of Section 4.2.5. They include the legacy neighboring candidate rows and are not strictly comparable to runs using the corrected node and edge masks. The distribution measurements remain a description of the inputs consumed by those historical encoders, not of the final local actor observation.

7.1 The target environment

Table 7.1 contrasts the source and target grids. The agent partitions follow the same principle in both cases, contiguous electrical areas, but WCCI is split into four regions rather than three, and the regions are markedly unequal: one agent controls seventeen substations while another controls five.

Table 7.1: Source and target environments. Both use difficulty 0, topology actions, decentralized agents and an 80/20 train/test chronic split with split seed 0.

	`bus14`	`bus36_wcci_nomaint`
Grid2Op environment	`l2rpn_case14_sandbox`	`l2rpn_wcci_2020`
Substations	14	36
Lines	20	59
Agents	3	4
Chronics	1,004	2,880
Episode length	up to 8,064 steps	up to 8,062 steps
Maintenance	none	disabled (Section 7.1.1)

7.1.1 Removing maintenance

Of the environments Grid2Op distributes, `l2rpn_case14_sandbox` is the only one above the toy scale that ships without scheduled maintenance. Every larger environment, WCCI 2020 included, injects planned line outages. That is a problem for a transfer study: a policy trained without maintenance and evaluated with it faces a hazard it has never seen, and the two observation spaces differ as well, since the maintenance countdown features are only present when maintenance exists.

The two settings are therefore aligned by removing maintenance from WCCI rather than inventing a schedule for `bus14`, for which Grid2Op provides no specification.

The effect is large. On a fixed sample of fifty held-out chronics, evaluated by name so that both settings see the same episodes, a do-nothing policy survives 21.7% of the horizon with maintenance and 56.0% without, and completes 6% of episodes with maintenance against 52% without. Maintenance was responsible for most of the failures on this grid. make a table to show the result and use actually the full test eval on maintenance and off maintenance to show the effect of not using maintenance.

7.1.2 Action space

The unreduced WCCI action space is not trainable. Enumerating topology actions over each agent's region gives the sizes in the first column of Table 7.2: nearly 67,000 actions in total, of which agent 1 alone accounts for 65,642. All experiments therefore use the reduced action space obtained by the brute-force outcome procedure, at $k = 256$ per agent.

Table 7.2: Per-agent action-space sizes on `bus36_wcci_nomaint`. Agents 0 and 2 are already exhaustive at $k = 256$, so the reduction only binds on agents 1 and 3.

Agent	Substations	Full	Reduced ($k = 256$)
`agent_0`	5	77	77
`agent_1`	5	65,642	256
`agent_2`	9	127	127
`agent_3`	17	1,119	256
Total	36	66,965	716

The choice of k was made by evaluating a one-step simulator-greedy policy over each candidate reduction on the same fifty chronics (Table 7.3). Performance saturates at $k = 256$: the difference between $k = 256$ and $k = 1024$ is four chronics won against three lost, which is noise, while both clearly beat $k \leq 128$. Since $k = 256$ is the smallest reduction in the saturated group, it gives the smallest action heads for no measurable loss in what the action set can achieve.

Table 7.3: One-step simulator-greedy policy over each reduced action space, on the same fifty held-out chronics of `bus36_wcci_nomaint`. The do-nothing row is the common reference; no reduction was evaluated against a different chronic set.

Policy	Mean survival	Full-episode survival	Episodes worse than do-nothing
Do nothing	0.560	52%	—
Greedy, $k = 64$	0.819	64%	3
Greedy, $k = 128$	0.847	76%	3
Greedy, $k = 256$	**0.976**	**92%**	0
Greedy, $k = 512$	0.883	82%	2
Greedy, $k = 1024$	0.953	90%	0

These two rows bracket the transfer experiments. A learned policy that lands near 0.56 mean survival has learned nothing beyond doing nothing; one approaching 0.9 is doing real work. The greedy row is not a strict upper bound, since it queries the simulator at every decision step and is myopic, but it is a strong reference.

7.2 Why the input distribution matters

Only the graph encoder can cross grids yet because with the heterogenous + line and adaptative pooling allows to have a transferable head. The per-agent action heads cannot: the partitions differ in size and count, so no weight in a head has a counterpart on the other grid. The centralized critic reads a flat, grid-sized observation and is likewise grid-specific. What transfers is the encoder, and what the encoder reads is the node and edge feature vectors.

Attention critic is fed the normalized values Those vectors are unnormalized.

A frozen `bus14` encoder therefore receives WCCI's raw physical quantities. Whether that is a problem is an empirical question about how similar the two distributions are.

7.3 Measurement protocol

Both environments were rolled out under a do-nothing policy for 6,000 environment steps, with a cap of 300 steps per chronic so the sample spreads over roughly twenty scenarios per grid rather than concentrating in one. At each step the per-agent local graphs were rebuilt and every node and active physical edge recorded, pooling agents together because a single encoder is shared across them. This yields 276,000 node observations for `bus14` and a capped 400,000 for WCCI, and 312,000 and 400,000 edge observations respectively.

Four statistics summarize each feature. Writing F_A and F_B for the two empirical distributions with means μ_A, μ_B and standard deviations σ_A, σ_B, and $\sigma_p = \sqrt{(\sigma_A^2 + \sigma_B^2)/2}$ for their pooled spread:

$$\text{offset} = \frac{\mu_B - \mu_A}{\sigma_p}, \qquad \text{scale ratio} = \frac{\sigma_B}{\sigma_A}, \tag{7.1}$$

$$\text{KS} = \sup_x |F_A(x) - F_B(x)|, \qquad W_1/\sigma_p = \frac{1}{\sigma_p} \int_0^1 |F_A^{-1}(q) - F_B^{-1}(q)| \, \mathrm{d}q. \tag{7.2}$$

The pairing is deliberate. The offset is blind to a change in spread and the scale ratio is blind to a displacement, so a feature is only unproblematic when both are near their neutral values. KS measures disagreement in the bulk of the distribution and is comparatively insensitive to tails, which W_1 does weigh.

7.4 What the encoder reads

The measurements below are taken on the `bus` graph, whose ten channels are specified in Tables 4.4 and 4.5. What matters here is not their definition but how often each one is zero, since that is what the following sections turn on. Table 7.4 repeats the channel list with that fraction measured on each grid.

Table 7.4: Node and edge features of the `bus` graph, with the fraction of sampled entries equal to zero on each grid. A busbar only carries generation or load when an asset is attached to it, which makes four of the six node channels structurally sparse.

Feature	Meaning	Unit	Zeros (%)	
			`bus14`	WCCI
Node features				
`gen_p`	Active power injected by attached generators	MW	84.0	88.0
`gen_theta`	Voltage angle at the generator connection	deg	84.8	80.3
`load_p`	Active power drawn by attached loads	MW	56.5	56.4
`load_theta`	Voltage angle at the load connection	deg	56.5	58.0
`time_before_cooldown_sub`	Steps until this substation may be reconfigured	steps	100	100
`domain_mask`	1 for the agent's own substations, 0 for legacy whole-substation context	—	39.1	35.7
Edge features				
`line_status`	1 connected, 0 disconnected	—	0	0
`rho`	Current flow divided by thermal limit	p.u.	0	0
`timestep_overflow`	Consecutive steps above the thermal limit	steps	100	100
`time_before_cooldown_line`	Steps until this line may be switched	steps	100	100

What is the zeros column why is there sometines 0 soemtimes 100 what does it tell us ??

One feature is dimensionless by construction. `rho` divides each line's current flow by that line's own thermal limit, so it is expressed in units the grid itself defines. Every other physical channel carries the units of the grid it came from.

Three channels are constant across the whole sample and carry no information here. Under a do-nothing policy no cooldown is ever triggered, and only active physical edges are recorded, which makes `line_status` identically one. They are reported for completeness and excluded from the discussion.

7.5 Results

7.5.1 Aggregate statistics

Table 7.5 reports the four statistics over all sampled entries.

Table 7.5: Distribution statistics over all sampled nodes and edges, ordered by KS. Offset and W_1 are in pooled standard deviations; the scale ratio is WCCI over `bus14`. Constant channels give undefined ratios.

Feature	μ_{bus14}	μ_{WCCI}	Offset	Scale ratio	KS	W_1/σ_p
`load_theta`	−2.402	0.614	+1.024	0.850	0.331	1.024
`rho`	0.364	0.319	−0.260	1.048	0.141	0.261
`gen_theta`	−0.633	0.215	+0.503	0.919	0.115	0.503
`gen_p`	8.645	9.451	+0.020	2.435	0.089	0.266
`load_p`	9.723	7.579	−0.063	2.363	0.040	0.238
`domain_mask`	0.609	0.643	+0.071	0.982	0.034	0.070
`timestep_overflow`	5.1e-5	7.5e-6	−0.007	0.403	0.000	0.000
`line_status`	1	1	—	—	0.000	—
`time_before_cooldown_line`	0	0	—	—	0.000	—
`time_before_cooldown_sub`	0	0	—	—	0.000	—

Read carelessly this table looks mild. Only `load_theta` exceeds KS = 0.2, and the power channels sit near KS = 0.05. But their scale ratios are 2.4, and that is exactly the pattern the offset–scale pairing was introduced to catch: the two distributions share a centre while differing in width, so KS stays small because the bulk still overlaps.

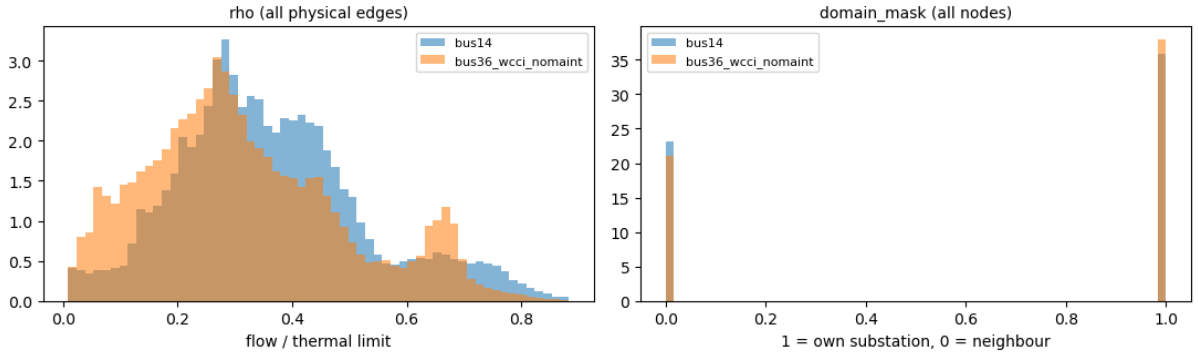


Figure 7.1: The two dense channels. `rho` is nearly identical on the two grids, as expected from a quantity already normalized by each grid's own thermal limits. `domain_mask` is structural, and its small difference reflects how many neighboring candidate rows the legacy construction pulled into each agent partition.

7.5.2 Conditioning on the nodes that carry signal

The aggregate table understates the problem, because most of its rows are structural zeros. A busbar with no generator contributes a zero to `gen_p` on both grids alike, and roughly 85% of rows are of that kind. Statistics computed over such a column largely describe how many zeros it contains.

Restricting each sparse channel to the nodes where the corresponding asset is actually attached gives Table 7.6. Every statistic roughly doubles.

Table 7.6: The four sparse node channels, restricted to nodes that carry the corresponding asset. Means and standard deviations are in MW for the power channels and degrees for the angles.

	`bus14`		WCCI					
Feature	μ	σ	μ	σ	Offset	Scale ratio	KS	W_1/σ_p
`load_theta`	−5.53	2.43	1.46	4.01	+2.106	1.648	**0.758**	2.106
`gen_theta`	−4.16	2.36	1.10	3.50	+1.757	1.482	**0.724**	1.757
`gen_p`	53.96	23.12	78.75	134.67	+0.257	**5.824**	0.496	0.744
`load_p`	22.36	22.87	17.37	65.69	−0.101	2.873	0.093	0.378

A KS of 0.758 means the two empirical CDFs differ by 76 percentage points at their widest separation.

These are not the same distribution in any useful sense. The angle means state it plainly: `bus14` sits near $-5.5°$ and $-4.2°$ while WCCI sits near $+1.5°$ and $+1.1°$, a systematic displacement of five to seven degrees. Generation conditioned on actual generators averages 54 ± 23 MW on `bus14` against 79 ± 135 MW on WCCI.

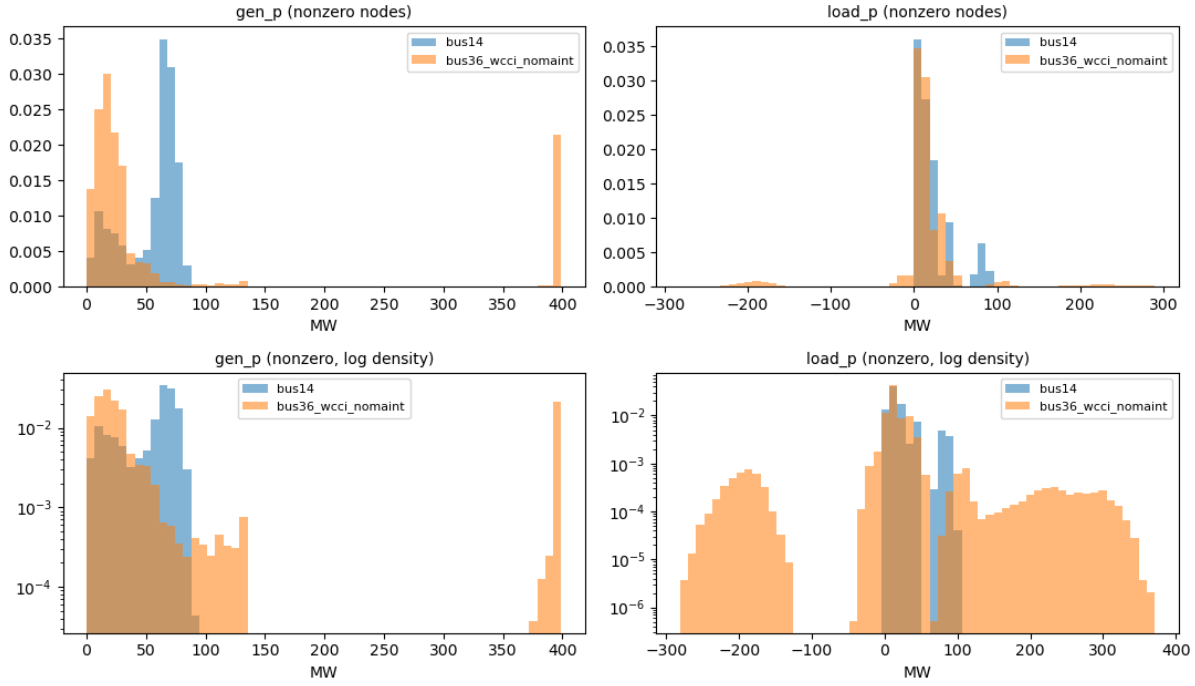


Figure 7.2: Power channels at nodes carrying the corresponding asset, on a linear density scale (top) and a logarithmic one (bottom). The logarithmic row is the informative one: WCCI carries a long right tail that `bus14` does not have, reaching 399 MW of generation against a 92 MW ceiling, and `load_p` extends below zero on 3.5% of WCCI load nodes while `bus14` never produces a negative value.

How is it that we haev negative load for wcci and not for bus14? One would expect either to say load are consumption so negative power consumption and have only negative values or a positive demand and only have positive values

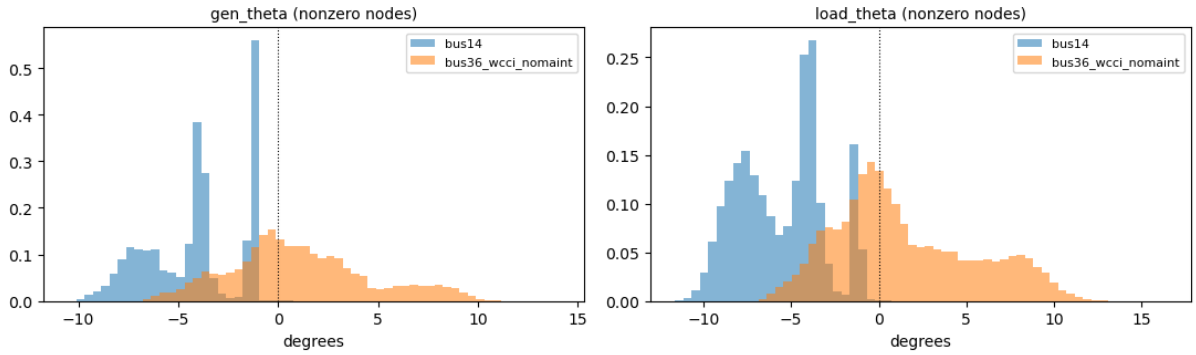


Figure 7.3: Voltage angles at nodes carrying the corresponding asset. The `bus14` distributions lie almost entirely on the negative side, range $[-11.6°, +1.5°]$, while WCCI spreads across both signs out to $+16.4°$. The distributions are displaced rather than merely rescaled.

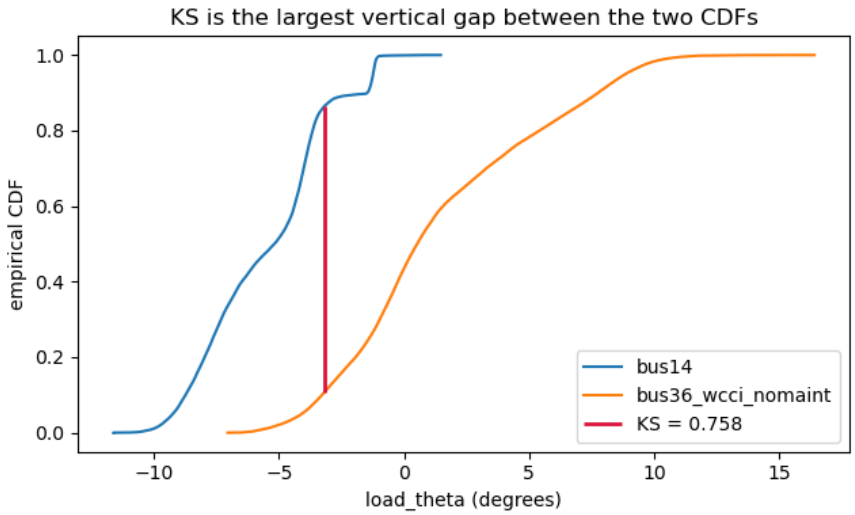


Figure 7.4: The empirical CDFs of `load_theta` at load-carrying nodes. The KS statistic is the largest vertical distance between the two curves, marked in red.

7.6 Two failure modes

Figure 7.5 separates the channels along the two axes. The distinction is not cosmetic, because the architecture responds to them differently.

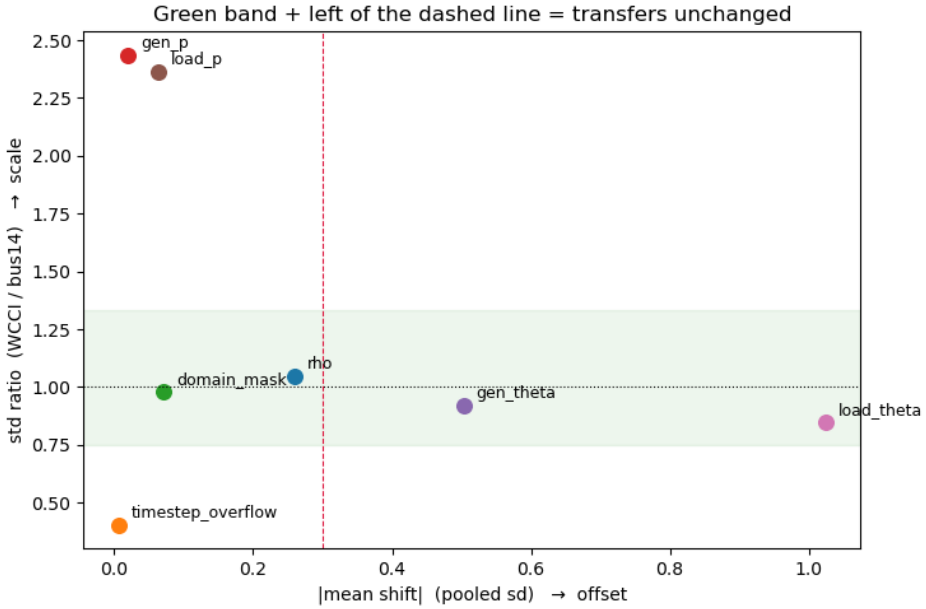


Figure 7.5: Absolute offset against scale ratio, over all sampled entries. The shaded band marks a scale ratio within a third of unity and the dashed line an offset of 0.3 pooled standard deviations. Channels outside those bounds fail in one of two distinct ways.

The actor's node pre-encoder is a linear map followed by ReLU and LayerNorm. LayerNorm absorbs much of a uniform change of *scale*: multiplying every input channel by a constant leaves the normalized activations largely unchanged. It does not absorb an *offset*. Displacing a channel by a constant moves the pre-activations to a different region of the ReLU, changing which units fire at all, and normalization applied afterwards cannot undo that.

This makes the angles, not the powers, the binding problem. The power channels differ in width by factors of 2.9 to 5.8, which the architecture partially self-corrects, though the negative `load_p` region on WCCI reaching −281 MW is genuinely unseen territory rather than a wider version of something familiar. The angles differ by roughly two pooled standard deviations of displacement, which it does not correct at all.

`rho`, meanwhile, needs no correction: a scale ratio of 1.048 and an offset of −0.26. The single most decision-relevant channel, the one that says how close a line is to its limit, is already comparable across grids because it was defined in per-unit terms. That is an argument for expressing features in grid-relative units wherever the physics permits it.

7.7 Implication for the transfer experiments

A frozen `bus14` encoder evaluated on WCCI is being asked to interpret inputs displaced by about two standard deviations on two of its six node channels, and widened several-fold on two more. Any deficit such a policy shows relative to one trained from scratch is therefore confounded: it may reflect a representation that does not generalize, or merely a representation reading miscalibrated inputs.

Two preprocessing mechanisms exist in the codebase. Deterministic physical scaling divides each channel by a per-environment physical constant. Running standardization maintains per-environment, per-node-type statistics over a feature set that includes both power channels and both angle channels, while deliberately excluding `rho`, which the measurements above confirm needs no help. On the diagnosis alone the second looks like the match, since it is the only one that can move a displaced channel. The next section shows that it is not, and that the angle problem is better solved by changing the representation than by normalizing it.

7.8 Correcting the inputs

7.8.1 Running standardization makes the mismatch worse

Standardizing each environment with its own statistics aligns the aggregate moments exactly: every channel comes out at offset 0.00 and scale ratio 1.00. It also moves the distributions further apart on every channel it touches (Table 7.7).

Table 7.7: KS between the two grids before and after per-environment standardization. The moments align perfectly and every channel degrades. The last column gives the standardized value that encodes "no asset is attached here" on `bus14` and on WCCI respectively.

Channel	KS raw	KS standardized	"No asset" encoding
`gen_p`	0.089	**0.843**	−0.396 vs −0.178
`gen_theta`	0.115	**0.752**	—
`load_p`	0.040	**0.603**	−0.520 vs −0.171
`load_theta`	0.331	**0.436**	—

The cause is the mixture structure documented in Table 7.6. These channels are a point mass at zero, meaning no generator or load is attached to this busbar, plus a continuous part. Subtracting a per-grid mean moves that point mass to a grid-dependent location. Since 84–88 % of busbars carry no generator, the encoder's single most frequent input stops meaning the same thing on the two grids, which is a larger mismatch than the displacement being corrected.

Two further properties rule the mechanism out independently of these numbers. Standardization is scale-invariant, so it erases any scaling applied before it: dividing by a constant and then standardizing gives KS identical to three decimals to standardizing alone, so the two mechanisms cannot be combined

and the second always overrides the first. And running statistics begin empty and converge during training, so a frozen encoder would spend early target training reading an input distribution that is itself still moving, while a deterministic scaling is fixed at construction and identical on both grids from the first step.

7.8.2 Physical scaling for the power channels

Physical scaling, defined in Section 4.1, divides the power channels by each environment's own maximum generator capacity. Two of its properties answer the failure diagnosed above: being multiplicative it leaves zeros at zero, so the point mass stays aligned on both grids, and being fixed at construction it does not drift during training the way estimated moments do.

The divisor is a single generator's nameplate rating, a crude proxy for grid scale, so it is worth checking against the alternatives (Table 7.8). It is the closest of the four to the divisor that would equalize the two spreads exactly.

Table 7.8: Candidate power normalizers and the `gen_p` scale ratio each would produce. A ratio of 1.00 would be exact; the raw ratio is 2.44.

Divisor	`bus14`	WCCI	Resulting ratio
Exact	—	—	1.00
Maximum $P_{\max}$ (used)	140	400	**0.85**
Total $P_{\max}$	540	2,450	0.54
Generator count	6	22	0.66
Median $P_{\max}$	85	71	2.94

7.8.3 The angle displacement is a gauge artifact

Scaling cannot repair the angle displacement, and the implementation does not attempt it: angle channels are divided by a constant 180, identical on both grids, so both sides move together and the relative displacement is untouched.

The measurement itself explains why. An absolute voltage angle is referenced to whichever bus is slack, and the two grids do not place theirs alike, so the 5 to 7° displacement visible in Figure 7.3 is largely gauge rather than physics. Nothing that rescales or recentres a reference-dependent quantity can fix that; the quantity has to be replaced.

The replacement is the angle drop along each line, defined and placed in Section 4.1.2. Being a difference it is gauge-invariant, and being defined per line it is dense, so it removes the mixture structure from the angle channels outright rather than working around it – which is what made standardization fail on them. This study therefore enables it, alongside physical scaling, in both the source and target environments.

7.8.4 The corrected distribution

Table 7.9 measures both changes together, under the same protocol as Section 7.3. The angle channel lands in the band occupied by `rho`, and the power spreads come within 15 % of parity.

Table 7.9: Distribution shift with `gnn_angle_representation = edge_diff` and `gnn_physical_scaling = true`, against the uncorrected baseline, over all sampled entries.

	Uncorrected		Corrected	
Channel	ratio	KS	ratio	KS
`load_theta`	0.85	0.331	replaced	
`gen_theta`	0.92	0.115	replaced	
`theta_diff`	—		**1.15**	**0.231**
`gen_p`	2.44	0.089	**0.85**	0.115
`load_p`	2.36	0.040	**0.83**	0.244
`rho`	1.05	0.141	1.05	0.142

The KS increase on the two power channels is the expected cost of the scaling rather than a regression. Scaling slides the continuous part of a channel while its point mass stays pinned at zero, so the two cumulative distributions separate slightly in the bulk even as their dynamic ranges align. Dynamic range is what a frozen encoder's weights are calibrated against, so the trade is favourable, and it accounts for `load_p` reading 0.244.

Corrected feature distributions, produced by the same collection tool with `-gnn-angle-representation edge_diff` and `-gnn-physical-scaling true`.

Figure 7.6: Corrected distributions, to be shown alongside Figures 7.2 and 7.3 for direct comparison.

7.9 Transfer experiment design

Four encoder architectures are carried over from Chapter 5, two from the message-direction screen and two from the structural screen. Each runs in two arms at three seeds. In the *frozen* arm the encoder is loaded from the paired `bus14` run and its parameters stop requiring gradients, so only the action heads train; in the *scratch* arm the same architecture starts from random initialization. Every other setting is copied from the corresponding `bus14` configuration, which is what makes the encoder weights loadable at all.

Loading needs care in one place. The encoder registers its edge index and node identifiers as persistent buffers, so they appear in the saved state dictionary at source-grid shape. They describe the source graph and are supplied per agent from the target environment's own specification at call time, so they are dropped during the load, while any other shape disagreement is treated as an architecture mismatch and raises rather than leaving part of the encoder randomly initialized.

Table 7.10: Transfer study design. The two arms of a pair differ only in the two transfer settings.

Component	Setting
Environment	`bus36_wcci_nomaint`, difficulty 0, topology actions, 80/20 split with split seed 0
Action space	Reduced at $k = 256$, giving 77 / 256 / 127 / 256 actions per agent
Architectures	`gb2a_la2b` and `gb2a_lb2a` (heterogeneous); `e0n0v0` and `e1n0v0` (bus)
Arms	Frozen encoder with trained heads; identical architecture from scratch
Seeds	0, 1, 2
Budget	15M steps, matching the `bus14` source runs
Inputs	`edge_diff` angles, physical scaling enabled

A first set of runs was launched with the uncorrected inputs before the analysis above was complete. Those are retained rather than discarded: comparing the scratch arm with and without the input correction isolates whether the correction helps or harms on the target grid, independently of any transfer effect, on otherwise identical settings.

7.10 First transfer results

The runs reported here are the first arm described above: the eight launched before the input analysis was complete, one seed each, with physical scaling and running standardization both disabled and the original angle representation. They are therefore the uncorrected-input condition, which is the one Section 7.7 predicts should be hardest on a frozen encoder. The corrected runs of Table 7.10 are still training.

Each value below is a deterministic evaluation of a run's best checkpoint over 50 held-out WCCI chronics. The reference to beat is the do-nothing policy, which survives 56% of those episodes; on this grid inaction is a far stronger baseline than on `bus14`, where it survives about 20%.

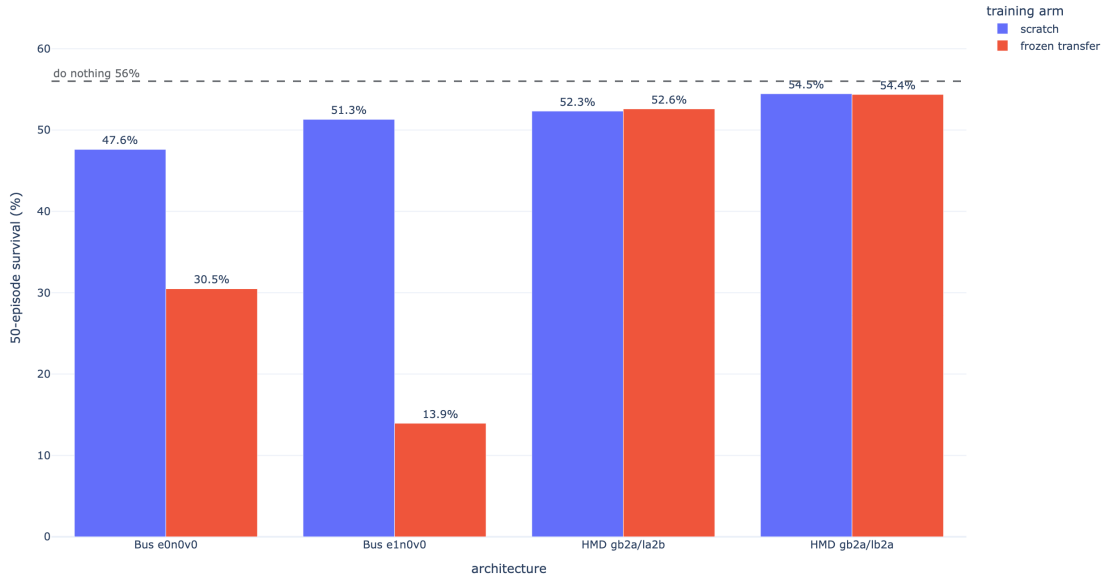


Figure 7.7: 50-episode full-test survival of the best checkpoint, by architecture and training arm, with uncorrected inputs and one seed. The dashed line is the do-nothing policy at 56%.

Table 7.11: 50-episode full-test survival and the step at which each run's best checkpoint was saved. One seed per cell, uncorrected inputs.

Architecture	Scratch (%)	Frozen (%)	Δ (pp)	Scratch ckpt (M)	Frozen ckpt (M)
`e0n0v0` (bus)	47.62	30.48	−17.13	7.38	0.58
`e1n0v0` (bus)	51.31	13.94	−37.37	4.31	13.02
`gb2a_la2b`	52.32	52.60	+0.27	7.38	14.68
`gb2a_lb2a`	**54.46**	54.37	−0.09	7.38	10.12
Do-nothing policy	56.0				

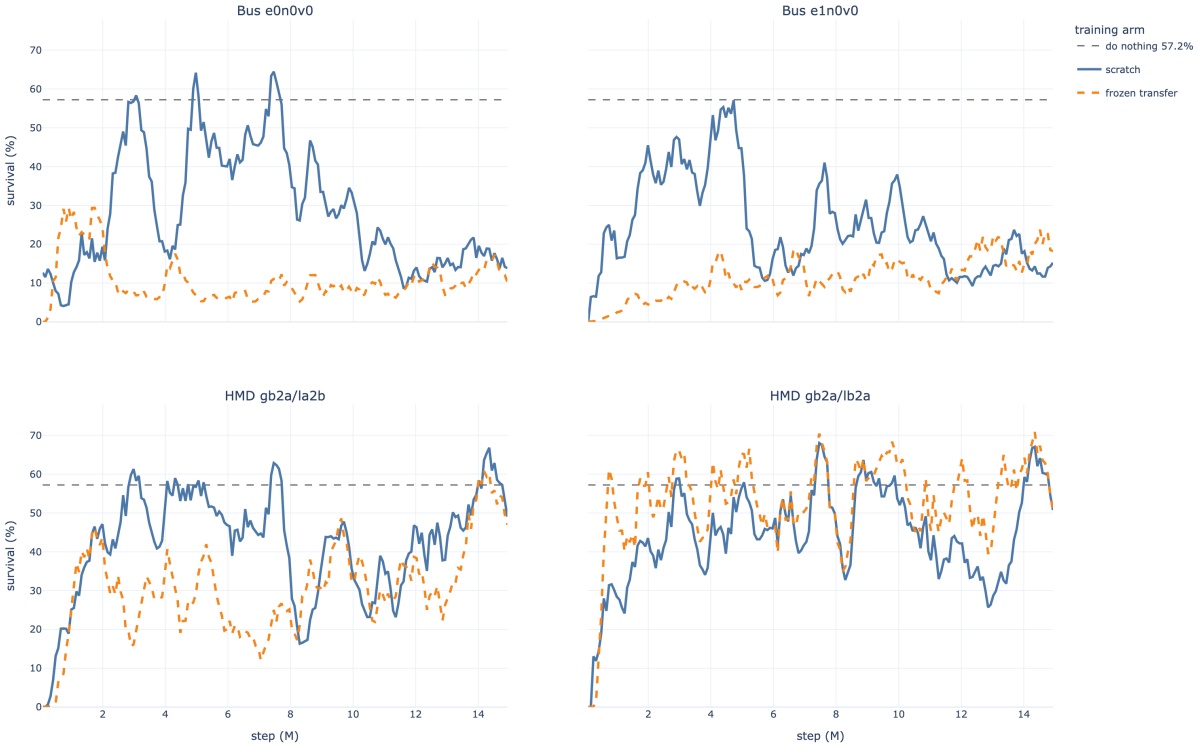


Figure 7.8: Test episodic survival during training, one panel per architecture, frozen transfer against training from scratch. The dashed line is the do-nothing reference at 57.2% on the training-time evaluation split.

No arm beats doing nothing. The best of the eight runs reaches 54.5% against 56.0% for a policy that never acts, and in Figure 7.8 the curves cross the reference only in transient peaks. Whatever the graph representation contributes on `bus14`, it does not yet produce a useful controller here, and every comparison below is between policies that are all worse than inaction.

Freezing is neutral on the disaggregated schema and destructive on the bus schema. The two heterogeneous architectures match their scratch counterparts to within 0.3 points, so the transferred encoder neither helps nor obstructs. The two bus architectures lose 17.1 and 37.4 points. This is the deficit Section 7.7 warned would be confounded, and the confound is real: with uncorrected inputs a frozen `bus14` encoder is reading displaced power and angle channels, so the loss cannot be attributed to the representation failing to generalize. The asymmetry between the schemas is the part worth pursuing. In the bus graph every node carries the power and angle channels, because generator and load values are aggregated onto their busbar, so a frozen encoder meets the miscalibrated quantities on every node it reads. In the disaggregated graph those channels are non-zero only on asset nodes, which are a minority of the node set, while busbars carry cooldown and connectivity, and `rho` rides on the edges where Section 7.6 measured it as already comparable across grids. That is a hypothesis this experiment does not test, but it is the one the corrected runs will separate.

The bus architectures are also the unstable ones. Their end-of-training survival, 10% and 18% on the smoothed curve, is far below their own best checkpoints at 47.6% and 51.3%: both peak early and decay for the rest of training. The disaggregated runs end near 47% and 51%, close to their best checkpoints, so their endpoint numbers describe a policy that still exists at the end of training rather than one that was passed through. The gap is worth keeping in view whenever a best-checkpoint statistic is quoted for this environment.

The `bus14` ordering partly survives. The two disaggregated architectures finish above the two bus ones, and `gb2a_lb2a`, the direction pair that won the message-direction screen, is the best of the four here as well. That is consistent with Chapter 5 without being evidence for it: one seed per cell and 2.1 points between the two disaggregated architectures cannot separate them.

What is still open. One seed per condition, uncorrected inputs, and an encoder frozen for all 15M steps rather than unfrozen after a warm start, which tests reuse without adaptation and not the only useful form of transfer. The first thing the corrected runs have to establish is not the transfer question at all, but whether any arm can clear 56%.

7.11 What this chapter establishes

Independently of how the two arms eventually compare, three findings hold.

- **The encoder is portable; its inputs are not.** Parameter shapes cross grids unchanged once the grid-shaped buffers are dropped. What does not cross is the distribution those parameters were calibrated on, and nothing in the training configuration was correcting it.
- **Per-feature standardization can be actively harmful when a channel is a mixture.** Standardizing a channel that is 85 % structural zeros aligns its moments and misplaces its most frequent value, degrading every channel it touches. This is not specific to power grids: it applies to any graph representation in which a node feature exists only for nodes carrying a particular kind of asset.
- **For gauge-dependent quantities, representation beats normalization.** The single largest mismatch measured here was an artifact of the voltage-angle reference. No choice of scaling addresses it, whereas expressing the angle as a difference across each line removes it by construction and makes the channel dense at the same time.

Appendix A

GINE and GAT Graph-Actor Architectures

This appendix specifies the complete edge-aware actor used in the graph experiments. GINE and GAT receive the same graph inputs and use the same pooling and action head. Their only architectural difference is the message-passing operator: GINE inserts an edge embedding into the message content, whereas GAT uses an edge embedding to decide how strongly the message is weighted.

A.1 Shared input and encoder pipeline

For agent a, the encoder receives the local graph $\mathcal{G}_t^{(a)}$. Candidate edges with `edge_mask`$= 0$ are removed before message passing, so the layer sees only the active directed set $\mathcal{E}_t^{(a)}$. The source of $u \to v$ sends a message and v aggregates it. Node and edge masks do not become learned features.

Each raw node vector $\mathbf{x}_v$ first passes through

$$\mathbf{h}_v^{(0)} = \mathrm{LayerNorm}(\mathrm{ReLU}(\mathbf{W}_x \mathbf{x}_v + \mathbf{b}_x)), \tag{A.1}$$

which produces a 128-dimensional node state. Learned global node-identifier embeddings and the optional external edge pre-encoder are disabled. Every GINE or GAT layer nevertheless contains its own trainable projection of the edge vector to the dimension required by that layer.

Two message-passing layers are used. After each layer, the implementation applies ReLU followed by LayerNorm:

$$\mathbf{h}_v^{(k+1)} = \mathrm{LayerNorm}\Big(\mathrm{ReLU}(\widetilde{\mathbf{h}}_v^{(k+1)})\Big). \tag{A.2}$$

The same encoder parameters are shared by all agents. Each agent nevertheless passes its own local graph, with its own node set, active edges, and one-hop context, through that shared encoder.

A.2 GINE message passing

For an active edge $u \to v$, layer k projects the complete edge feature vector and adds it to the sender state:

$$\widetilde{\mathbf{e}}_{uv}^{(k)} = \mathbf{W}_e^{(k)} \mathbf{e}_{uv} + \mathbf{b}_e^{(k)}, \tag{A.3}$$

$$\mathbf{m}_{u\to v}^{(k)} = \mathrm{ReLU}\Big(\mathbf{h}_u^{(k)} + \widetilde{\mathbf{e}}_{uv}^{(k)}\Big), \tag{A.4}$$

$$\widetilde{\mathbf{h}}_v^{(k+1)} = \mathrm{MLP}^{(k)}\left((1+\epsilon)\mathbf{h}_v^{(k)} + \sum_{u:(u,v)\in\mathcal{E}_t} \mathbf{m}_{u\to v}^{(k)}\right). \tag{A.5}$$

The implementation uses the PyTorch Geometric default $\epsilon = 0$ without a trainable epsilon parameter. Each internal GINE MLP is Linear–ReLU–Linear and outputs 128 features. Equation A.5 shows why

edge attributes are part of the message content. Two identical node states can send different messages when one edge is an overloaded physical line and the other is an asset-attachment relation. Likewise, reversing a typed heterogeneous edge activates a different relation-one-hot column and can produce a different message.

GINE uses an unweighted sum over incoming active edges. It does not learn a softmax competition between neighbors. The edge embedding changes what is sent; the number of incoming messages and their vector sum determine the aggregate.

A.3 GAT message passing

GAT first computes a score for every active incoming edge and attention head. For head r, the implementation has the form

$$s_{uv}^{(k,r)} = \mathrm{LeakyReLU}\Big(\mathbf{a}_{s,r}^{\top}\mathbf{W}_{s,r}\mathbf{h}_u^{(k)} + \mathbf{a}_{d,r}^{\top}\mathbf{W}_{d,r}\mathbf{h}_v^{(k)} + \mathbf{a}_{e,r}^{\top}\mathbf{W}_{e,r}\mathbf{e}_{uv}\Big), \tag{A.6}$$

$$\alpha_{uv}^{(k,r)} = \frac{\exp s_{uv}^{(k,r)}}{\sum_{w:(w,v)\in\mathcal{E}_t} \exp s_{wv}^{(k,r)}}, \tag{A.7}$$

$$\widetilde{\mathbf{h}}_v^{(k+1)} = \frac{1}{H}\sum_{r=1}^{H}\sum_{u:(u,v)\in\mathcal{E}_t} \alpha_{uv}^{(k,r)}\mathbf{W}_r\mathbf{h}_u^{(k)}. \tag{A.8}$$

GAT uses $H = 4$ heads and `concat=false`, so the four head outputs are averaged and the layer still returns 128 features. The edge embedding affects α_{uv}: it changes how much attention the receiver assigns to the sender. In the standard GAT operator used here, the edge vector is not added to the value vector $\mathbf{W}_r\mathbf{h}_u$. This is the central difference from GINE.

GAT internally adds a self-loop for every node. That computational self-loop is separate from any explicit self edge in the graph schema, which these representations do not create.

Table A.1: Exact architectural difference between the implemented GINE and GAT actors.

Component	GINE	GAT
Edge use	Added to the sender state inside every message	Enters the attention score for every incoming edge
Neighbor aggregation	Unweighted sum	Softmax-weighted sum
Self information	Explicit $(1+\epsilon)\mathbf{h}_v$ term	Internal GAT self-loop
Multiple heads	Not used	Four heads, averaged rather than concatenated
Post-layer operation	ReLU, then LayerNorm	ReLU, then LayerNorm
Pooling and action head	Shared implementation	Shared implementation

A.4 Pooling and graph output

With mean pooling, the graph representation after the second message-passing layer is

$$\overline{\mathbf{h}}_G = \frac{\sum_v m_v\mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \tag{A.9}$$

where m_v is the dynamic visibility mask. This is one mean over all visible node types, not a separate mean per type. Therefore:

- the busbar graph pools controlled busbars and only the far-end busbars currently attached by live boundary lines;
- the direct heterogeneous graph additionally pools controlled generators and loads, but never neighboring private assets;

- the explicit-line graph additionally pools transmission-line nodes and contains no neighboring equipment.

An inactive contextual candidate contributes neither to the numerator nor to the denominator. The ordinary GINE/GAT mean readout includes visible context; `controlled_mean` replaces m_v by the product of the visibility and controlled-node masks. Visible context can still influence controlled nodes through message passing, but it is not aggregated directly. Sum and maximum pooling, including their `controlled_*` variants, change only the readout and do not change message passing or the information boundary.

The pooled 128-dimensional vector then passes through the shared graph output projection

$$\mathbf{z}_a = \mathrm{ReLU}(\mathbf{W}_o \overline{\mathbf{h}}_G + \mathbf{b}_o), \qquad \mathbf{z}_a \in \mathbb{R}^{128}. \tag{A.10}$$

When virtual-node readout is selected, the final virtual-node state replaces $\overline{\mathbf{h}}_G$; the output projection and its dimension stay the same.

A.5 Agent-specific action head

The flat observation is not concatenated to $\mathbf{z}_a$. Each agent has its own two-layer MLP action head:

$$\mathbf{q}_a^{(1)} = \mathrm{ReLU}(\mathbf{W}_a^{(1)} \mathbf{z}_a + \mathbf{b}_a^{(1)}), \tag{A.11}$$

$$\mathbf{q}_a^{(2)} = \mathrm{ReLU}(\mathbf{W}_a^{(2)} \mathbf{q}_a^{(1)} + \mathbf{b}_a^{(2)}), \tag{A.12}$$

$$\boldsymbol{\ell}_a = \mathbf{W}_a^{(3)} \mathbf{q}_a^{(2)} + \mathbf{b}_a^{(3)}, \qquad \boldsymbol{\ell}_a \in \mathbb{R}^{|\mathcal{A}_a|}, \tag{A.13}$$

$$\pi_a(\cdot \mid \mathcal{G}_t^{(a)}) = \mathrm{Categorical}(\mathrm{logits} = \boldsymbol{\ell}_a). \tag{A.14}$$

Both hidden layers contain 128 units. During training, an action is sampled from this categorical distribution. Deterministic evaluation selects $\arg\max_j \ell_{a,j}$. Action zero is the local do-nothing action.

Only the GNN encoder and graph output projection are shared across agents. The action heads are agent-specific because agents can have different local action sets and therefore different output widths $|\mathcal{A}_a|$. The critic is a separate MLP over the centralized flat state and does not reuse the actor GINE/GAT embedding.

Equation A.14 is the default head. The optional candidate-action head of Section 4.5 keeps this pipeline up to $\mathbf{z}_a$ and additionally consumes the post-message-passing node states $\mathbf{h}_v^{(K)}$, which are the same states that Equation A.9 averages.

A.6 Exact node inputs by representation

All node types of one representation receive the same feature columns. Columns not listed for a node type are zero. Type identity reaches GINE and GAT through the `is_*` columns. The inventory below describes visible nodes. For an inactive contextual candidate, the complete row—including type and role columns—is zero, all incident edges are inactive, and the node is excluded from readout.

Table A.2: Node types and nonzero input features received by GINE and GAT, under the default `gnn_angle_representation = node`. Under `edge_diff` the `gen_theta`, `load_theta` and `theta` entries leave the node rows in every representation; the busbar and disaggregated graphs move the angle drop onto the physical line edge, while the explicit-line graph writes it on the transmission-line row as `theta_diff` (Section 4.1.2).

Representation	Node type	Populated input features
`bus`	Busbar	`gen_p`, `gen_theta`, `load_p`, `load_theta`, `time_before_cooldown_sub`, `domain_mask`
`heterogeneous`	Busbar	`is_busbar`, `time_before_cooldown_sub`, `domain_mask`
	Generator	`is_generator`, `p`=`gen_p`, `theta`=`gen_theta`, `domain_mask`
	Load	`is_load`, `p`=`load_p`, `theta`=`load_theta`, `domain_mask`
`heterogeneous_line`	Busbar	`is_busbar`, `time_before_cooldown_sub`, `domain_mask`
	Generator	`is_generator`, `p`=`gen_p`, `theta`=`gen_theta`, `domain_mask`
	Load	`is_load`, `p`=`load_p`, `theta`=`load_theta`, `domain_mask`
	Transmission line	`is_transmission_line`, `line_status`, `rho`, `timestep_overflow`, `time_before_cooldown_line`, optional maintenance times, `domain_mask`
Any augmented schema	Substation summary	Learned projection of the substation structural descriptor; no raw physical feature row
	Virtual node	One learned vector shared across graphs; no raw physical feature row

A.7 Exact edge inputs by representation

The edge vector contains all edge information consumed by GINE and GAT. Inactive candidates are filtered out before the vectors in Table A.3 reach a convolution.

Table A.3: Directed edge relations and edge-feature vectors received by GINE and GAT, under the default `gnn_angle_representation = node`. "Relation" denotes one active relation-one-hot column. Under `edge_diff` the physical-line block of the busbar and disaggregated graphs gains a `theta_diff` column; the explicit-line rows are unchanged, because there the drop is a line-node feature and the attachment edges stay free of physical channels.

Representation	Directed relation	Edge features
`bus`	Physical busbar ↔ busbar	`line_status`, `rho`, `timestep_overflow`, `time_before_cooldown_line`, optional maintenance times; no relation one-hot
	Optional self or same-substation	Same width; all physical values zero; no relation one-hot
`heterogeneous`	Physical busbar ↔ busbar	Complete measured physical-line block plus `relation_physical_line`
	Generator ↔ busbar	Neutral physical block plus direction-specific generator relation
	Load ↔ busbar	Neutral physical block plus direction-specific load relation
	Optional self or same-substation	Neutral physical block plus the corresponding relation
`heterogeneous_line`	Busbar ↔ line node	One origin/extremity-and-direction relation; no line-state attributes
	Generator ↔ busbar	One direction-specific generator relation
	Load ↔ busbar	One direction-specific load relation
	Optional self or same-substation	One corresponding relation
Any augmented schema	Busbar → summary/virtual, with optional reverse	Zero edge vector; the relation carries no flow

The complete relation names and activation rules are given in Tables 4.7 and 4.9. Those tables distinguish, for example, generator-to-busbar from busbar-to-generator and origin from extremity line attachments. These distinctions matter directly to both edge-aware models: GINE changes message content and GAT changes attention.

Appendix B

Graph Construction Details

This appendix collects the construction and sizing details of the three graph schemas of Section 4.2: how large each candidate graph is, how inactive candidates are neutralized, and how a local actor graph is cut out of the full grid. None of it changes the representations themselves.

B.1 Candidate graph sizes

Let S be the number of included substations, B the busbars per substation, L the included physical lines, and N_g, N_d the generators and loads. Write $d_g, d_d, d_\ell \in \{1, 2\}$ for the number of retained directions of the generator, load, and line-node attachments: 2 for `bidirectional` and 1 for either one-way mode. Let P_ℓ be the number of represented line endpoints: an internal line contributes two and a boundary line in a local explicit-line graph contributes only its controlled endpoint. Thus $L \le P_\ell \le 2L$. Table B.1 gives the size of the fixed candidate graph before any mask is applied.

Table B.1: Size of the candidate graph of each schema. Enabling self edges adds $|\mathcal{V}|$ directed edges and enabling same-substation edges adds $SB(B-1)$, in every schema.

Schema	Candidate nodes	Candidate directed edges
`bus`	SB	$2B^2L$
`heterogeneous`	$SB + N_g + N_d$	$2B^2L + B(d_gN_g + d_dN_d)$
`heterogeneous_line`	$SB + N_g + N_d + L$	$Bd_\ell P_\ell + B(d_gN_g + d_dN_d)$

Three consequences are worth naming. A busbar graph with both optional relations enabled therefore has $2B^2L + SB + SB(B-1)$ candidate directed edges. The disaggregated schema adds $N_g + N_d$ nodes and between $B(N_g + N_d)$ and $2B(N_g + N_d)$ asset candidates to the busbar graph, and buys individual asset states with them. For a complete state graph, $P_\ell = 2L$, so the explicit-line schema replaces the $2B^2L$ busbar-to-busbar term by $2Bd_\ell L$: identical at $B = 2$ and cheaper for $B > 2$. A local boundary line has candidates only at its controlled endpoint, which reduces P_ℓ and prevents construction of a far-end attachment. The schema adds L line nodes and one message-passing hop for internal endpoint exchange.

The hierarchical augmentations of Section 4.3 add on top of any of these: one substation-summary node per substation, with SB directed edges for `toward_summary` or $2SB$ for `bidirectional`; and one virtual node, with N_{bus} or $2N_{\text{bus}}$ edges, or S or $2S$ when summary nodes are also enabled.

B.2 Indexing and masking conventions

Grid2Op numbers busbars from one in its topology vector; those identifiers are converted to the zero-based index b used in Equation 4.4.

An inactive candidate edge keeps its row in the edge-feature tensor, so the tensor shape is fixed for an episode, but the row is zero-filled and its `edge_mask` entry is 0. Masked edges are removed before the

encoder is called, so those zero rows never generate a message. This is what allows a topology change to alter connectivity without rebuilding the graph. An inactive contextual node likewise retains its candidate row, but its complete feature vector is zero and its `node_mask` entry is 0. Every incident physical or structural edge is deactivated. The row therefore participates in neither message passing nor graph readout; a mean readout excludes it from both numerator and denominator.

B.3 Local actor graphs

A local actor graph always contains the controlled substations and every line touching them. Additional membership is schema-specific. The `bus` graph contains only the far-end busbar currently attached by each live boundary line, and that busbar exposes aggregated neighboring power and angle. The direct `heterogeneous` graph contains the same far-end busbar, but never its generator or load nodes. The `heterogeneous_line` graph represents the shared line itself and therefore constructs no neighboring busbar or private asset and no far-end attachment edge. The centralized state graph is intentionally unrestricted for centralized training. Ordinary pooling aggregates all visible nodes, whereas `controlled_*` pooling restricts readout to action-domain nodes; neither choice changes the construction boundary.

B.4 Boundary verification

The regression suite checks that contextual visibility follows a live busbar attachment, moves when the attachment is switched, and disappears after a line outage; controlled nodes remain present; structural relations cannot make a hidden candidate visible; inactive rows are zero; neighboring private assets are absent from direct heterogeneous graphs; and all neighboring equipment is absent from explicit-line local graphs. Candidate-action tests additionally verify that a boundary line remains addressable through its line node without adding far-end equipment. Controlled-readout tests verify both that the pooled set is fixed and that gradients can still reach visible context through message passing.

Appendix C

Experiment Configurations

This appendix provides the complete run configurations for the GINE encoder and heavy GINE ablations.

Table C.1: GINE rerun configuration summary. The reference run is `Heavy GINE Baseline`.

Run	Actor/critic encoder	GNN hidden/layers	Actor/critic heads	Concat flat	Edge pre-enc.	Entropy	Init action-0	Opt. critic	Difference from `Heavy GINE Baseline`
`Heavy GINE Baseline`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.7	false	Baseline: heavy concat-flat GINE actor, GNN critic, and legacy critic updates
`Light GINE (Concat, MLP Critic)`	`gnn` / `mlp`	64 / 1	2×128 / 2×128	true	false	0.02	0.7	true	Light GINE actor, MLP critic, smaller heads, no edge pre-encoder, optimized critic updates
`Heavy GINE (No Bias)`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.0	false	Bias ablation: same as baseline, but the initial do-nothing prior is 0.0
`No Bias, Opt Critic`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.0	true	Bias 0.0 plus optimized critic updates; otherwise heavy concat-flat GINE with GNN critic
`Optimized Heavy GINE`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.01	0.0	true	Optimized heavy GINE: bias 0.0, lower entropy 0.01, optimized critic updates
`Optimized Light GINE`	`gnn` / `mlp`	64 / 1	2×128 / 2×128	false	false	0.01	0.0	true	Light optimized GINE: MLP critic, no concat-flat, lower entropy 0.01, bias 0.0, optimized critic updates

Table C.2: Heavy-GINE parameter comparison from `configs/gine_s0_s1_s2`.

Run	Main change from baseline	GNN	GNN size	Critic	Actor/critic heads	Concat	Shared	Edge pre.	Node ID	Entropy	Init action-0	Opt. critic
`No Flat Concat`	Removes flat-observation concatenation	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	no	yes	yes	yes	0.02–0.02	0.7	no
`Heavy GINE Baseline`	Reference: heavy concat-flat GINE with GNN critic	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Opt. Critic Updates`	Enables optimized critic updates	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	yes
`MLP Critic`	Replaces the GNN critic with an MLP critic	`gine`	2 × 128	`mlp`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Unshared Actor`	Uses non-shared actor GNN weights	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	no	yes	yes	0.02–0.02	0.7	no
`Light GINE Encoder`	Uses a lighter GINE encoder	`gine`	1 × 64	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Zero Entropy Decay`	Decays entropy from 0.02 to 0.0	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.00	0.7	no
`No Node-ID Embs`	Removes learned node-ID embeddings	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	no	0.02–0.02	0.7	no
`GAT Message Passing`	Replaces GINE with GAT message passing	`gat`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Weighted GCN`	Replaces GINE with weighted GCN message passing	`gcn`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Light GINE (No Concat, MLP Critic)`	Light no-concat GINE with MLP critic and optimized critic updates	`gine`	1 × 64	`mlp`	2 × 128 / 2 × 128	no	yes	no	yes	0.02–0.02	0.7	yes
`Light GINE (Concat, MLP Critic)`	Light concat-flat GINE with MLP critic and optimized critic updates	`gine`	1 × 64	`mlp`	2 × 128 / 2 × 128	yes	yes	no	yes	0.02–0.02	0.7	yes
`No Initial Bias`	Removes the initial do-nothing bias	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.0	no

Appendix D

Full-Test Checkpoint Registry

Table D.1 reports the checkpoint step used for each full-test result table. The values are the `checkpoint_global_step` fields stored in the corresponding JSON files under `Topology_Task/outputs/full_test_eval`. The do-nothing policy has no learned checkpoint. For the behavioral cloning students, the stored step is the student checkpoint's optimizer-step count rather than a MAPPO environment step.

Table D.1: Checkpoint steps used for the full-test evaluations.

Run type	Seed 0 checkpoint step	Seed 1 checkpoint step	Seed 2 checkpoint step
Do-nothing policy	–	–	–
Baseline flat policy	13,600,000	13,040,000	13,760,000
Global rho heuristic, $\rho_{safe} = 0.90$	13,920,000	14,320,000	14,800,000
Local rho heuristic, $\rho_{safe} = 0.90$	14,160,000	14,400,000	14,160,000
Intervention gate, final-action MAP	12,000,000	12,720,000	13,760,000
Intervention gate, hierarchical greedy	7,120,000	14,720,000	13,760,000
Flat Sparse16, $\lambda_{fixed} = 0.000$	9,600,000	12,160,000	7,920,000
Flat Sparse16, $\lambda_{fixed} = 0.001$	10,640,000	11,520,000	11,600,000
Flat Sparse16, $\lambda_{fixed} = 0.003$	12,000,000	12,960,000	12,480,000
Flat Sparse16, $\lambda_{fixed} = 0.010$	8,960,000	5,760,000	12,560,000
Gated Sparse16, $\lambda_{fixed} = 0.000$	8,480,000	4,320,000	6,000,000
Gated Sparse16, $\lambda_{fixed} = 0.001$	5,760,000	7,600,000	3,280,000
Gated Sparse16, $\lambda_{fixed} = 0.003$	8,960,000	7,120,000	7,600,000
Gated Sparse16, $\lambda_{fixed} = 0.010$	12,000,000	7,840,000	7,920,000
Flat local-safe AIB, $d = 0.20$	13,120,000	9,360,000	12,160,000
Flat local-safe AIB, $d = 0.10$	12,880,000	13,200,000	12,720,000
Flat local-safe AIB, $d = 0.35$	12,800,000	5,360,000	11,520,000
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	14,960,000	5,280,000	16,800,000
Flat non-idle AIB, $d = 0.20$	12,480,000	14,880,000	11,920,000
Teacher: MAPPO with local rho heuristic	14,160,000	14,400,000	14,160,000
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	12,375	14,994	13,617
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	12,375	14,994	13,617

Bibliography

[1] E. B. Fisher, R. P. O'Neill, and M. C. Ferris, "Optimal transmission switching," *IEEE Transactions on Power Systems*, vol. 23, no. 3, pp. 1346–1355, 2008.

[2] A. Marot et al., "Learning to run a power network challenge for training topology controllers," *Electric Power Systems Research*, 2021.

[3] J. Viebahn, M. Naglic, A. Marot, B. Donnot, and S. H. Tindemans, "Potential and challenges of AI-powered decision support for short-term system operations," in *CIGRE Paris Session*, 2022.

[4] B. Donnot, "Grid2op: A testbed platform to model sequential decision making in power systems," *arXiv preprint arXiv:2011.07929*, 2020.

[5] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *International Conference on Machine Learning*, 1993.

[6] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, 2017.

[7] C. Yu et al., "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in Neural Information Processing Systems*, 2022.

[8] M. Hassouna, C. Holzhüter, P. Lytaev, J. Thomas, B. Sick, and C. Scholz, *Graph reinforcement learning for power grids: A comprehensive survey*, 2024. arXiv: 2407.04522. [Online]. Available: https://arxiv.org/abs/2407.04522

[9] M. Subramanian, J. Viebahn, S. H. Tindemans, B. Donnot, and A. Marot, "Exploring grid topology reconfiguration using a simple deep reinforcement learning approach," in *IEEE Madrid PowerTech*, 2021.

[10] D. Yoon, S. Hong, B.-J. Lee, and K.-E. Kim, "Winning the L2RPN challenge: Power grid management via semi-Markov afterstate actor-critic," in *International Conference on Learning Representations*, 2021.

[11] A. Chauhan, M. Baranwal, and A. Basumatary, "PowRL: A reinforcement learning framework for robust management of power networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 14 757–14 764.

[12] M. Lehna, J. Viebahn, A. Marot, S. Tomforde, and C. Scholz, "Managing power grids through topology actions: A comparative study between advanced rule-based and reinforcement learning agents," *Energy and AI*, vol. 14, p. 100 276, 2023.

[13] B. Manczak, J. Viebahn, and H. van Hoof, *Hierarchical reinforcement learning for power network topology control*, 2023. arXiv: 2311.02129. [Online]. Available: https://arxiv.org/abs/2311.02129

[14] E. van der Sar, A. Zocca, and S. Bhulai, *Multi-agent reinforcement learning for power grid topology optimization*, 2023. arXiv: 2310.02605. [Online]. Available: https://arxiv.org/abs/2310.02605

[15] B. de Mol, D. Barbieri, J. Viebahn, and D. Grossi, "Centrally coordinated multi-agent reinforcement learning for power grid topology control," in *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems (e-Energy)*, 2025.

[16] E. Marchesini et al., "MARL2Grid-TR: A multi-agent RL benchmark in power grid operations," in *International Conference on Learning Representations*, 2026.

[17] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *International Conference on Learning Representations*, 2017.

[18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations*, 2018.

[19] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" *International Conference on Learning Representations*, 2019.

[20] M. de Jong, J. Viebahn, and Y. Shapovalova, *Generalizable graph neural networks for robust power grid topology control*, 2025. arXiv: 2501.07186. [Online]. Available: https://arxiv.org/abs/2501.07186

[21] E. Marchesini et al., *RL2Grid: Benchmarking reinforcement learning in power grid operations*, 2025. arXiv: 2503.23101. [Online]. Available: https://arxiv.org/abs/2503.23101

[22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[23] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," *International Conference on Machine Learning*, 2017. [Online]. Available: https://arxiv.org/abs/1705.10528

[24] A. Stooke, J. Achiam, and P. Abbeel, "Responsive safety in reinforcement learning by PID lagrangian methods," *International Conference on Machine Learning*, 2020. [Online]. Available: https://arxiv.org/abs/2007.03964

[25] N. Carrara, E. Leurent, R. Laroche, T. Urvoy, and O.-A. Maillard, "Budgeted reinforcement learning in continuous state space," *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://arxiv.org/abs/1903.01004

[26] L. Lautenbacher et al., *Multi-objective reinforcement learning for power grid topology control*, 2025. arXiv: 2502.00040. [Online]. Available: https://arxiv.org/abs/2502.00040

[27] W. Hu et al., "Strategies for pre-training graph neural networks," in *International Conference on Learning Representations*, 2020.